Programming your GPU with OpenMP
A Hands-on Introduction

Tim Mattson
University of Bristol

While we wait to get started, setup an account on the system we'll use for our GPUs

To get an account on our cluster (AMD AAC6 HPC cluster), open this URL:
https://tinyurl.com/OpenMPAMDSC2026

This web-form will ask for name, email address, and SSH public key.
Your SSH public key is on your laptop (usually in ~/.ssh/id_rsa.pub

Or scan this code to fill the form on your phone:
(first copy your SSH public key to your phone so you can paste it)

Course Materials: git clone https://github.com/tgmattso/OMP_SC.git

Acknowledgement: Much of this content was created with Tom Deakin and Simon McIntosh-Smith of the University of Bristol

Preliminaries: Part 1

- Disclosures
  - The views expressed in this tutorial are those of the people delivering the tutorial.
    - We are not speaking for our employers.
    - We are not speaking for the OpenMP Architecture Review Board

- We take these tutorials VERY seriously:
  - Help us improve … tell us how you would make this tutorial better.

Nothing in these slides say ANYTHING about future products from anyone.

Data presented in these slides were NOT generated in a formal benchmarking effort. Hence, performance numbers MUST NOT be used for comparing systems.

Course Materials: git clone https://github.com/tgmattso/OMP_SC.git

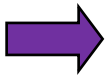
Preliminaries: Part 2

- Our plan ... Active learning!
  - We will mix short lectures with short exercises.
  - The HPC team at AMD is donating time on one of their clusters for this tutorial.

- Please follow these simple rules
  - Do the exercises that we assign and then change things around and experiment.
    - Embrace active learning!
  - Don't cheat: Do Not look at the solutions before you complete an exercise … even if you get really frustrated.

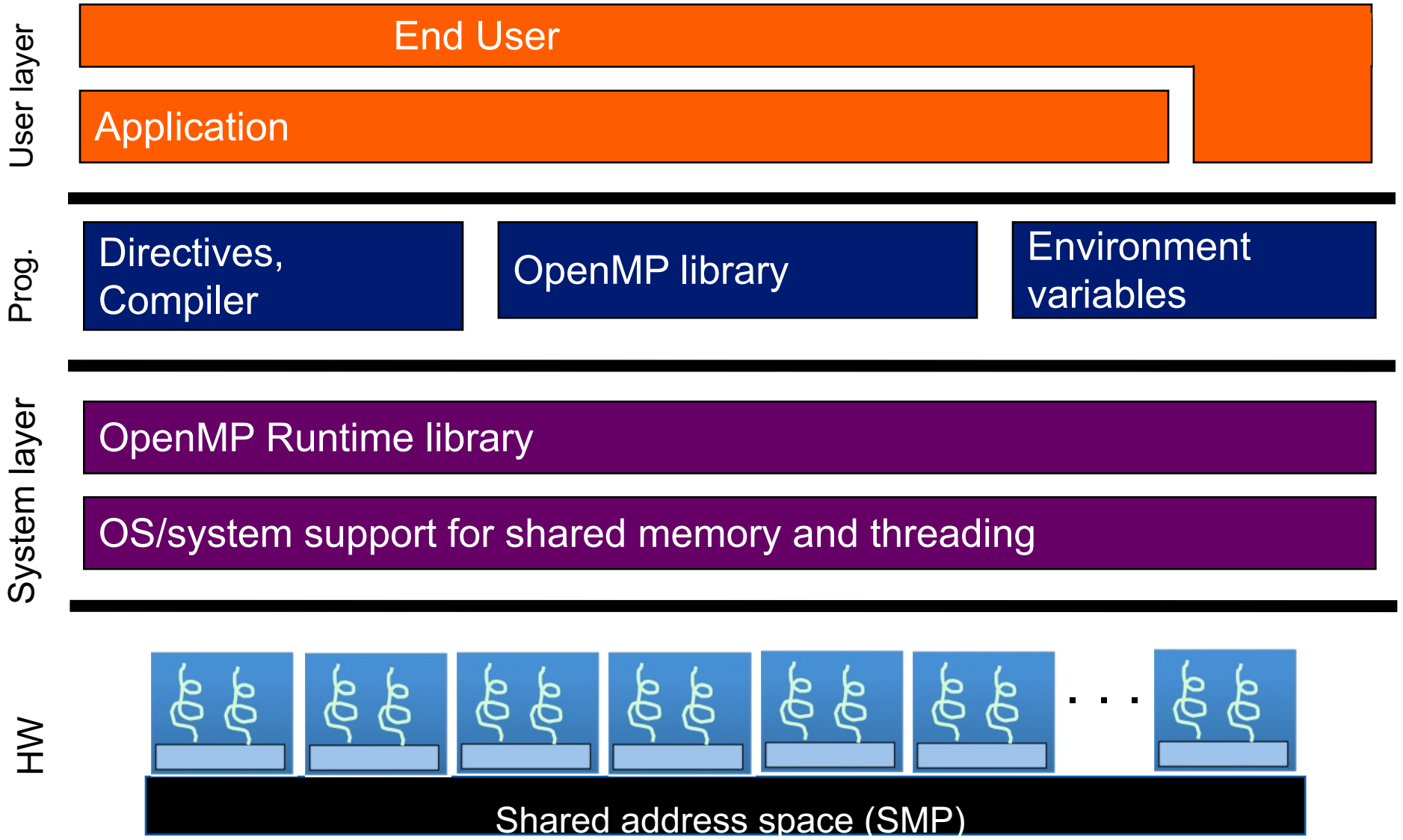
Course Materials: git clone https://github.com/tgmattso/OMP_SC.git

Outline



- Introduction to OpenMP
- OpenMP and GPU Programming
- Managing memory on the Host and the GPU
- Fundamental principles of GPU program optimization
- Explicit control of how code maps onto a GPU
- More about Host/GPU memory management
- Using team-local memory: Blocked Matrix Multiplication
- This is the end: Summary/Conclusion
- Extra Content

OpenMP Basic Definitions: Basic Solution Stack

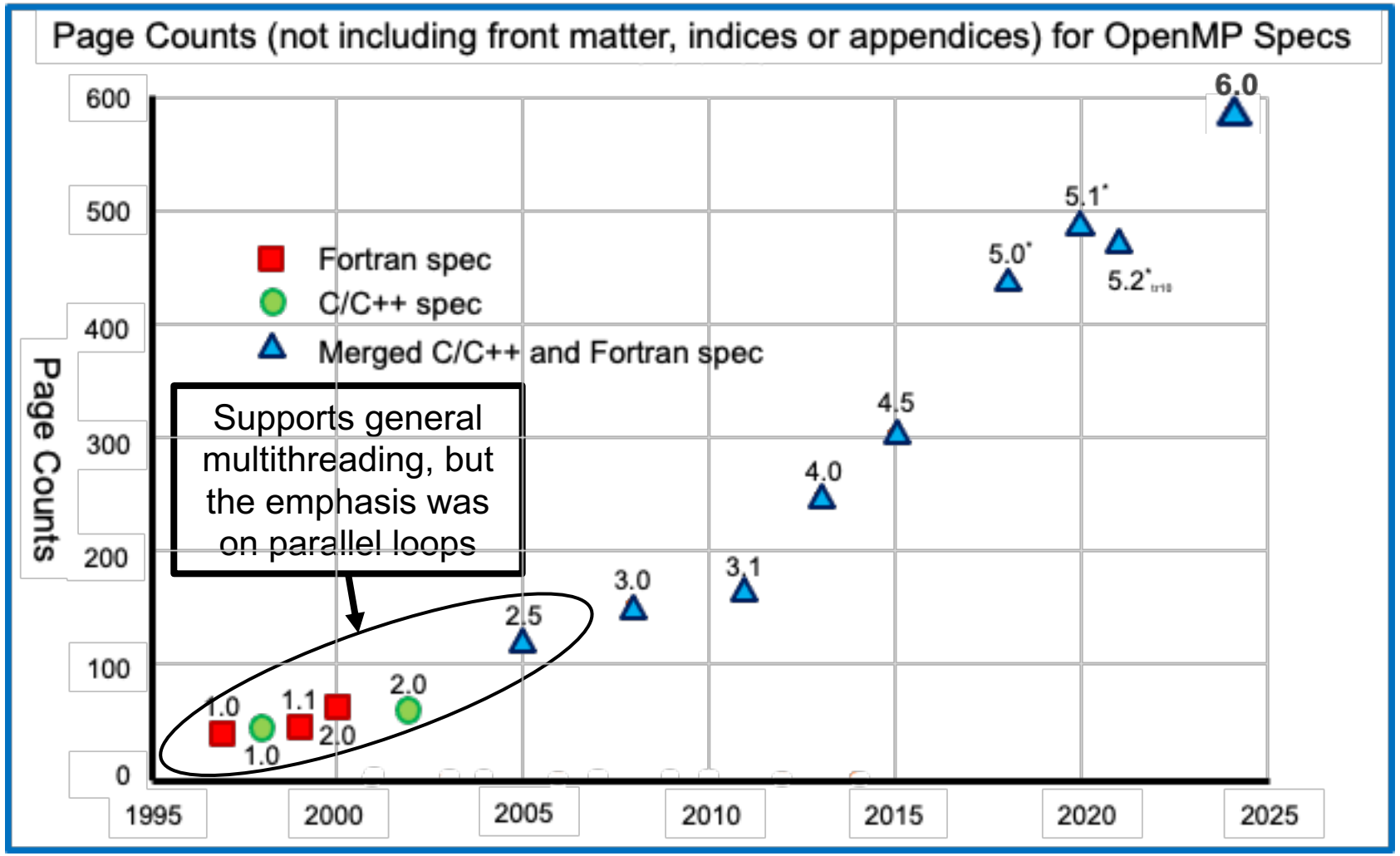


OpenMP started with a focus on CPUs as Symmetric Multiprocessors ….
i.e., lots of threads with “equal cost access” to memory

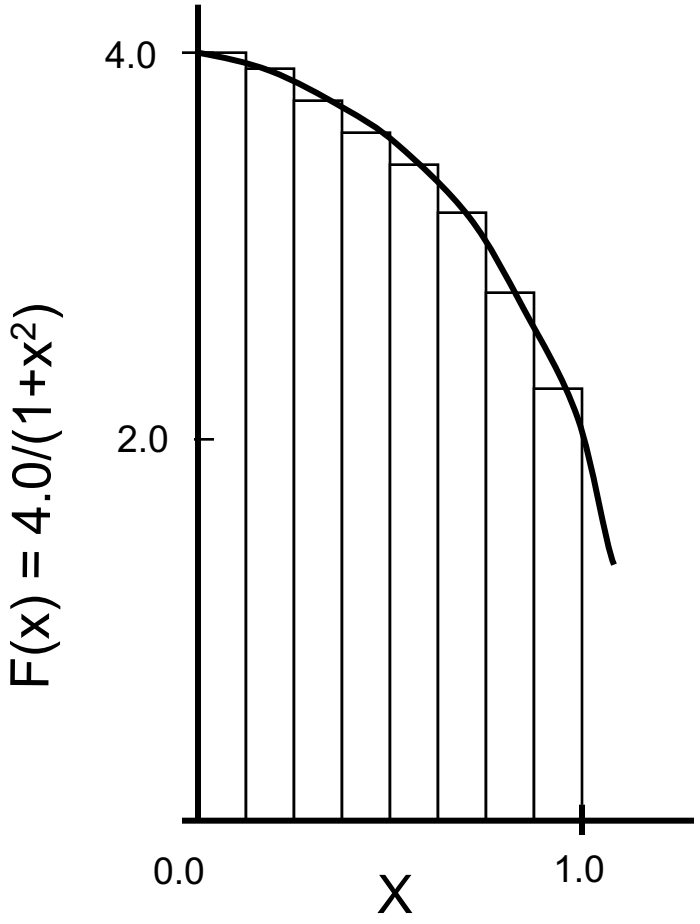
The growth of complexity in OpenMP

- OpenMP started out in 1997 as a simple interface for the application programmers more versed in their area of science than computer science.

- The complexity has grown considerably over the years!



Numerical integration: the Pi program



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)}\,dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^{N} F(x_i)\Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i.

Serial Pi program

```
static long num_steps = 100000;
double step;
int main ()
{           int i;      double x, pi, sum = 0.0;

            step = 1.0/(double) num_steps;

            for (i=0; i< num_steps; i++){
                        x = (i+0.5)*step;
                        sum = sum + 4.0/(1.0+x*x);
            }
            pi = step * sum;
}
```

Example: Pi in OpenMP with a loop & reduction

```
#include <omp.h>
static long num_steps = 100000;  double step;
void main ()
{    int i;           double x, pi, sum = 0.0;
     step = 1.0/(double) num_steps;

#pragma omp parallel for private(x) reduction(+:sum)
     for (i=0; i< num_steps; i++){
        x = (i+0.5)*step;
        sum = sum + 4.0/(1.0+x*x);
     }

     pi = step * sum;
}
```

Include file to hold function prototypes, types and other items needed to support OpenMP

Create a team of threads (parallel)… share the work of the for loop between the threads (for).

Note … the loop index (i) of the parallel loop is made local to each thread.

Create a variable local to each thread for the value of x at the center of each interval

Create a copy of the variable, sum, for each thread. Initialize it with the identity of the reduction operator (+ is the operator, 0 is the identity)

Threads wait here until all threads complete their work (a barrier).

Finish the reduction … combine sum from each thread using the reduction operator (+). Then combine with the "original" value of sum using the reduction operator.

Example: combining loops to create more work for the parallel loop.

Matrix multiplication: C = C + A * B where $C \in R^{N,M}$, $A \in R^{N,P}$, $B \in R^{P,M}$

```
void mm_par(int N, int M, int P, float *A, float *B, float *C)
{

    #pragma omp parallel for collapse(2)
    for (int i=0;  i<N;  i++)
       for (int j=0;  j<M;  j++)
          for(int k=0; k<P; k++)
               *(C+(i*M+j)) += *(A+(i*P+k)) *  *(B+(k*M+j));
}
```

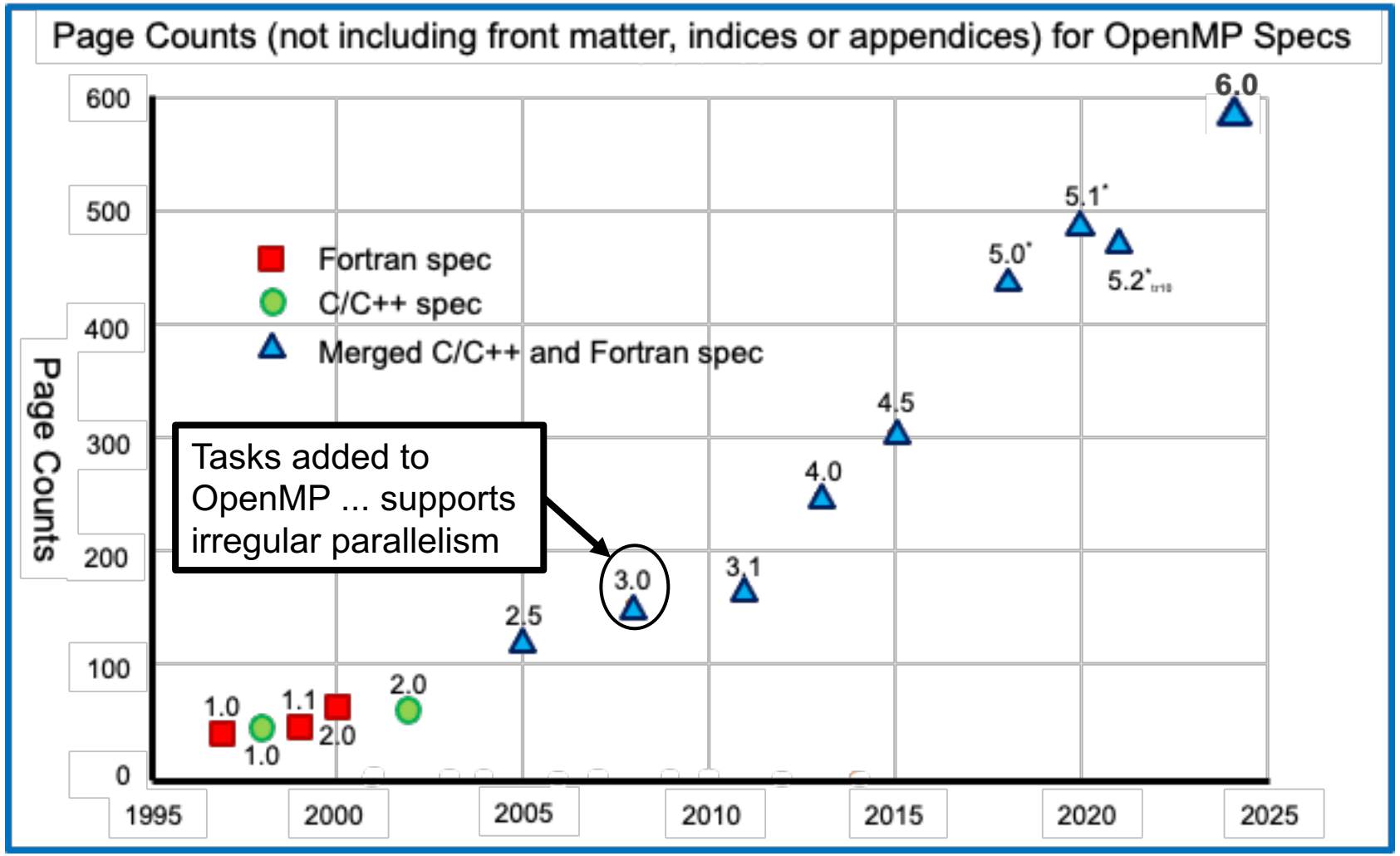
If N and M are not large, you may not have enough work to keep all the threads busy.

collapse(2) will collapse the following 2 nested loops into one larger loop … therefore giving the parallel loop more work to do.

The growth of complexity in OpenMP

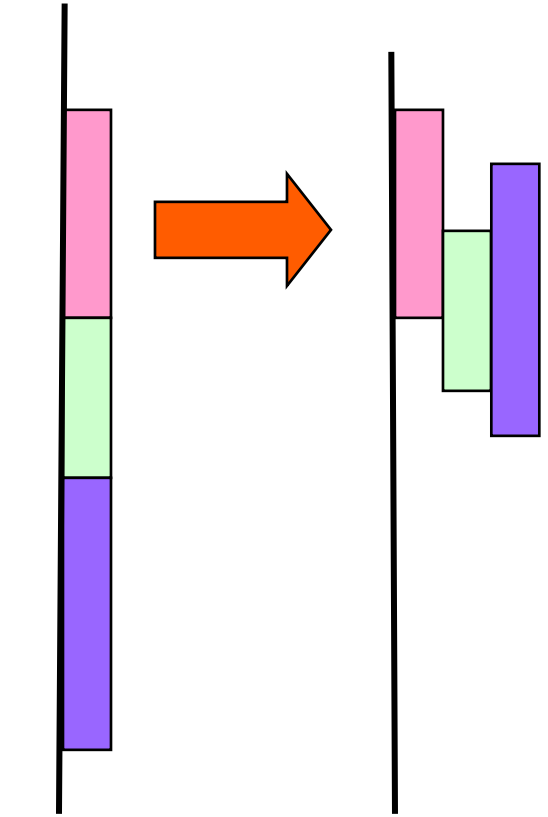
- OpenMP started out in 1997 as a simple interface for the application programmers more versed in their area of science than computer science.

- The complexity has grown considerably over the years!



What are Tasks?

- Tasks are independent units of work
- Tasks are composed of:
  - code to execute
  - data to compute with



- You fill a queue with tasks and then a team of threads does the work … Supports irregular workflows.

```
#pragma omp parallel
{
  #pragma omp single
   {
    p = listhead ;
    while (p) {
       #pragma omp task firstprivate(p)
       {
         process (p);
       }
       p=next (p) ;
     }
   }
}
```

Create a team of threads

One thread walks the list. The others wait at the barrier at the end of the single construct

Create a task, a private copy of p for each task, and assign p the value of p when the task is created (i.e. firstprivate)

An implied barrier at the end of the single construct. All tasks complete before continuing past the barrier

Outline

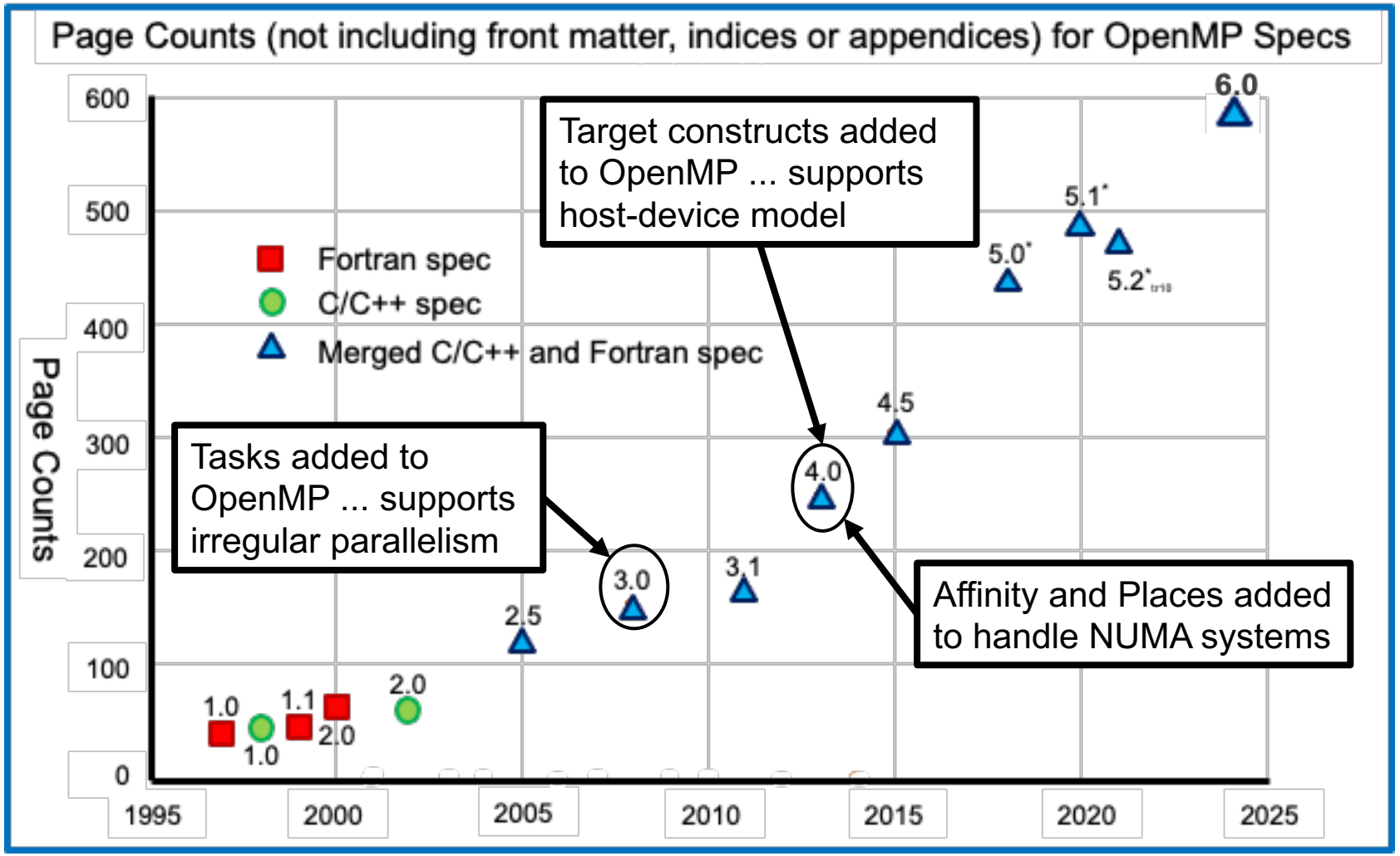


- Introduction to OpenMP
- OpenMP and GPU Programming
- Managing memory on the Host and the GPU
- Fundamental principles of GPU program optimization
- Explicit control of how code maps onto a GPU
- More about Host/GPU memory management
- Using team-local memory: Blocked Matrix Multiplication
- This is the end: Summary/Conclusion
- Extra Content

The growth of complexity in OpenMP

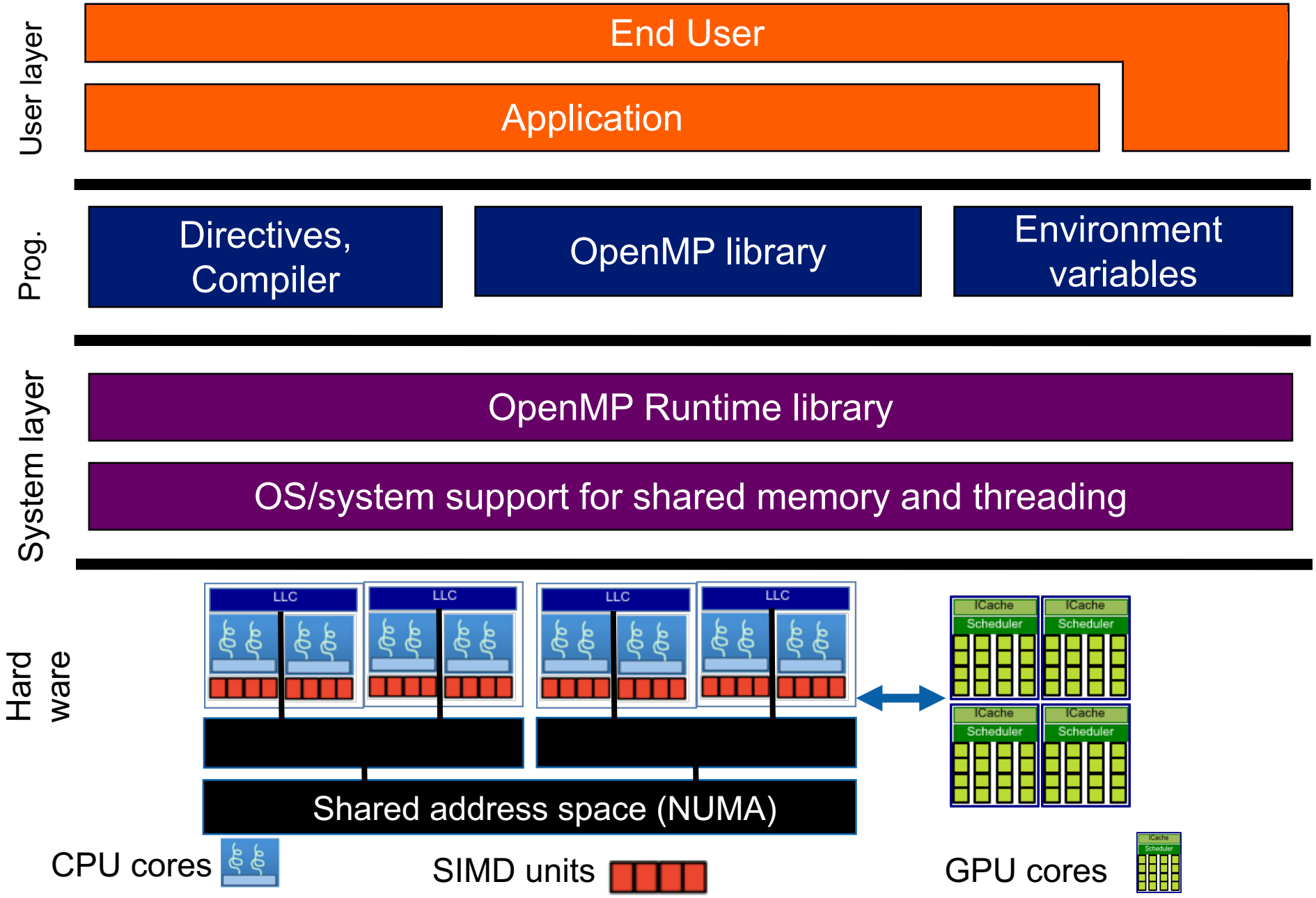
- OpenMP started out in 1997 as a simple interface for the application programmers more versed in their area of science than computer science.

- The complexity has grown considerably over the years!



NUMA: non-uniform memory architecture

OpenMP basic definitions: the solution stack



Why is OpenMP so important?

OpenMP is the **only** model with **full** GPU support from **all** vendors for C/C++ and Fortran

	CUDA*		HIP		SYCL		OpenACC		OpenMP			Standard		Kokkos		ALPAKA	
	C	F	C	F	C	F	C	F	C	F	P	C	F	C	F	C	F
NVIDIA	● 1	● 2	◒ 3	★ / 4	▲ 5	/ 6	● 7	● 8	■ ▲ 9	■ ● 10	▲ 48	● 11	● 12	▲ 13	★ 14	▲ 15	/ 16
AMD	◒ ★ 19	★ 20	● 21	★ / 4	▲ 22	/ 6	▲ 23	▲ ★ 24	● ▲ 25	● 26	?	▲ ■ ★ 27	■ 28	▲ 29	★ 14	▲ 30	/ 16
Intel	◒ ▲ 33	/ 34	▲ 35	/ 36	● 37	/ 6	★ 38	★ 39	● 40	● 41	★ 48	● ■ 42	● 43	▲ 44	★ 14	▲ 45	/ 16

Data Current as of Feb 11, 2025

- ● Full vendor support
- ◒ Indirect, but comprehensive support, by vendor
- ■ Vendor support, but not (yet) entirely comprehensive
- ▲ Comprehensive support, but not by vendor
- ★ Limited, probably indirect support -- but at least some
- / No direct support available, but of course one could ISO-C-bind your way through it or directly link the libraries
- C C++ (sometimes also C)
- F Fortran
- P Python

***While not shown in the table, CUDA and HIP support GPU programming in Python through CuPy**

Table updated Feb 11, 2025. The numbers in the table refer to notes about each listed option. The Table is available at the following URL: https://x-dev.pages.jsc.fz-juelich.de/models/
Table from https://doi.org/10.1145/3624062.3624178 "Many Cores, Many Models: GPU Programming Model vs. Vendor compatibility Overview", Andreas Herten.
The table was modified by adding the python column for OpenMP (ref: **48**, PyOMP, https://ieeexplore.ieee.org/document/9658236)

The “BIG idea” Behind GPU programming

Traditional Loop based vector addition (vadd)

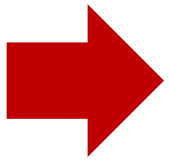
```
int main() {
   int N = . . . ;
   float *a, *b, *c;

   a* =(float *) malloc(N * sizeof(float));

   // ... allocate other arrays (b and c)
   // and fill with data

   for (int i=0;i<N; i++)
      c[i] = a[i] + b[i];

}
```



Data Parallel vadd with HIP

```
// Compute sum of length-N vectors: C = A + B
void __global__
vecAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) c[i] = a[i] + b[i];
}

int main () {
    int N = ... ;
    float *a, *b, *c;
    hipMalloc (&a,  sizeof(float) * N);
  // ... allocate other arrays (b and c)
  // and fill with data

  // Use thread blocks with 256 threads each
    vecAdd <<< (N+255)/256, 256 >>> (a, b, c, N);
}
```

Assume a GPU with unified shared memory … allocate on host, visible on device too

How do we execute code on a GPU: The SIMT model (Single Instruction Multiple Thread)

1. Turn kernel code into a scalar work-item

```
// Compute sum of order-N matrices: C = A + B
__global__ void
matAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < N && j<N) c[i][j] == a[i][j] + b[i][j];
}

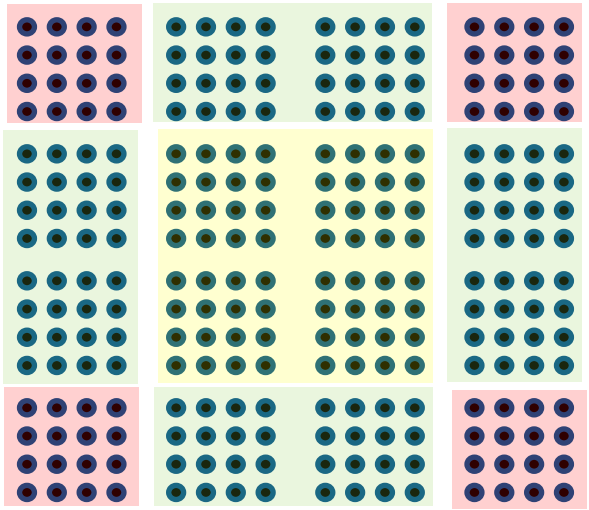
int main () {
    int N = ... ;
    float *a, *b, *c;
    hipMalloc (&a,  sizeof(float) * N);
  // ... allocate other arrays (b and c)
  // and fill with data

  // define threadBlocks and the Grid
   dim3 dimBlock(4,4);
   dim3 dimGrid(4,4);

  // Launch kernel on Grid
    matAdd <<< digGrid,dimBlock>>> (a, b, c, N);
}
```

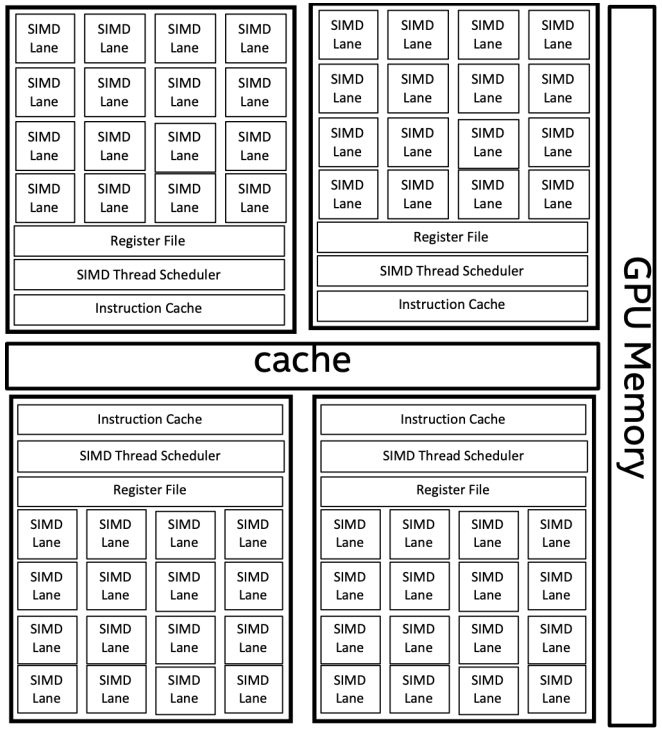
This is HIP code

2. Map work-items onto an N dim index space.



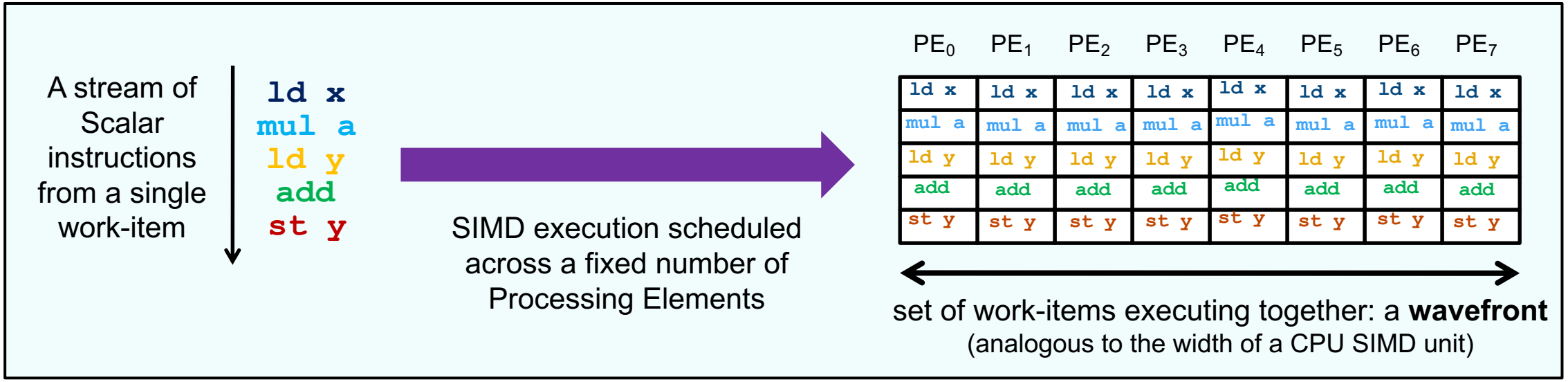
3. Map data structures onto the same index space

4. Run on GPU hardware designed around the SIMT execution model



SIMT: One instruction stream maps onto many Processing Elements

- SIMT model: Individual scalar instruction streams are grouped together for SIMD execution on hardware



GPU nomenclature is really messed up. (sorry about that … we tried to unify around OpenCL but failed).

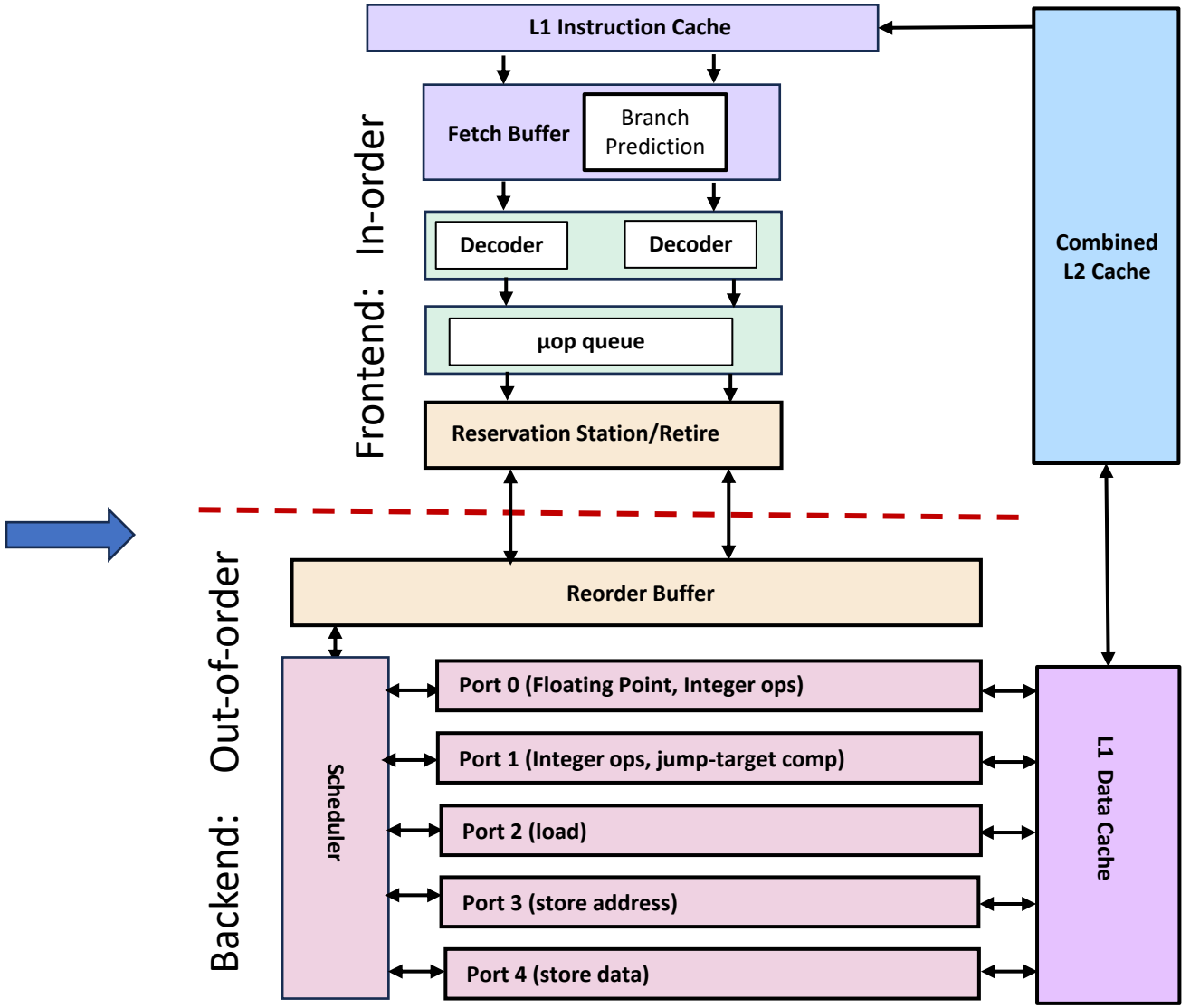
Instruction stream at finest grain	**work-item**, CUDA Thread
Blocks for scheduling work-items	**work-group**, thread block
Execution width for work-items	Subgroup, warp, **wavefront**
Finest grained processing element (PE) in a GPU	**SIMD Lane**, Processing Element, CUDA Core
Block of PEs driven by a single Instruction sequencer	multithreaded SIMD processor, **Compute Unit**, Streaming multiprocessor

These names are particularly awful since they conflict with established names from CPU Computing.

AMD terminology, in bold font, tracks the OpenCL terminology (which is great!!)

What is a core?

Core 0



Frontend processes instructions for execution.

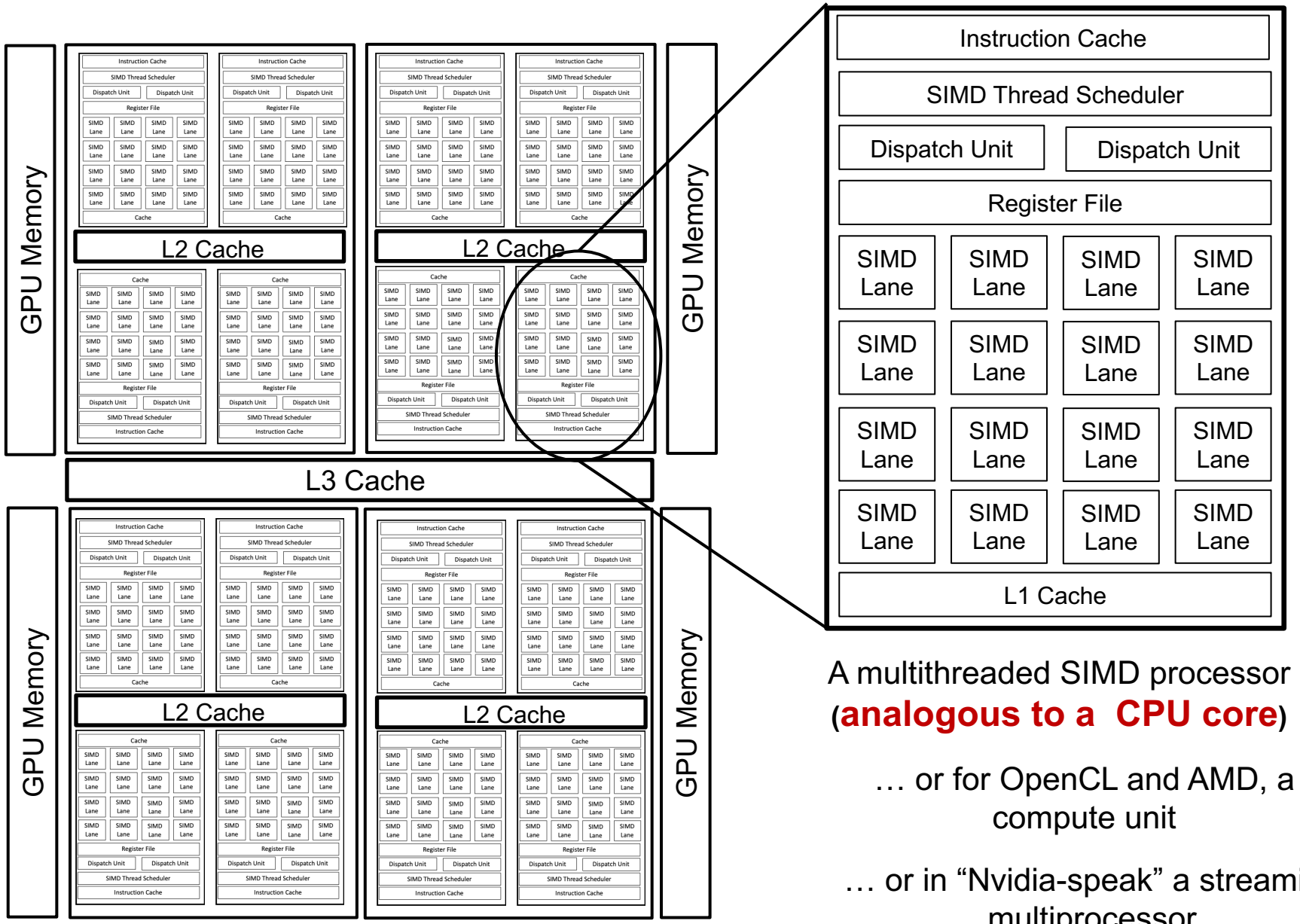
Maps them into micro-operations (**µop**) and manages them (and the register file) so they retire "in order"

Backend executes decoded instructions.

Instructions are pipelined and include multiple ports and SIMD execution units.

Execution is "out of order" to give the scheduler maximum flexibility to maximize instruction-level parallelism

A Generic GPU (following Hennessey and Patterson)



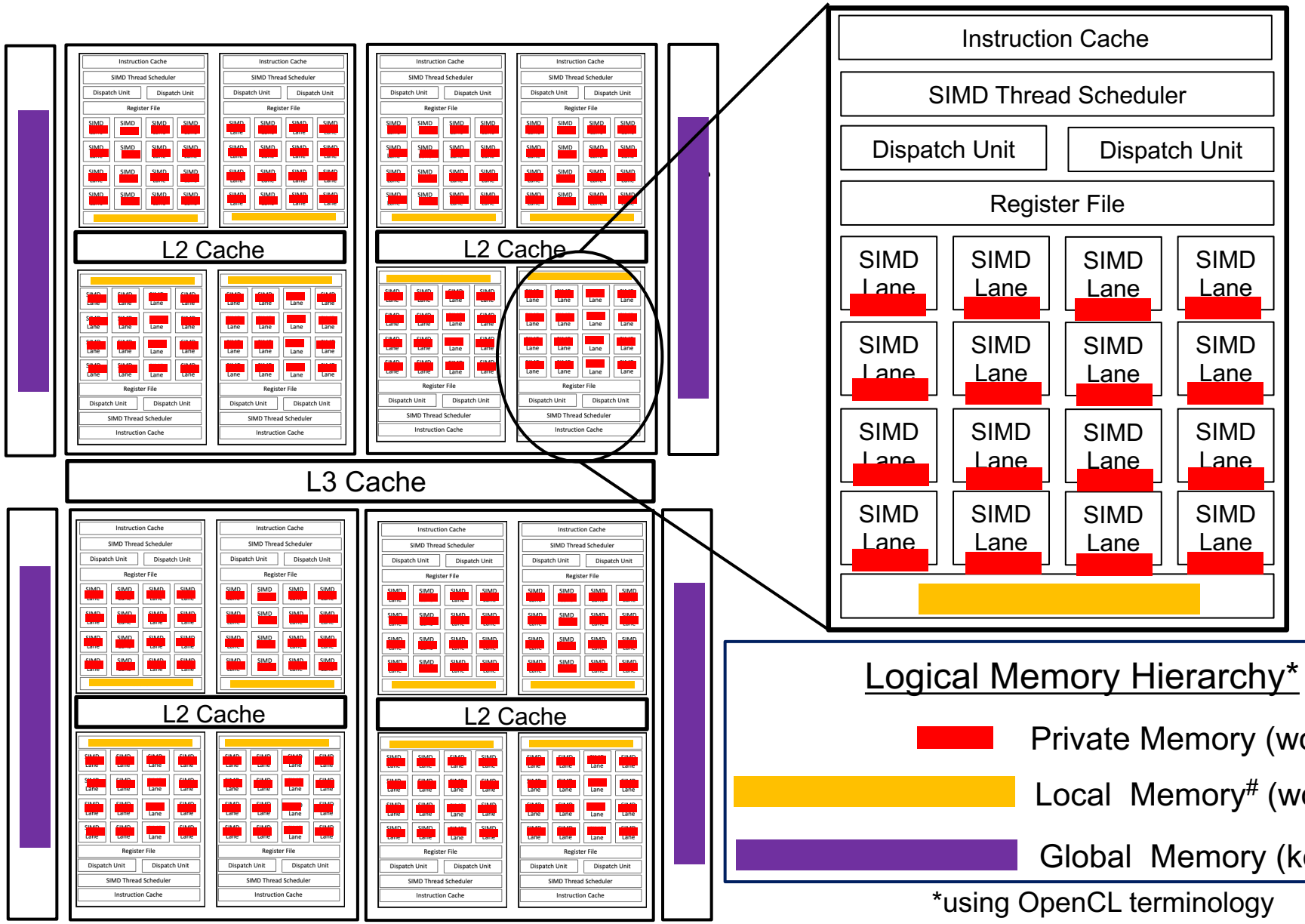
A multithreaded SIMD processor (**analogous to a CPU core**)

… or for OpenCL and AMD, a compute unit

… or in "Nvidia-speak" a streaming multiprocessor

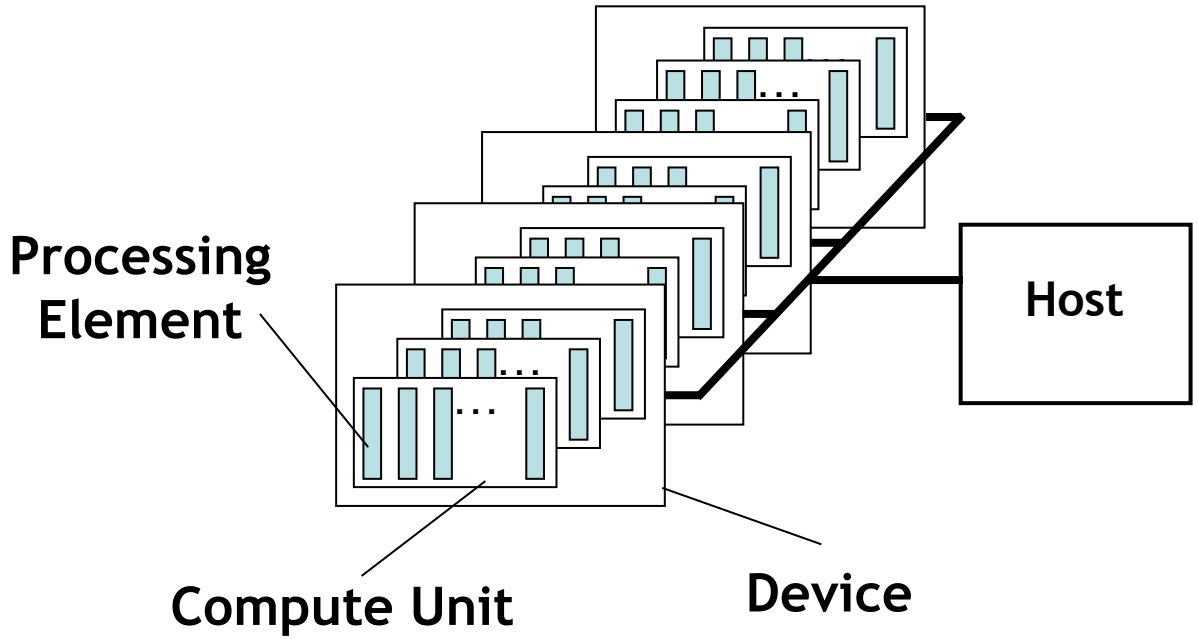
Computer Architecture: A Quantitative Approach, John L. Hennessy and David A. Patterson.

A Generic GPU (following Hennessey and Patterson)



Computer Architecture: A Quantitative Approach, John L. Hennessy and David A. Patterson.

A Generic Host/Device Platform Model*

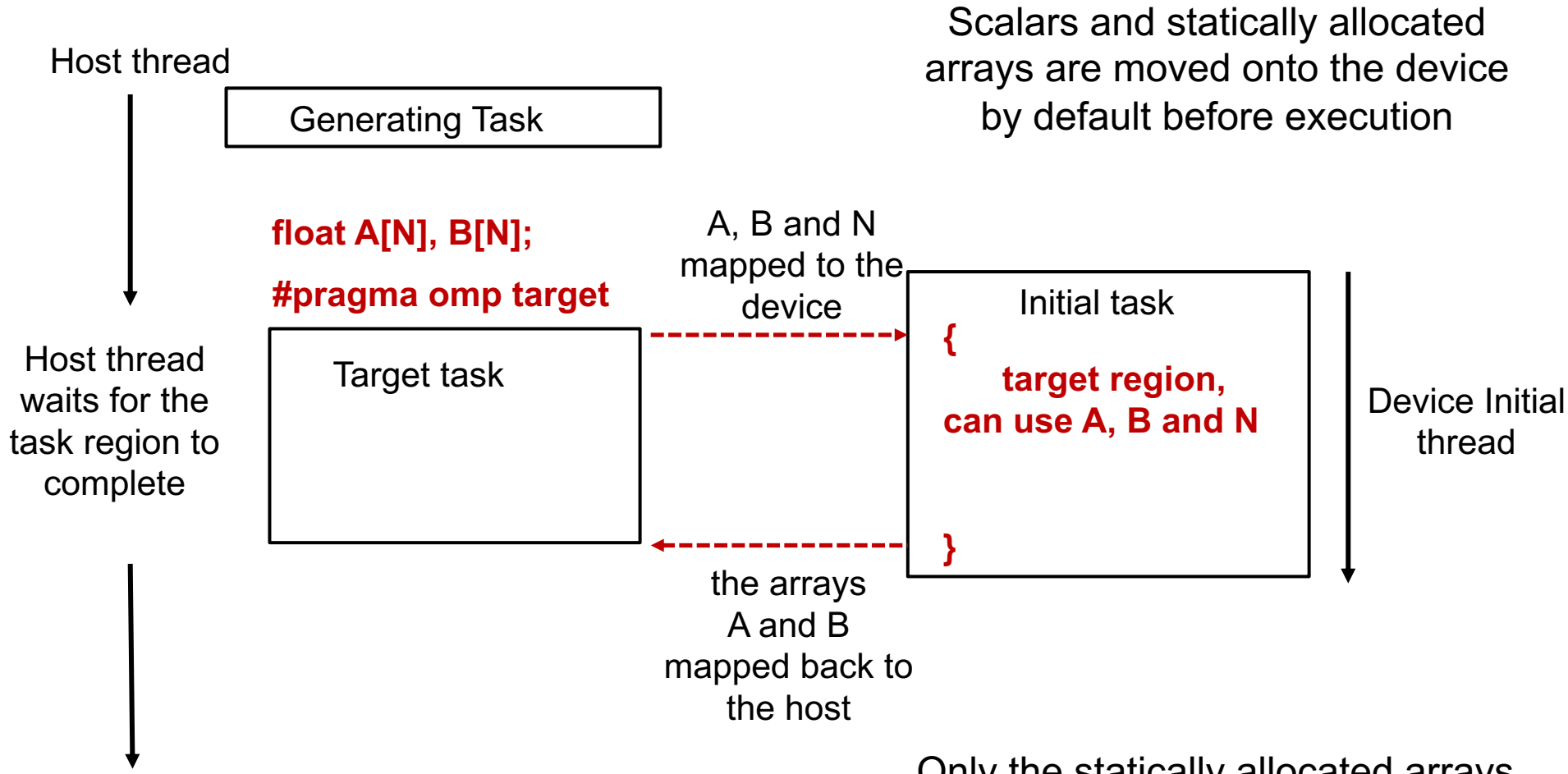


- One ***Host*** and one or more ***Devices***
  - Each Device is composed of one or more Compute Units
  - Each Compute Unit is divided into one or more ***Processing Elements***

- Memory divided into ***host memory*** and ***device memory***

*using OpenCL terminology

Running code on the GPU:
The target construct and default data movement



Based on figure 6.4 in **Using OpenMP – The Next Step** by van der Pas, Stotzer and Terboven, MIT Press, 2017

Default Data Sharing: example

```
int main(void) {
  int N = 1024;
  double A[N], B[N];

  #pragma omp target
  {

    for (int ii = 0; ii < N; ++ii) {

      A[ii] = A[ii] + B[ii];

    }

  } // end of target region
}
```

1. Variables created in host memory.

2. Scalar **N** and stack arrays **A** and **B** are copied *to* device memory. Execution transferred to device.

3. **ii** is **private** on the device as it's declared within the target region

4. Execution on the device.

5. stack arrays **A** and **B** are copied *from* device memory back to the host. Host resumes execution.

Now let's run code in parallel on the device

```
int main(void) {
  int N = 1024;
  double A[N], B[N];

  #pragma omp target
  {
    #pragma omp loop
    for (int ii = 0; ii < N; ++ii) {

      A[ii] = A[ii] + B[ii];

    }

  } // end of target region
}
```

The loop construct tells the compiler:

"this loop will execute correctly if the loop iterations run in any order. You can safely run them concurrently. And the loop-body doesn't contain any OpenMP constructs. So do whatever you can to make the code run fast"

The loop construct is a **declarative** construct. You tell the compiler what you want done but you DO NOT tell it how to "do it".

This is new for OpenMP .. And different implementations of OpenMP handle it differently.

Now let's run code in parallel on the device

```
int main(void) {
  int N = 1024;
  double A[N], B[N];

  #pragma omp target
  {
    #pragma omp teams loop
    for (int ii = 0; ii < N; ++ii) {

      A[ii] = A[ii] + B[ii];

    }

  } // end of target region
}
```

The loop construct tells the compiler:

"this loop will execute correctly if the loop iterations run in any order. You can safely run them concurrently. And the loop-body doesn't contain any OpenMP constructs. So do whatever you can to make the code run fast"

Some systems will run the loop construct on the encountering thread alone… which, when the loop construct binds to the target construct, is the initial thread on the target device.

We'll explain the details later … but to force this to always spread the work out among the compute units on a GPU, you need to make this a combined construct with the **teams** construct

Let’s get ready for our hands-on exercises.

To do this, you’ll need accounts on our system and you’ll need to know how to compile and run code on the system

System for hands-on Exercises

- AMD has made their AAC6 cluster available to us.

- We will use their HIP (**H**eterogenous-compute **I**nterface for **P**ortability) programing environment

To get an account on our cluster (AMD AAC6 HPC cluster), open this URL:

https://tinyurl.com/OpenMPAMDSC2026

This web-form will ask for name, email address, and SSH public key.
Your SSH public key is on your laptop (usually in ~/.ssh/id_rsa.pub

Or scan this code to fill the form on your phone:
(first copy your SSH public key to your phone so you can paste it)

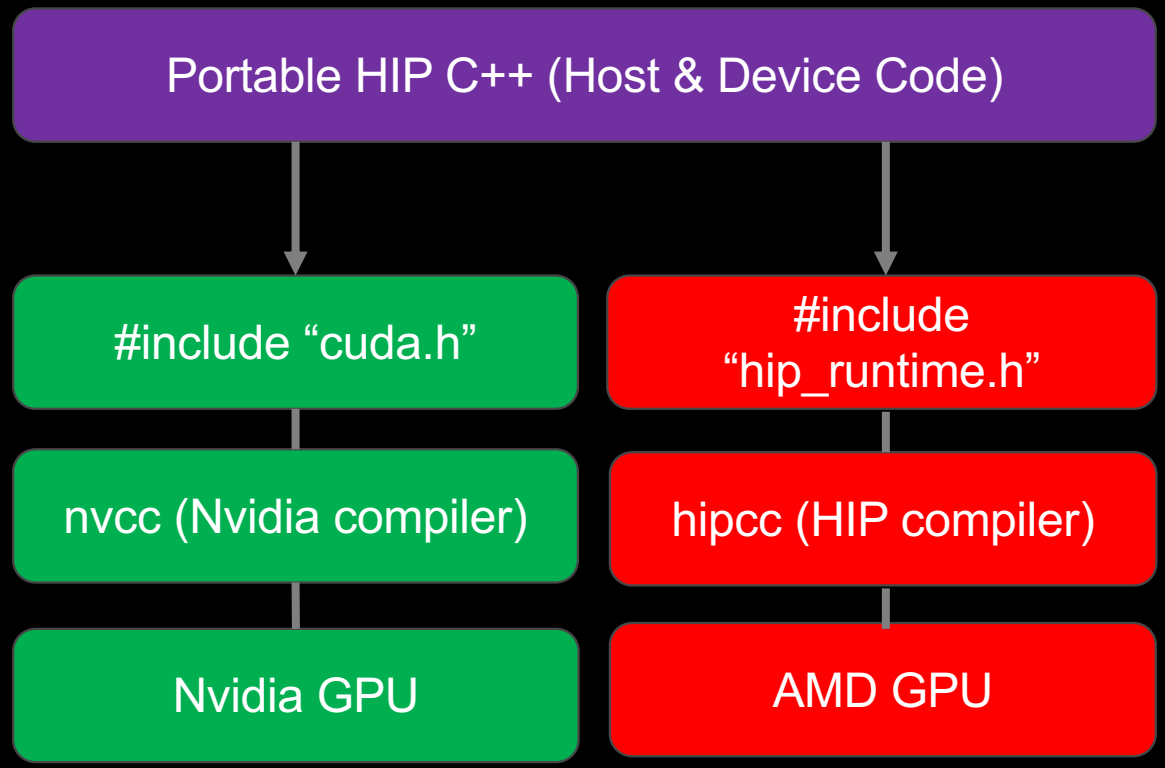
Course Materials: git clone https://github.com/tgmattso/OMP_SC.git

[Public]

What is HIP?

AMD's **H**eterogeneous-compute **I**nterface for **P**ortability, or **HIP**, is a C++ runtime API and kernel language that allows developers to create portable applications that can run on AMD accelerators as well as CUDA accelerators

- **Open-source**
- Syntactically similar to CUDA. Most CUDA API calls can be converted in place from CUDA to HIP.
- Supports a strong subset of CUDA runtime functionality



AMD
together we advance_

[Public]

AAC6

AAC6 is has pre-production and production parts

Login or Control Node

aac6:
aac6-fe2

You are here after logging in

SLURM

srun
salloc
sbatch

Queue PPAC_MI300A_CPX (Default)

ppac-pl1-s24-35

Queue MI300A_SPX

Queue PPAC_MI300A_SPX

ppac-pl1-s24-16

c-pl1-s24-26

pl1-s24-30

Queue SMC_MI300A_SPX

smc300a-ccs1-p03-28 • • •

Queue SH5_MI300A_CPX

sh5-pl1-s12-12

sh5-pl1-s12-33

Queue MI250

gct-ccs1-d21-03

gct-ccs1-d21-05 • • •

4xMI300A (CPX*)
24 logical GPUs
48 CPUs
128 GB memory

4xMI300A (SPX)
4 logical GPUs
192 CPUs
512 GB memory

1xMI300A (CPX*)
6 logical GPUs
48 CPUs
128 GB memory

MI300A CPU:
24 "Zen 4" cores (3 CCDs × 8)
Shares the 128 GB HBM3

* CPX compute partitioning exposes each XCD of the MI300A as single GPU device allowing to make more GPUs available for training purposes.
See: Deep dive into the MI300 compute and memory partition modes — ROCm Blogs

XCD: Accelerator Complex Die which contains 38 Active Compute Units each with 32 Kbytes L1 cache

AMD
together we advance_

Using the GPUs in the AAC6 cluster

- HIP (heterogeneous compute interface for portability) is designed to be a high-level portability layer between ROCm™ and CUDA. HIP/ROCm compilers are based on amdclang++ and LLVM.
- Compilers are named amdclang, amdclang++, and amdflang. Hipcc is basically amdclang --x hip
- HIP libraries are a wrapper on top of ROCm and CUDA libraries – interfaces are mostly compatible, but the underlying code for ROCm is customized for AMD GPUs.

Compile for For OpenMP

```
module load rocm amdclang
amdclang -fopenmp --offload-arch=gfx942  <prog_name>.c  -lm
```

Compile For HIP

```
module load rocm
hipcc --offload-arch=gfx942 <prog_name>.cpp  -lm
```

- The cluster is managed with slurm. Use an interactive session with salloc or prepare a batch-queue file and submit jobs with: sbatch, srun

```
salloc -N1 --gpus=1 -t 10:00 -p MI300A_CPX
```

Other options for partitions include:
[MI300A_SPX | PPAC_MI300A_SPX | SH5_MI300A_SPX | SH5_MI300A_CPX | MI250]

```
#!/bin/bash
#SBATCH -p MI300A_CPX
#SBATCH -N 1
#SBATCH --gpus=1
#SBATCH -t 10:00

module load rocm amdclang
./myprog
```

The CPX partition exposes the 4 physical GPUs as 24 distinct logical GPUs (XCDs*). Use this partition for code development and testing. The SPX partition exposes 4 distinct GPUs and is the partition to use for formal benchmarking

*XCD: Accelerator Complex Die … 6 XCD chiplets in an AMD Instinct™ MI300A GPU

We are now ready for our first exercise ... which is mostly a chance for you to get on the system and compile/run a program on a GPU.

To get an account on our cluster (AMD AAC6 HPC cluster), open this URL:

https://tinyurl.com/OpenMPAMDSC2026

This web-form will ask for name, email address, and SSH public key.
Your SSH public key is on your laptop (usually in ~/.ssh/id_rsa.pub

Or scan this code to fill the form on your phone:
(first copy your SSH public key to your phone so you can paste it)

Course Materials: git clone https://github.com/tgmattso/OMP_SC.git

Exercise: Parallel vector addition on a GPU

- Make a copy of the vadd.c program in OMP_GPU_Exercises
  - vadd.c Adds together two arrays, element by element:

```
for(i=0;i<N;i++) c[i]=a[i]+b[i];
```

- Parallelize your vadd program for a GPU.
  - module load rocm amdclang
  - amdclang –fopenmp --offload-arch=gfx942 vadd.c -lm
- If the system gave you a GPU that isn't gfx942, figure out which one to use with the command
  - rocminfo | grep gfx
- Time your vadd progam for large N and save the result. How does it compare to the CPU version?
  - double omp_get_wtime();
  - #pragma omp target
  - #pragma omp teams loop
  - sbatch gpu_job.sh
- Fetch an interactive session for a GPU node
  - salloc -N1 --gpus=1 -t 10:00 -p PPAC_MI300A_CPX

For tiny programs, OpenMP may opt to run the code on the host. You can force the OpenMP runtime to use the GPU by setting the OMP_TARGET_OFFLOAD environment variable when you run the program (./a.out)

```
> OMP_TARGET_OFFLOAD=MANDATORY ./a.out
```

gpu_job.sh

```
#!/bin/bash
#SBATCH -p MI300A_CPX
#SBATCH -N 1
#SBATCH --gpus=1
#SBATCH -t 10:00

module load rocm amdclang

OMP_TARGET_OFFLOAD=MANDATORY ./a.out
```

git clone https://github.com/tgmattso/OMP_SC.git

Solution: Simple vector add in OpenMP on GPU

```
int main()
{
    float a[N], b[N], c[N], res[N];
    int err=0;

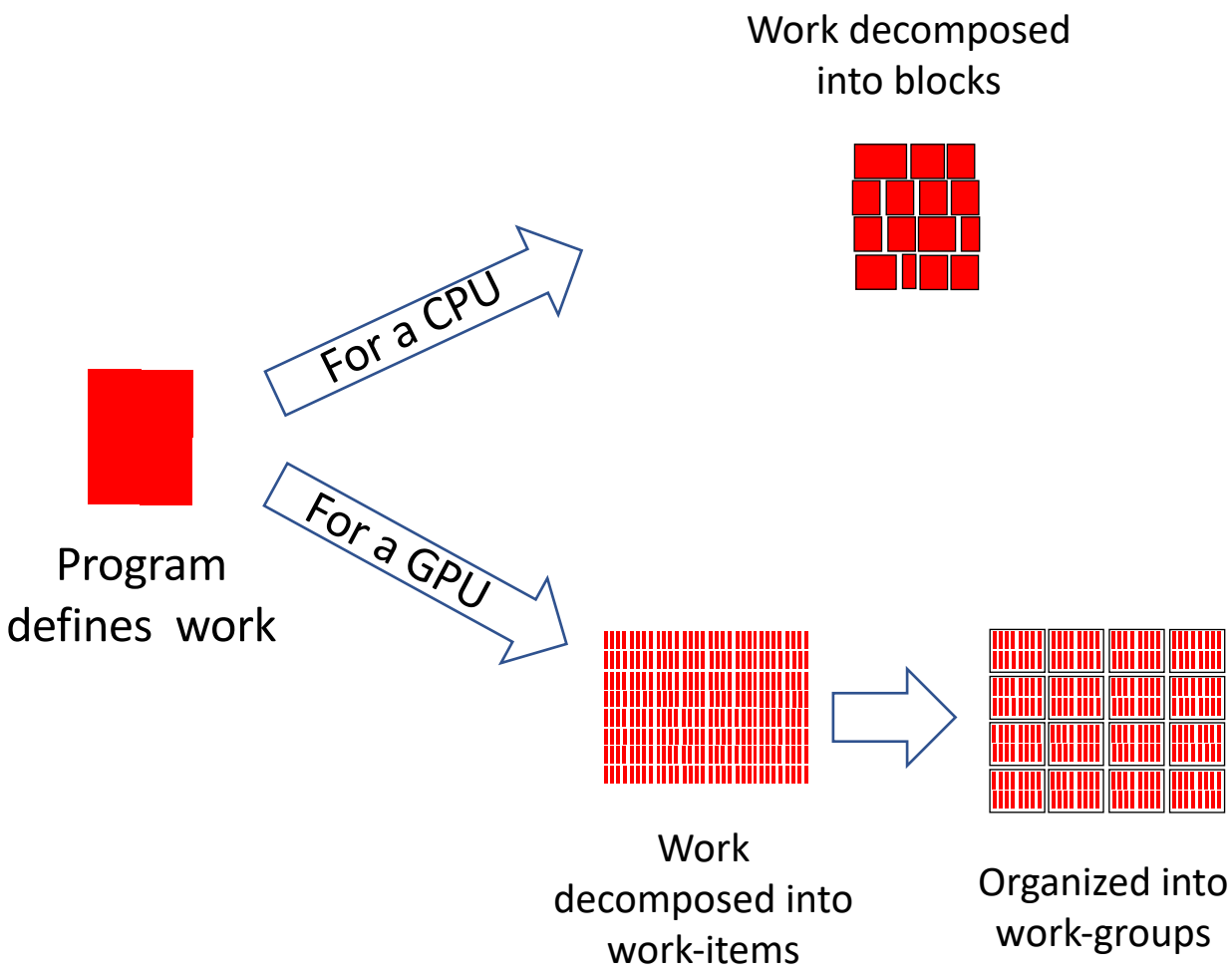
   // fill the arrays
   #pragma omp parallel for
   for (int i=0; i<N; i++){
      a[i] = (float)i;
      b[i] = 2.0*(float)i;
      c[i] = 0.0;
      res[i] = i + 2*i;
   }

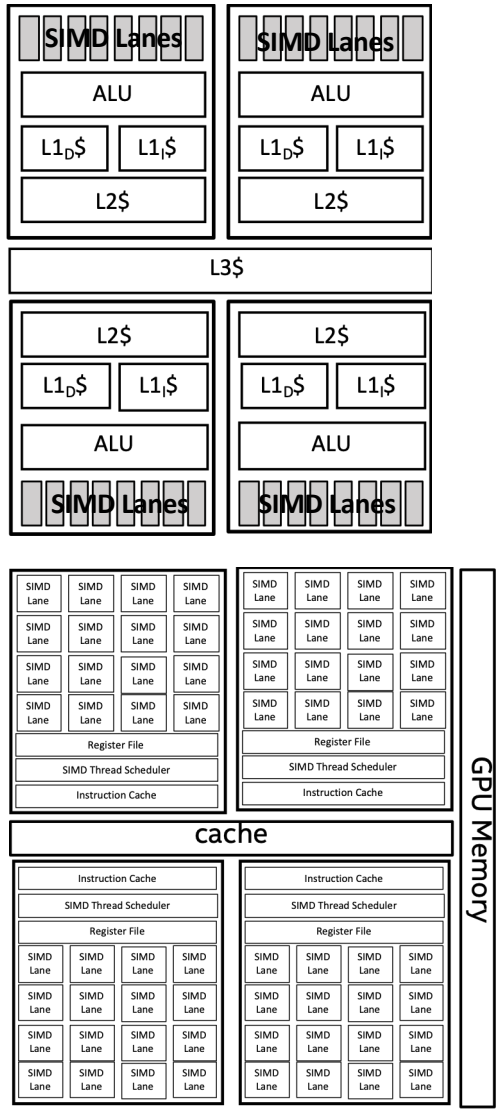
   // add two vectors
   #pragma omp target
   #pragma omp teams loop
   for (int i=0; i<N; i++){
      c[i] = a[i] + b[i];
   }

   // test results
   #pragma omp parallel for reduction(+:err)
   for(int i=0;i<N;i++){
      float val = c[i] - res[i];
      val = val*val;
      if(val>TOL) err++;
   }
   printf("vectors added with %d errors\n", err);
   return 0;
}
```

Let’s compare/contrast concurrency on a CPU and a GPU

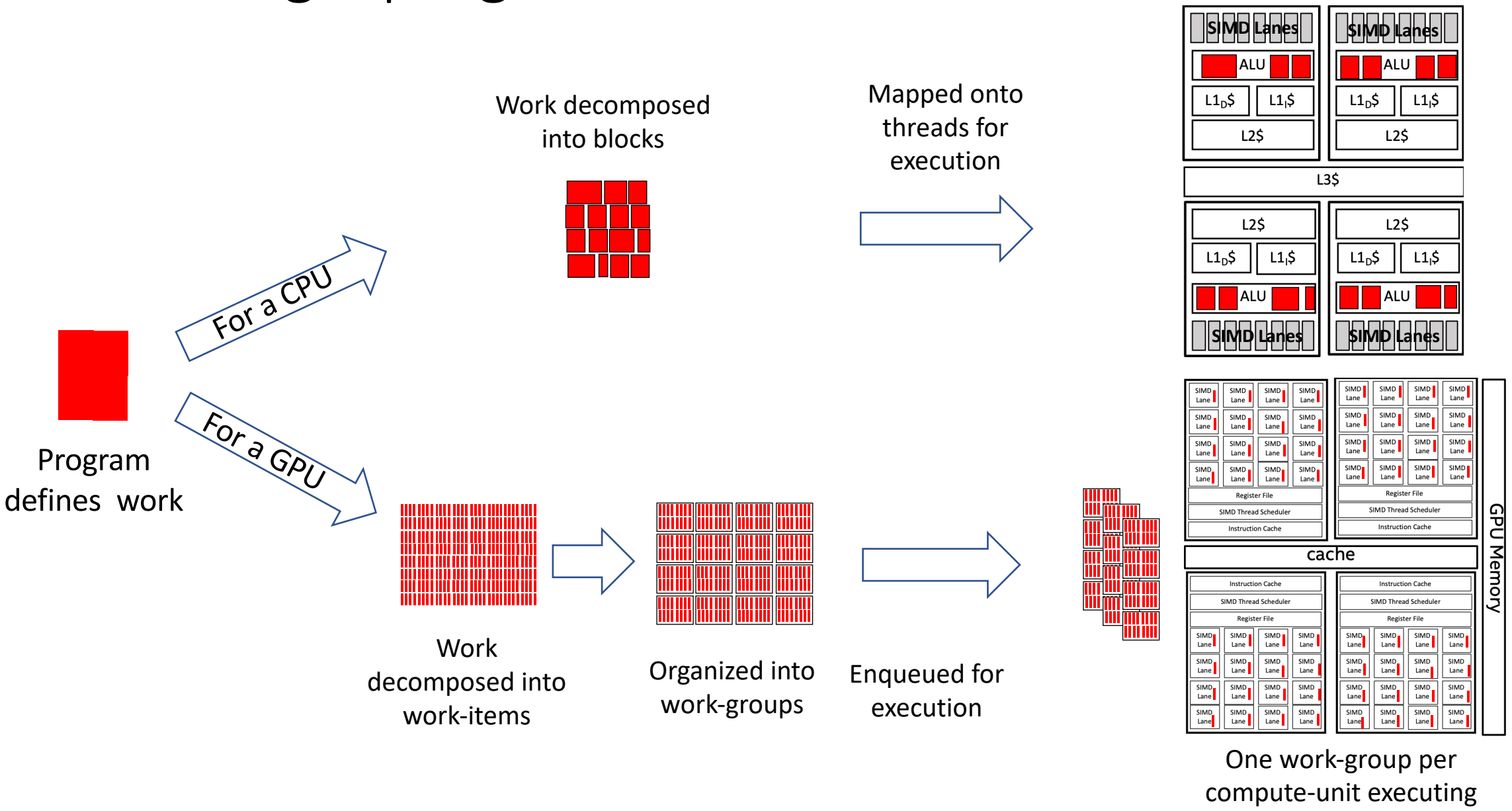
Executing a program on CPUs and GPUs



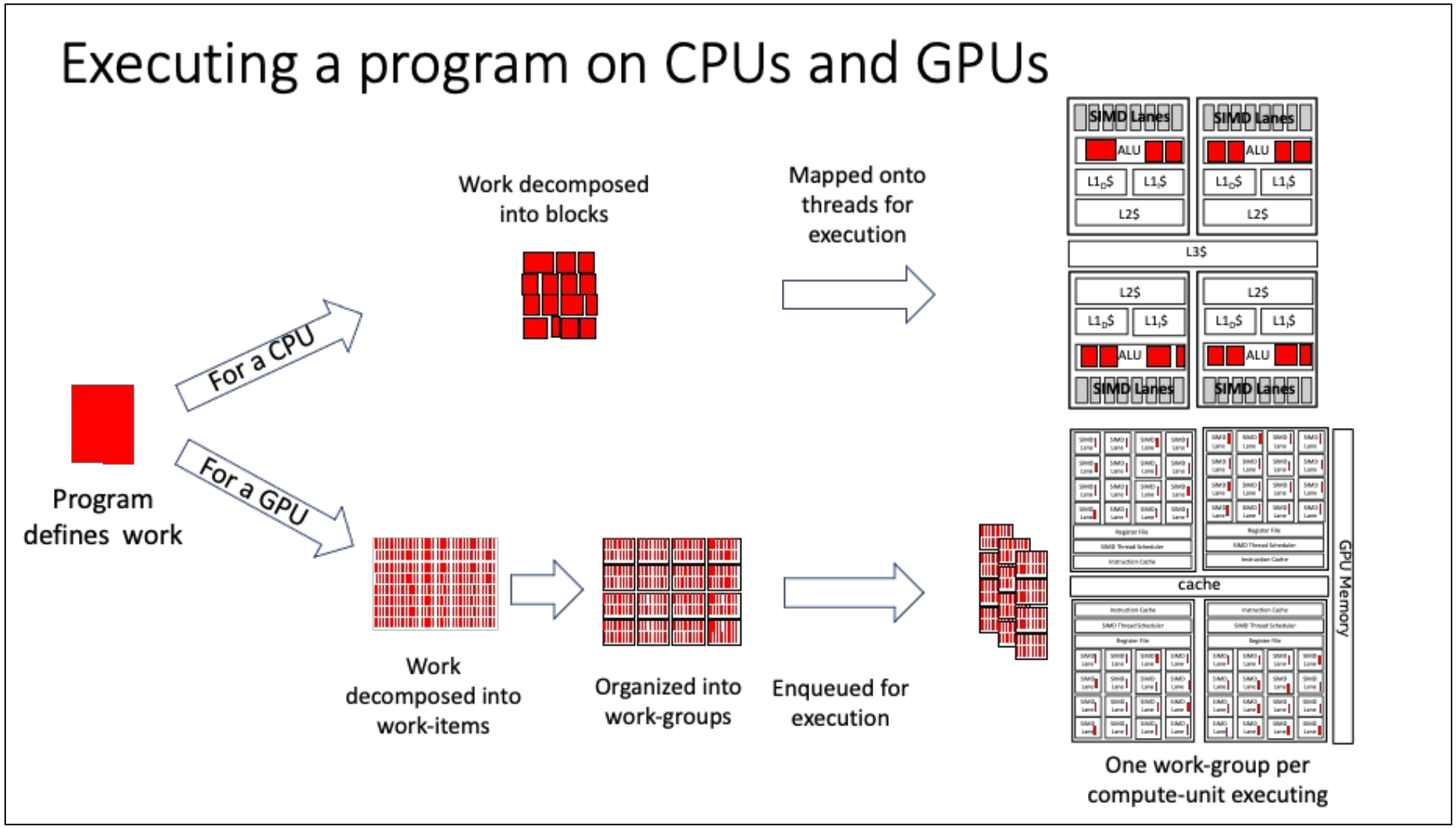


One work-group per compute-unit executing

Executing a program on CPUs and GPUs



CPU/GPU execution modesl



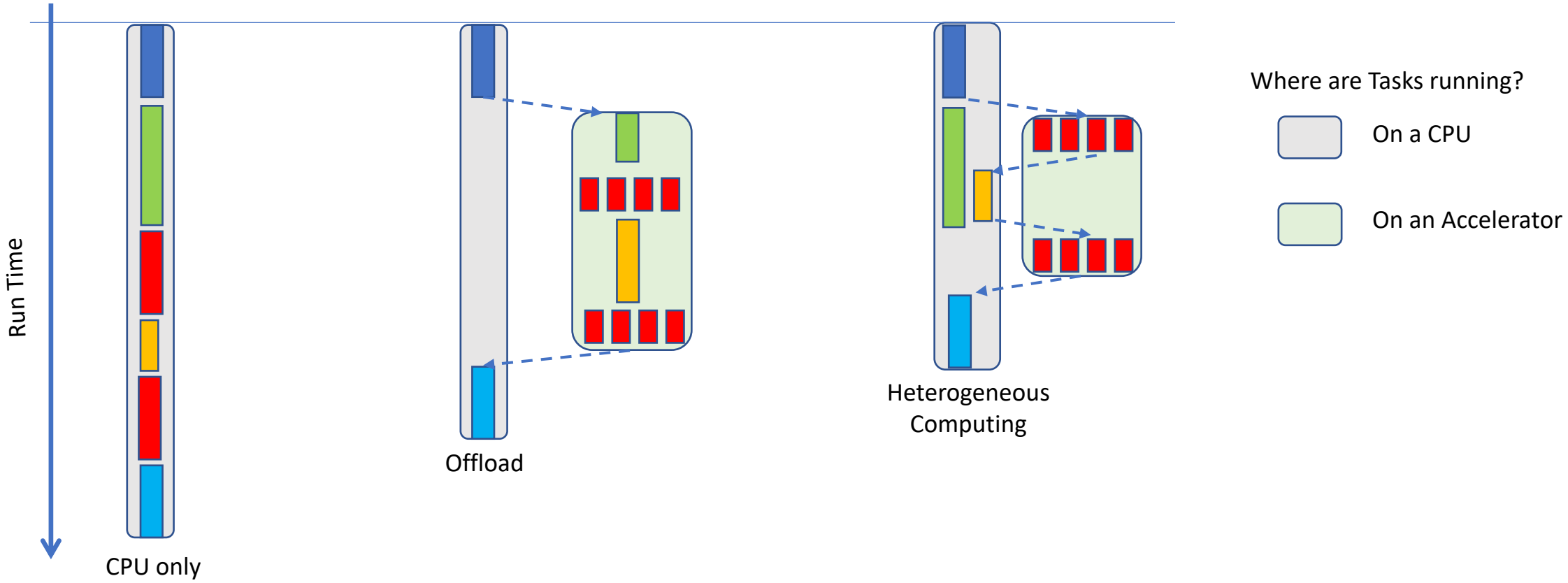
For a CPU, the threads are all active and able to make forward progress.

For a GPU, any given work-group might be in the queue waiting to execute.

No single processor is best at everything

- The idea that you should move everything to the GPU makes no sense

- **Heterogeneous Computing**: Run sub-problems in parallel on the hardware best suited to them.



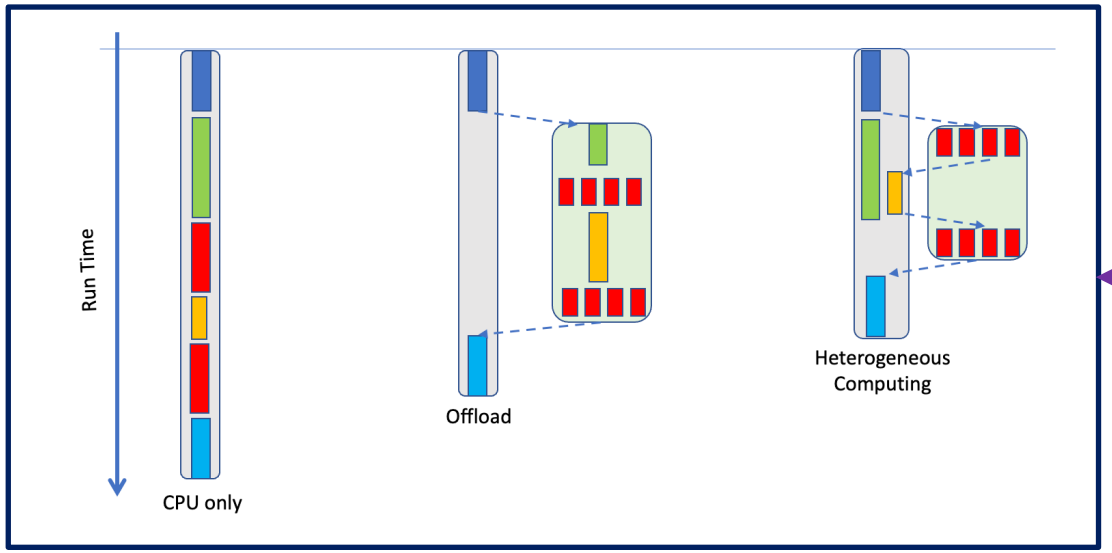
Solution: Simple vector add in OpenMP on GPU

```
int main()
{
    float a[N], b[N], c[N], res[N];
    int err=0;

   // fill the arrays
   #pragma omp parallel for
   for (int i=0; i<N; i++){
      a[i] = (float)i;
      b[i] = 2.0*(float)i;
      c[i] = 0.0;
      res[i] = i + 2*i;
   }

   // add two vectors
   #pragma omp target
   #pragma omp teams loop
   for (int i=0; i<N; i++){
      c[i] = a[i] + b[i];
   }

   // test results
   #pragma omp parallel for reduction(+:err)
   for(int i=0;i<N;i++){
      float val = c[i] - res[i];
      val = val*val;
      if(val>TOL) err++;
   }
   printf("vectors added with %d errors\n", err);
   return 0;
}
```



This is why the fact OpenMP lets us easily mix CPU and GPU code is so important

Implicit data movement covers a small subset of the cases you need in a real program.

To be more general … we need to manage data movement explicitly

Outline



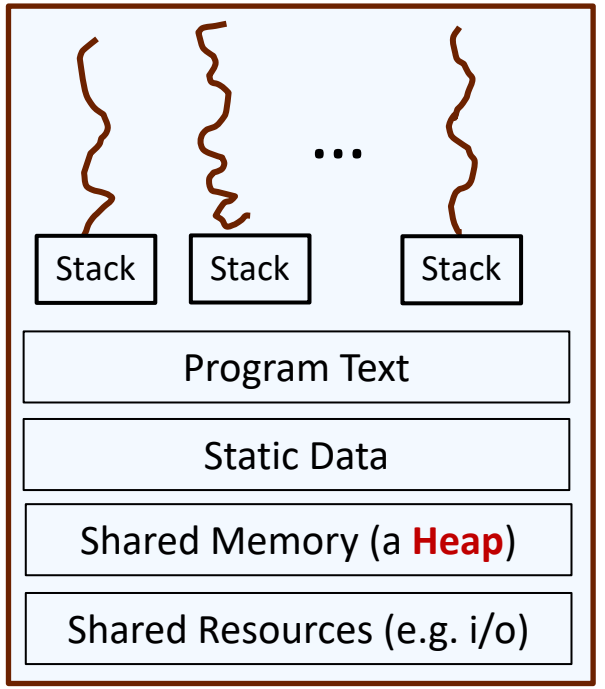
- Introduction to OpenMP
- OpenMP and GPU Programming
- Managing memory on the Host and the GPU
- Fundamental principles of GPU program optimization
- Explicit control of how code maps onto a GPU
- More about Host/GPU memory management
- Using team-local memory: Blocked Matrix Multiplication
- This is the end: Summary/Conclusion
- Extra Content

Explicit data movement

- Previously, we described the rules for *implicit* data movement.
- We can *explicitly* control the movement of data using the **map** clause.
- **Data allocated on the heap needs to be explicitly copied to/from the device**:

```
int main(void) {
   int  ii=0, N = 1024;
   int* A = (int *)malloc(sizeof(int)*N);

   #pragma omp target
   {
     // N, ii and A all exist here
     // The data that A points to (*A , A[ii]) DOES NOT exist here!
   }
}
```



A **process**: An OS managed instance of a program execution

Moving data with the map clause

```
int main(void) {
   int  N = 1024;
   int* A = malloc(sizeof(int)*N);

   #pragma omp target map(A[0:N])
   {
     // N, ii and A all exist here
     // The data that A points to DOES exist here!
   }
}
```

Default mapping **map(tofrom: A[0:N])**

Copy **to** the GPU at start of the target region and **from** the GPU onto the CPU at the end of target region.

Controlling data movement: the simple* case

```
int i, a[N], b[N], c[N];
#pragma omp target map(to:a,b) map(tofrom:c)
```

Data movement defined from the *host* perspective.

- The various forms of the map clause
  - **map(to:list)**: On entering the region, variables in the list are initialized on the device using the original values from the host (host to device copy).
  - **map(from:list)**: At the end of the target region, the values from variables in the list are copied into the original variables on the host (device to host copy). On entering the region, the initial value of the variables on the device are not initialized.
  - **map(tofrom:list)**: the effect of both a map-to and a map-from (host to device copy at start of region, device to host copy at end).
  - **map(alloc:list)**: On entering the region, data is allocated and uninitialized on the device.
  - **map(list)**: equivalent to **map(tofrom:list)**.

* By ”simple” we mean that we do not have nested regions with variables appearing in multiple map clauses

Vector add with dynamic memory on GPU

```
int main()
{
    float *a   = (float *) malloc(sizeof(float) * N);
    float *b   = (float *) malloc(sizeof(float) * N);
    float *c   = (float *) malloc(sizeof(float) * N);
    float *res = (float *) malloc(sizeof(float) * N);
    int err=0;

   // fill the arrays <<<code not shown>>>>

   // add two vectors
   #pragma omp target map(to: a[0:N],b[0:N]) map (tofrom: c[0:N])
   #pragma omp teams loop
   for (int i=0; i<N; i++){
      c[i] = a[i] + b[i];
   }

  // test results <<<code not shown>>>>

  printf("vectors added with %d errors\n", err);
  return 0;
}
```

Commonly used clauses on target and teams loop constructs

- The basic construct* is:

  #pragma omp target *[clause[[,]clause]...]*

  #pragma omp teams loop *[clause[[,]clause]...]*

  for-loops

- The most commonly used clauses are:
  - **map(to | from | tofrom list)** ← default is tofrom
  - **private(***list***)** **firstprivate(***list***)** **lastprivate(***list***)** **shared(***list***)**
    - behave as data environment clauses in the rest of OpenMP, but note values are only created or copied into the region, not back out “at the end”.
  - **reduction(***reduction-identifier* **:** *list***)**
    - behaves as in the rest of OpenMP
  - **collapse(***n***)**
    - Combines loops before the distribute directive splits up the iterations between teams

*Remember … The OpenMP term “construct” refers to a directive plus all the code associated with that directive

Going beyond simple vector addition …

Using OpenMP for GPU application programming … the heat diffusion problem

5-point stencil: the heat program

- The heat equation models changes in temperature over time.

$$\frac{\partial u}{\partial t} - \alpha \nabla^2 u = 0$$

- We'll solve this numerically on a computer using an explicit **finite difference** discretisation.
- $u = u(t, x, y)$ is a function of space and time.
- Partial differentials are approximated using diamond difference formulae:

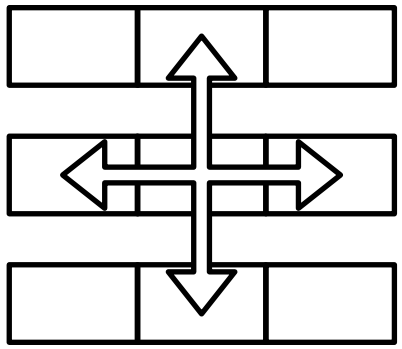
$$\frac{\partial u}{\partial t} \approx \frac{u(t+1, x, y) - u(t, x, y)}{dt}$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u(t, x+1, y) - 2u(t, x, y) + u(t, x-1, y)}{dx^2}$$

  - Forward finite difference in time, central finite difference in space.

5-point stencil: the heat program

- Given an initial value of u, and any boundary conditions, we can calculate the value of u at time t+1 given the value at time t.
- Each update requires values from the north, south, east and west neighbours only:



- Computation is essentially a weighted average of each cell and its neighbouring cells.
- If on a boundary, look up a boundary condition instead.

Method of Manufactured Solution

- Stencil codes are notoriously difficult to **know** if the answer is “correct”.

- Analytic solutions hard to come by:
  - It’s why you’re using a computer to solve the equation approximately after all!

- Method of Manufactured Solution (MMS) is a way to help determine if the code is correct.
- An approach often used to find errors in CFD codes and check convergence properties.

Method of Manufactured Solution

- **Choose** a function for $u(t, x, y)$, substitute into the equation and work through the algebra.

- Ideally like the equation to evaluate to zero so don't need to consider a right-hand side to the equation.

- $u(0, x, y)$ gives the initial conditions.
- Can evaluate boundary conditions, e.g. bottom boundary $u(0,0, y)$

- Because u is **known** for all timesteps (it was chosen!), the exact solution is **known**.

- Compare the **computed** solution to the known u to compute an error.
- Any differences come from approximations in the method, or a bug in your code.

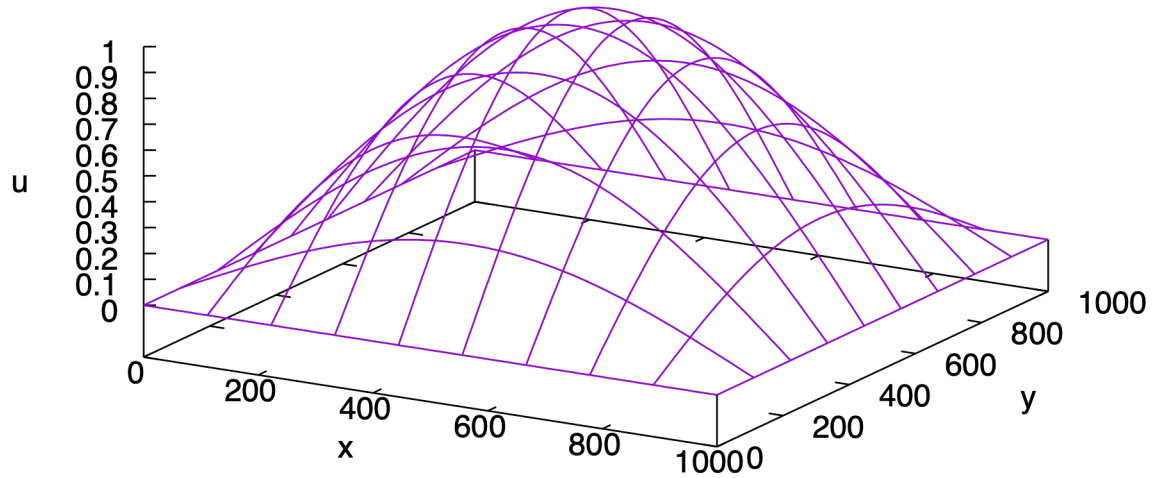
Method of Manufactured Solution

- For the problem of length l, choose u:

$$u(t,x,y) = e^{\frac{-2\alpha\pi^2 t}{l^2}} \sin\frac{\pi x}{l} \sin\frac{\pi y}{l}$$

- Boundary conditions: u is always zero on the boundaries

- Initial value of grid is then $u(0,x,y) = \sin\frac{\pi x}{l} \sin\frac{\pi y}{l}$



Heat program …

```
// Loop over time steps
for (int t = 0; t < nsteps; ++t) {

  // solve over spatial domain for step t
  solve(n, alpha, dx, dt, u, u_tmp);

  // Pointer swap to get ready for next step
  tmp = u;
  u = u_tmp;
  u_tmp = tmp;
}
```

Note: Swapping pointers on the host before entering the target region on the next iteration works on a GPU.

When you map pointers between the host and the device, OpenMP remembers the address.

Swapped addresses on the hosts swaps addresses on the device

- Takes two optional command line arguments: <ncells> <nsteps>
  - E.g. ./heat 1000 10
  - 1000x1000 cells, 10 timesteps (the default problem size).
- If no command line arguments are provided, it uses a default:
  - These two commands both run the default problem size of 1000x1000 cells, 10 timesteps.
  - ./heat
  - ./heat 1000 10
- A sensible bigger problem is 8000 x 8000 cells and 10 timesteps.

git clone https://github.com/tgmattso/OMP_SC.git …. Then look at the file GPU_Exercises/heat.c

Serial CPU code

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
 const double r2 = 1.0 - 4.0*r;

 // malloc and initialize u_tmp and u (code not shown)

 for (int t = 0; t < nsteps; ++t) {

    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        u_tmp[i+j*n] =  r2 * u[i+j*n]          +
            r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
            r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
            r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
            r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
      }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
 }
}
```

Loop over time steps

Loop over NxN spatial domain

Update the 5-point stencil. Boundary conditions on the edges of the domain are fixed at zero.

Note: to show code in PowerPoint it's more compact to show the loops in a single loop-nest rather than the i/j loops in a separate solve() function (as is the case in the program heat.c)

Heat diffusion problem: 5-point stencil code

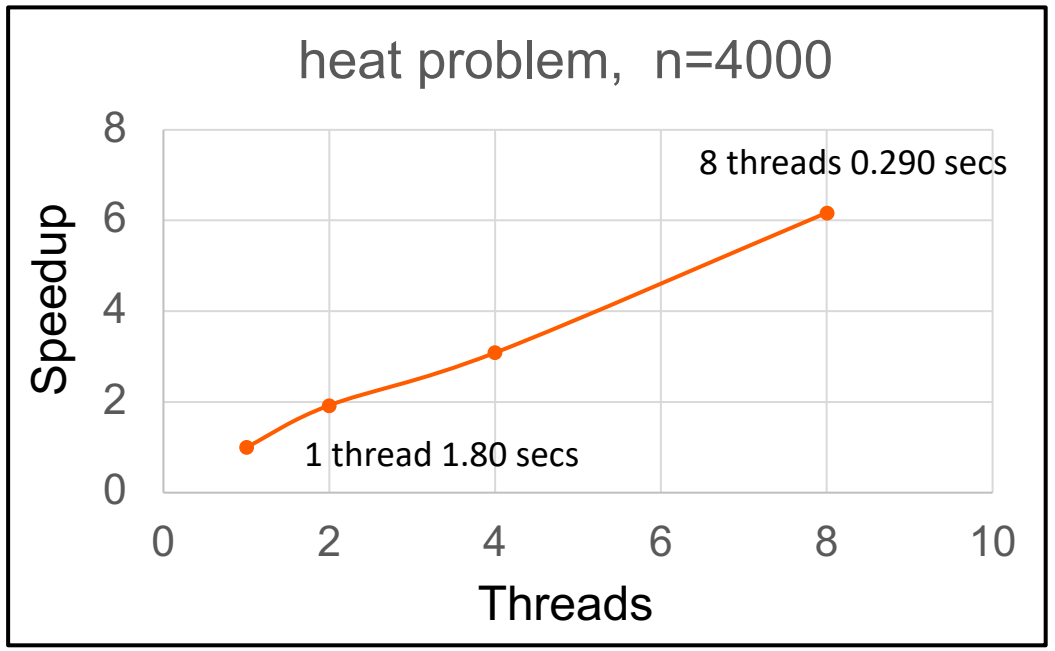
Parallel CPU code, n=4000, nsteps = 10

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)

for (int t = 0; t < nsteps; ++t) {

    #pragma omp parallel for collapse(2)
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        u_tmp[i+j*n] =  r2 * u[i+j*n]           +
            r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
            r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
            r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
            r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
      }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
  }
}
```



Intel® Xeon™ Gold 5218 @ 2.3 Ghz, 8 cores. Nvidia HPC Toolkit compiler nvc –fast –fopenmp heat.c

Exercise: parallel stencil (heat)

- Take the provided heat stencil code (heat.c)
- Add OpenMP directives to parallelize the loops on the GPU
- Most of the runtime occurs in the solve() routine. Focus on that function. The rest of the code is there to just support the work inside solve.

```
module load rocm amdclang
amdclang –fopenmp --offload-arch=gfx942 heat.c
sbatch job_gpu.sh
```

gpu_job.sh

```
#!/bin/bash
#SBATCH -p MI300A_CPX
#SBATCH -N 1
#SBATCH --gpus=1
#SBATCH -t 10:00

module load rocm amdclang

OMP_TARGET_OFFLOAD=MANDATORY ./a.out
```

```
… or fetch an interactive session
    salloc -N1 --gpus=1 -t 10:00 -p MI300A_CPX
```

- double omp_get_wtime();
- #pragma omp parallel
- #pragma omp for
- #pragma omp parallel for
- #pragma omp task
- #pragma omp taskwait
- #pragma single
- #pragma omp target
- #pragma omp teams loop
- Plus the clauses
  - private(), firstprivate(), reduction(+:var), collapse(n)
  - map(to:vptr[Lower:Count]) map(from:vptr[Lower:Count]) map(tofrom:vptr[Lower:Count])

git clone https://github.com/tgmattso/OMP_SC.git …. Then look at the file GPU_Exercises/heat.c

Heat diffusion problem: 5-point stencil code

Parallel GPU code, n=4000 nsteps=10




```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)

for (int t = 0; t < nsteps; ++t) {
   #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
   #pragma omp teams loop
   for (int i = 0; i < n; ++i) {
     for (int j = 0; j < n; ++j) {
       u_tmp[i+j*n] =  r2 * u[i+j*n]          +
           r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
           r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
           r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
     }
   }
   // Pointer swap to get ready for next step
   tmp = u;
   u = u_tmp;
   u_tmp = tmp;
 }
}
```

When you map pointers between the host and the device, OpenMP remembers the address.

Swapped addresses on the hosts swaps addresses on the device

GPU Solver time = 0.22 secs

This isn't much better than the time for 8 cores on a CPU (0.29 secs).

Why is the performance so bad?

AMD Instinct™ MI300A GPU, a single XCD
amdclang –fopenmp –O3 --offload-arch=gfx942 heat .c –lm

Heat diffusion problem: 5-point stencil code

Parallel GPU code, n=4000 nsteps=10




```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)

for (int t = 0; t < nsteps; ++t) {
   #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
   #pragma omp teams loop
   for (int i = 0; i < n; ++i) {
     for (int j = 0; j < n; ++j) {
       u_tmp[i+j*n] =  r2 * u[i+j*n]          +
           r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
           r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
           r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
     }
   }
   // Pointer swap to get ready for next step
   tmp = u;
   u = u_tmp;
   u_tmp = tmp;
}
}
```

At the beginning of each iteration, copy $(2*N^2)*sizeof(TYPE)$ bytes to the device

With a runtime of 0.22 secs, we see that Data Movement dominates performance.

At the end of each iteration, copy $(2*N^2)*sizeof(TYPE)$ bytes from the device

Finer control over data movement

- Recall that data is mapped to/from the device at start/end of target region

```
#pragma omp target map(tofrom: A[0:N])
{
  …
}
```

- Inefficient to move data around all the time
- We need to keep data resident on the device *between* target regions.
- We need to manage the device data environment independently from the target regions that launch kernels.

Target enter/exit data constructs

- Create a data region on the target device (a device data environment) with two standalone directives:

```
#pragma omp target enter data map(…)
#pragma omp target exit data map(…)
```

- The **target enter data** maps variables to the device data environment.

- The **target exit data** unmaps variables from the device data environment.

- Once created, subsequent **target** regions inherit the existing data environment.

Target enter/exit data example

```
void init_array(int *A, int *B, int N) {
   for (int i = 0; i < N; ++i) { A[i] = i;  B[i]=2*I;}

  #pragma omp target enter data map(to: A[0:N], B[0:N])
}

int main(void) {

   int N = 1024;
   int *A = malloc(sizeof(int) * N);
   int *B = malloc(sizeof(int) * N);
   init_array(A, B, N);

   #pragma omp target
   #pragma omp teams loop
   for (int i = 0; i < N; ++i)
      A[i] = A[i] * B[i];

 #pragma omp target exit data map(from: A[0:N])
}
```

Exercise

- Modify your parallel heat code from the last exercise.
- Use the ‘target data’ constructs to control the device data environment.
- Think about the map clauses and how to use them to minimize data movement.
- Question … will the pointer swap on the host still work?

  - #pragma omp target
  - #pragma omp target enter data map(to: list)
  - #pragma omp target exit data map(from: list)
  - #map(to:list) map(from:list) map(tofrom:list)
  - #pragma omp target teams loop

If your runtimes are tiny, you may want to work with a larger problem.

Try a 4000 by 4000 gird for 10 steps:

./heat 4000 10

```
module load rocm amdclang
amdclang –fopenmp --offload-arch=gfx942 heat .c -lm
sbatch job_gpu.sh
```

```
#!/bin/bash
#SBATCH -p PPAC_MI300A_CPX
#SBATCH -N 1
#SBATCH --gpus=1
#SBATCH -t 10:00

module load rocm amdclang

OMP_TARGET_OFFLOAD=MANDATORY ./a.out
```

… or fetch an interactive session

```
salloc -N1 --gpus=1 -t 10:00 -p PPAC_MI300A_CPX
```

Heat diffusion problem: 5-point stencil code

Parallel results, n=4000 nsteps=10




```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
// malloc and initialize u_tmp and u (code not shown)
#pragma omp target enter data map(to: u[0:n*n], u_tmp[0:n*n])

for (int t = 0; t < nsteps; ++t) {
   #pragma omp target
   #pragma omp teams loop
   for (int i = 0; i < n; ++i) {
     for (int j = 0; j < n; ++j) {
       u_tmp[i+j*n] =  r2 * u[i+j*n]          +
           r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
           r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
           r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
     }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
  }
#pragma omp target exit data map(from: u[0:n*n])
}
```

Create a data region and map indicated data on entry

GPU Solver time* = **0.047** secs

This is a general principal … if you want performance, you must optimize data movement.

*includes time for target enter/exit data

AMD Instinct™ MI300A GPU, a single XCD
amdclang –fopenmp –O3 --offload-arch=gfx942 heat .c –lm

Exit the data region and map indicated data

Getting the data movement between host memory and device memory is key.

What are the other major issues to consider when optimizing performance?

Outline



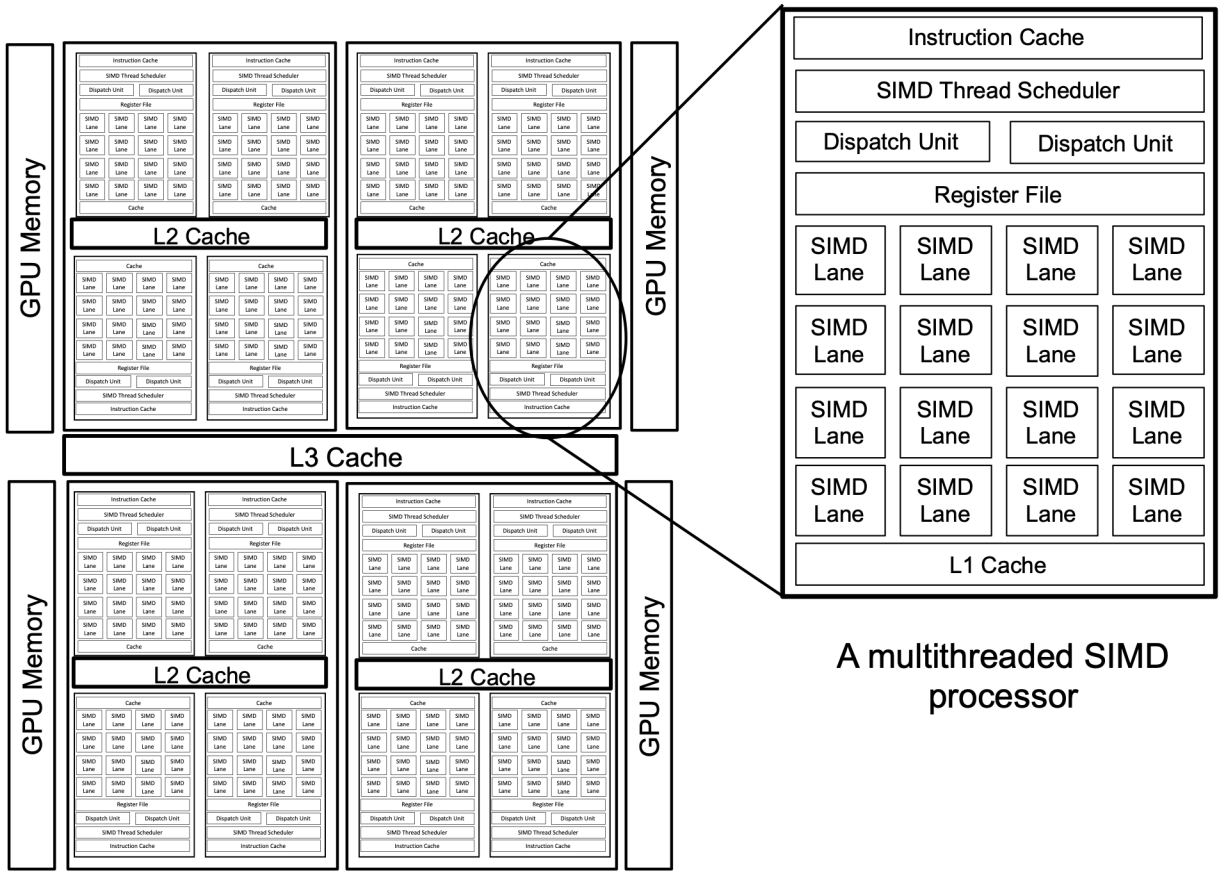
- Introduction to OpenMP
- OpenMP and GPU Programming
- Managing memory on the Host and the GPU
- Fundamental principles of GPU program optimization
- Explicit control of how code maps onto a GPU
- More about Host/GPU memory management
- Using team-local memory: Blocked Matrix Multiplication
- This is the end: Summary/Conclusion
- Extra Content

Occupancy: Keep all the GPU resources busy

- In our "GPU cartoon" we have 16 multithreaded SIMD processors each with 16 SIMD lanes …. For a total of 16^2=256 processing elements.

- You want all resources busy at all times. You do that by keeping excess work for the multithreaded SIMD processors … if they are other busy on some high latency operation, you want a new work-group is ready to be scheduled for execution.

- *Occupancy* having enough work-groups to keep the GPU busy. To support high occupancy, you need many more work-items than SIMD-lanes.



A multithreaded SIMD processor

```
#pragma omp parallel for
for(int i=0;i<N;i++)
  for(int j=0;j<N;j++)
    for(int k=0;k<N;k++)
      *(C+(i*N+j)) += *(A+(i* N +k)) *  *(B+(k* N +j));
```

Parallelize i-loop parallelism O(N)

```
#pragma omp parallel for collapse(2)
for(int i=0;i<N;i++)
  for(int j=0;j<N;j++)
    for(int k=0;k<N;k++)
      *(C+(i*N+j)) += *(A+(i* N +k)) *  *(B+(k* N +j));
```

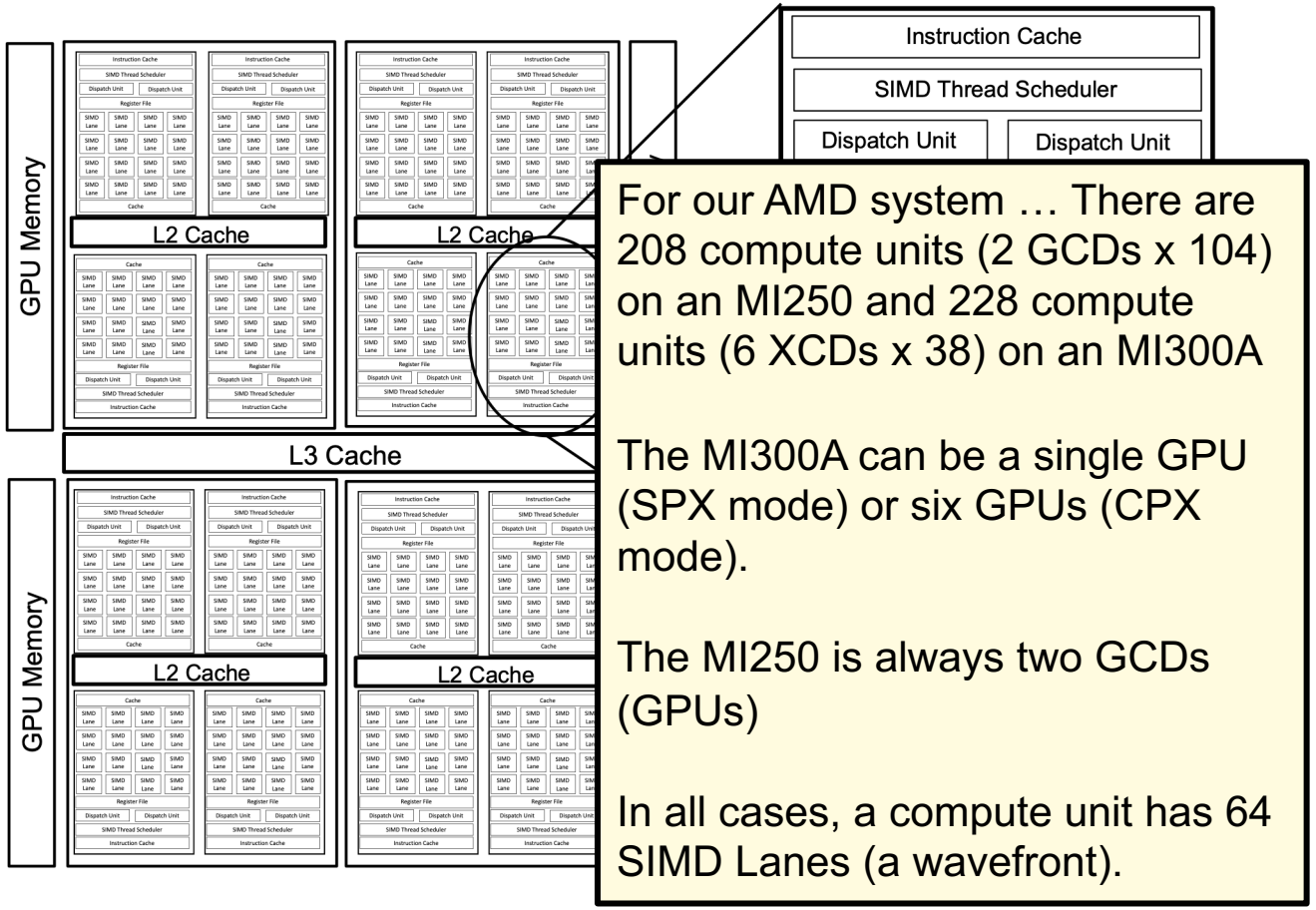
Parallelize combined i/j-loops parallelism O(N^2)

Occupancy: Keep all the GPU resources busy

- In our "GPU cartoon" we have 16 multithreaded SIMD processors each with 16 SIMD lanes …. For a total of 16^2=256 processing elements.

- You want all resources busy at all times. You do that by keeping excess work for the multithreaded SIMD processors … if they are other busy on some high latency operation, you want a new work-group is ready to be scheduled for execution.

- *Occupancy* having enough work-groups to keep the GPU busy. To support high occupancy, you need many more work-items than SIMD-lanes.



For our AMD system … There are 208 compute units (2 GCDs x 104) on an MI250 and 228 compute units (6 XCDs x 38) on an MI300A

The MI300A can be a single GPU (SPX mode) or six GPUs (CPX mode).

The MI250 is always two GCDs (GPUs)

In all cases, a compute unit has 64 SIMD Lanes (a wavefront).

```
#pragma omp parallel for
for(int i=0;i<N;i++)
  for(int j=0;j<N;j++)
    for(int k=0;k<N;k++)
      *(C+(i*N+j)) += *(A+(i* N +k)) *  *(B+(k* N +j));
```

Parallelize i-loop parallelism O(N)



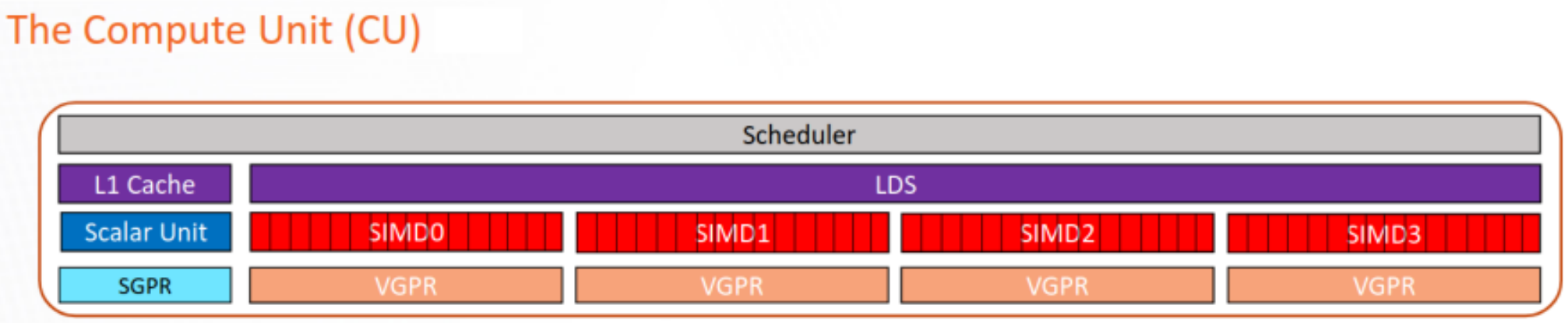
```
#pragma omp parallel for collapse(2)
for(int i=0;i<N;i++)
  for(int j=0;j<N;j++)
    for(int k=0;k<N;k++)
      *(C+(i*N+j)) += *(A+(i* N +k)) *  *(B+(k* N +j));
```

Parallelize combined i/j-loops parallelism O(N^2)

Memory Resources that affect Occupancy

Compute Units have finite resources that are shared between work items

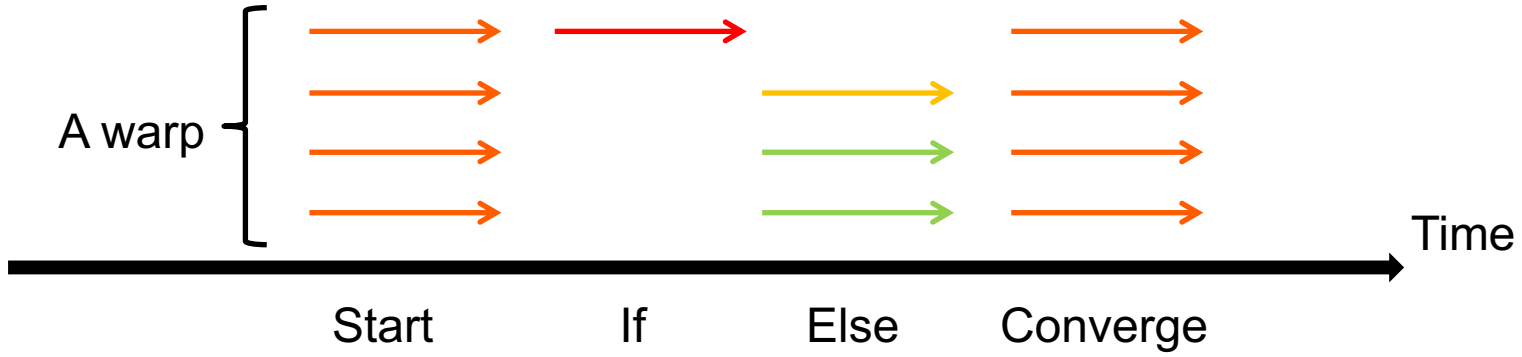
- Local Data Share (LDS)
- Vector General Purpose Registers (VGPRs)
- Scalar General Purpose Registers (SGPRs)



The GPU can only schedule more work if there are enough resources available

Converged Execution: Single Instruction Multiple Data

- Individual work-items of a warp start together at the same program address
- Each work-item has its own instruction address counter and register state
  - Each work-item is free to branch and execute independently
  - Supports the Single Program Multiple Data (SPMD) pattern.
- Branch behavior
  - Each branch will be executed serially
  - Work-items not following the current branch will be disabled



Converged Execution: Branching

- GPUs tend not to support speculative execution, which means that branch instructions have high latency
- This latency can be hidden by switching to alternative work-items/work-groups, but avoiding branches where possible is still a good idea to improve performance
- When different work-items executing within the same SIMD ALU array take different paths through conditional control flow, we have ***divergent branches*** *(vs.* ***uniform branches****)*
- Divergent branches are bad news: some work-items will stall while waiting for the others to complete
- We can use predication, selection and masking to convert conditional control flow into straight line code and significantly improve the performance of code that has lots of conditional branches

Branching

Conditional execution

```
// Only evaluate expression
// if condition is met
if (a > b)
{
  acc += (a - b*c);
}
```

Selection and masking

```
// Always evaluate expression
// and mask result
temp = (a - b*c);
mask = (a > b ? 1.f : 0.f);
acc += (mask * temp);
```

Coalesced memory accesses

- ***Coalesced memory accesses*** are key for high performance code, especially on GPUs

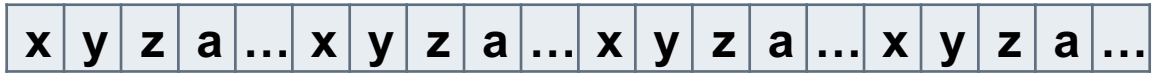
- In principle, it's very simple, but frequently requires transposing or transforming data on the host before sending it to the GPU

- Sometimes this is an issue of Array of Structures vs. Structure of Arrays (AoS vs. SoA)

Memory layout is critical to performance

- Structure of Arrays vs. Array of Structures
  - Array of Structures (AoS) more natural to code:

```
struct Point{ float x, y, z, a; };
Point *Points;
```



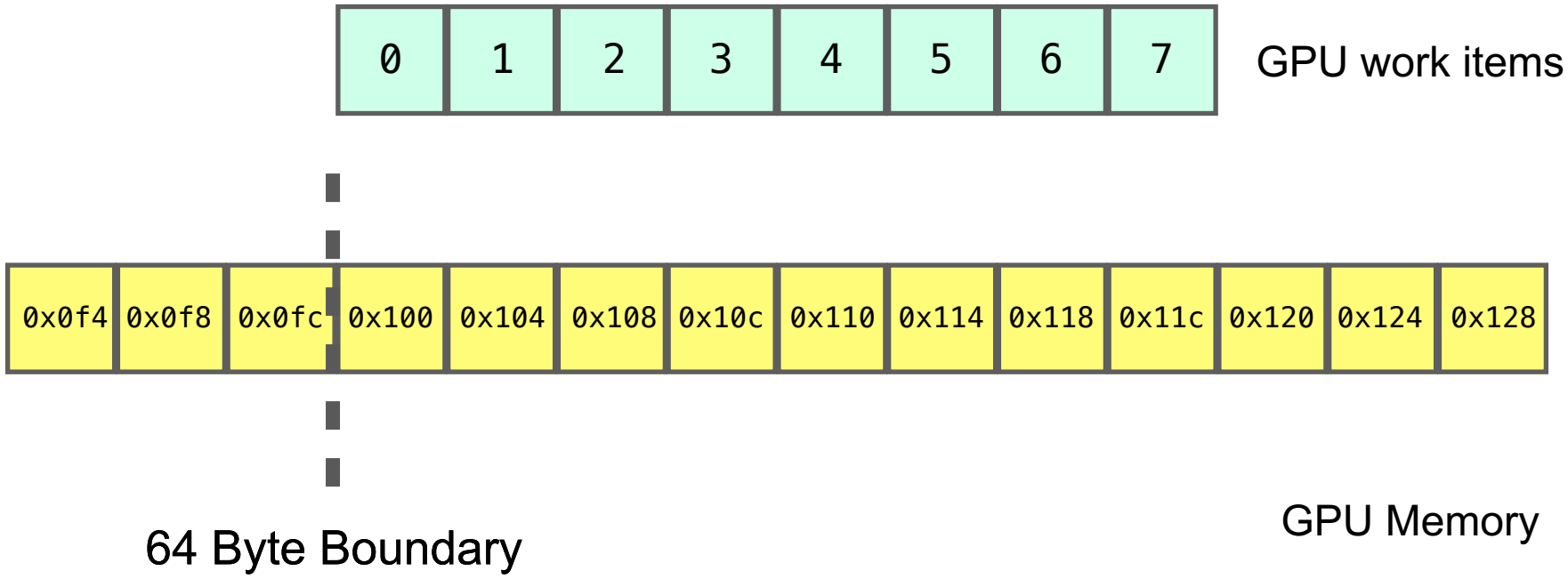
  - Structure of Arrays (SoA) suits memory coalescence in vector units

```
struct { float *x, *y, *z, *a; } Points;
```



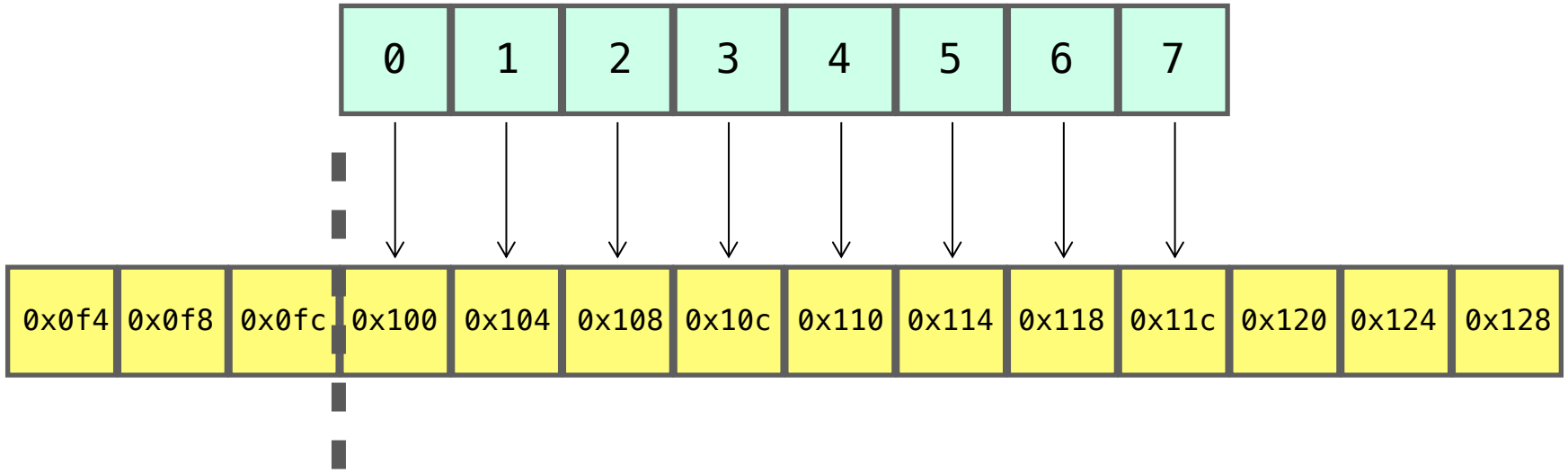
Adjacent work-items/vector-lanes like to access adjacent memory locations

Memory access patterns



Memory access patterns

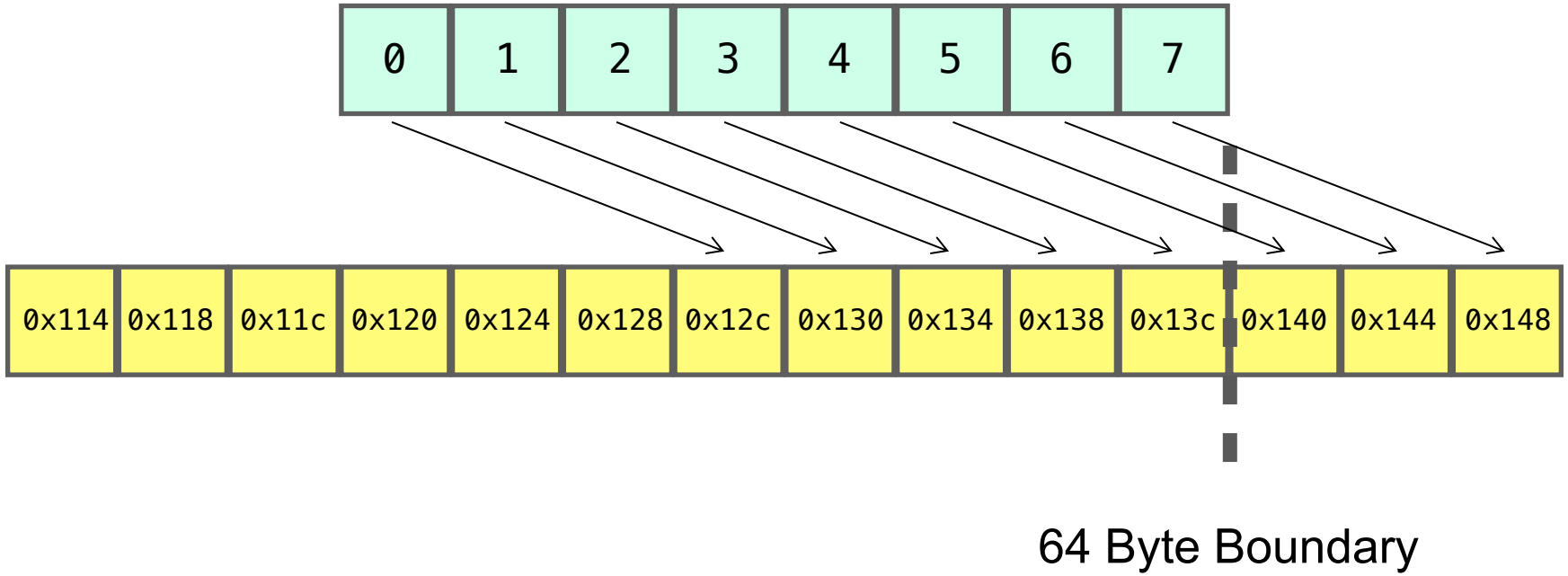
```
float val1 = memA[id];
```



64 Byte Boundary

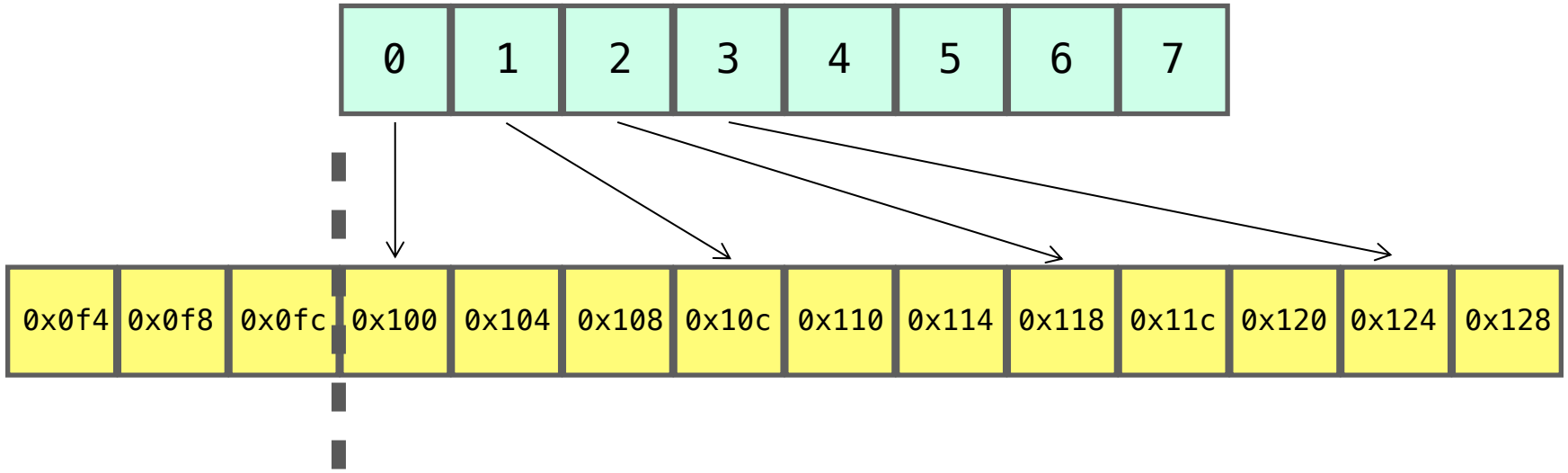
Memory access patterns

```
const int c = 3;
float val2 = memA[id + c];
```



Memory access patterns

```
float val3 = memA[3*id];
```

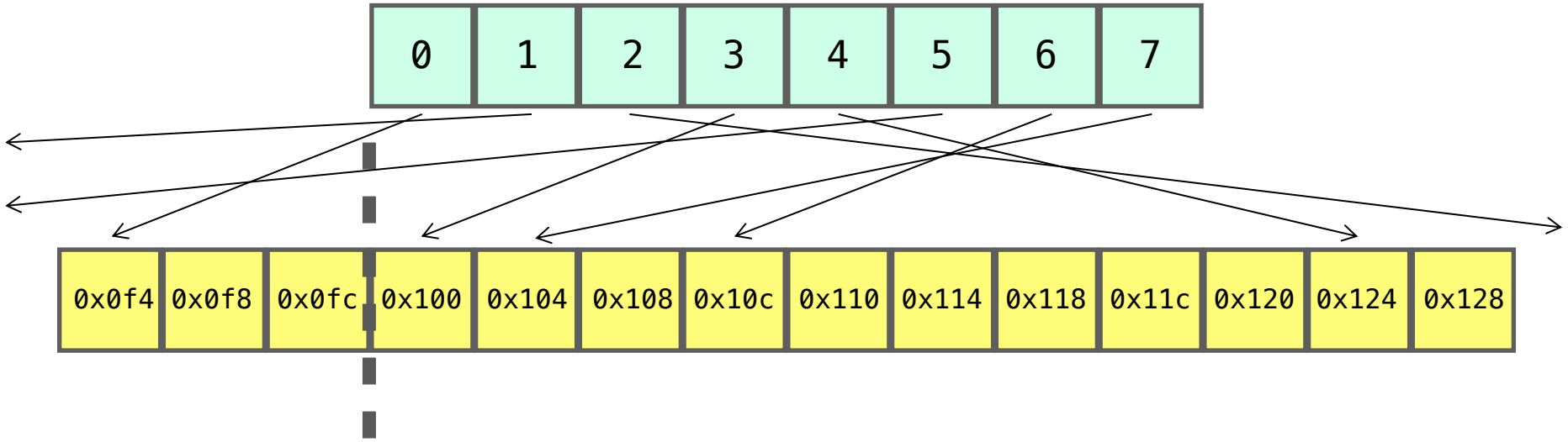


Strided access results in multiple memory transactions (and kills throughput)

Memory access patterns

```
const int loc = some_strange_func(id);

float val4 = memA[loc];
```



64 Byte Boundary

Coalescence

- **Coalesced Memory access patterns** - organize memory accesses across work-items to optimize overall performance
- Coalesced memory accesses are key for high bandwidth
- Simply, it means, if thread *i* accesses memory location *n* then thread *i+1* accesses memory location *n+1*
- In practice, it's not quite as strict…

```
for (int id = 0; id < size; id++)
{
  // ideal
     float val1 = memA[id];

  // still pretty good
     const int c = 3;
     float val2 = memA[id + c];

  // stride size is not so good
     float val3 = memA[c*id];

  // terrible
     const int loc =
       some_strange_func(id);

     float val4 = memA[loc];
}
```

Exercise

- Optimize the stencil ‘solve’ kernel.
- Start with your code with optimized memory movement from the last exercise.
- Experiment with the optimizations we’ve discussed.
- Focus on the memory access pattern.
- Try different input sizes to see the effect of the optimizations.
- Keep an eye on the solve time as reported by the application.

- #pragma omp target
- #pragma omp target enter data map(to: list)
- #pragma omp target exit data map(from: list)
- #pragma omp target teams loop
- map(to:list) map(from:list) map(tofrom:list)
- collapse(n)

```
module load rocm
module load amdclang
$CC –fopenmp –offload-arch=gfx942 vadd.c
sbatch job_gpu.sh
```

gpu_job.sh

```
#!/bin/bash
#SBATCH -p PPAC_MI300A_CPX
#SBATCH -N 1
#SBATCH --gpus=1
#SBATCH -t 10:00

module load rocm amdclang

OMP_TARGET_OFFLOAD=MANDATORY ./a.out
```

… or fetch an interactive session

```
salloc -N1 --gpus=1 -t 10:00 -p PPAC_MI300A_CPX
```

Heat diffusion problem: 5-point stencil code

Parallel CPU and GPU results, n=4000 nsteps=10

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
// malloc and initialize u_tmp and u (code not shown)


for (int t = 0; t < nsteps; ++t) {

    #pragma omp parallel for
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        u_tmp[i+j*n] =  r2 * u[i+j*n]                +
            r * ((i < n-1) ? u[i+1+j*n]   : 0.0) +
            r * ((i > 0)   ? u[i-1+j*n]   : 0.0) +
            r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
            r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
      }
    }
    // Pointer swap for  next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}

}
```

This is the ij loop order. Swap these loops to get the ji order.

This GPU code used target enter data and target exit data

What if I optimize the CPU code?

… just swap the i and j loops. Using collapse(2) did not help on the CPU but it did for the GPU

CPU

Num threads	ij loop order	ji loop order
1	1.512849	0.262260
2	0.776229	0.132453
4	0.400822	0.064220
8	0.227317	0.046586

Intel® Xeon™ Gold 5218 @ 2.3 Ghz, 8 cores.
Nvidia HPC Toolkit compiler nvc –fast –fopenmp heat.c

All times in seconds … just time for the solver*.

GPU

ij loop order	ji loop order
0.046	0.029

AMD Instinct™ MI300A GPU, a single XCD
amdclang –fopenmp –O3 --offload-arch=gfx942 heat .c –lm
Used collapse(2) on i/j loops

* All times are “solver times” and include loading data onto the GPU and the loops over calls to the solver.

Solution: collapse + swap loop order

```
// Compute the next timestep, given the current timestep
void solve(const int n, const double alpha, const double dx, const double dt, const double * restrict u,
double * restrict u_tmp) {
  // Finite difference constant multiplier
  const double r = alpha * dt / (dx * dx);
  const double r2 = 1.0 - 4.0*r;

  // Loop over the nxn grid
  #pragma omp target
  #pragma omp teams loop collapse(2)
  for (int j = 0; j < n; ++j) {
    for (int i = 0; i < n; ++i) {


      // Update the 5-point stencil, using boundary conditions on the edges of the domain.
      // Boundaries are zero because the MMS solution is zero there.
      u_tmp[i+j*n] =  r2 * u[i+j*n] +
      r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
      r * ((i > 0)   ? u[i-1+j*n] : 0.0) +
      r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
      r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
}}}
```

Create more work … to better fill the processing elements of the GPU

Swap the i and j loops so that the i+j*n memory accesses are contiguous

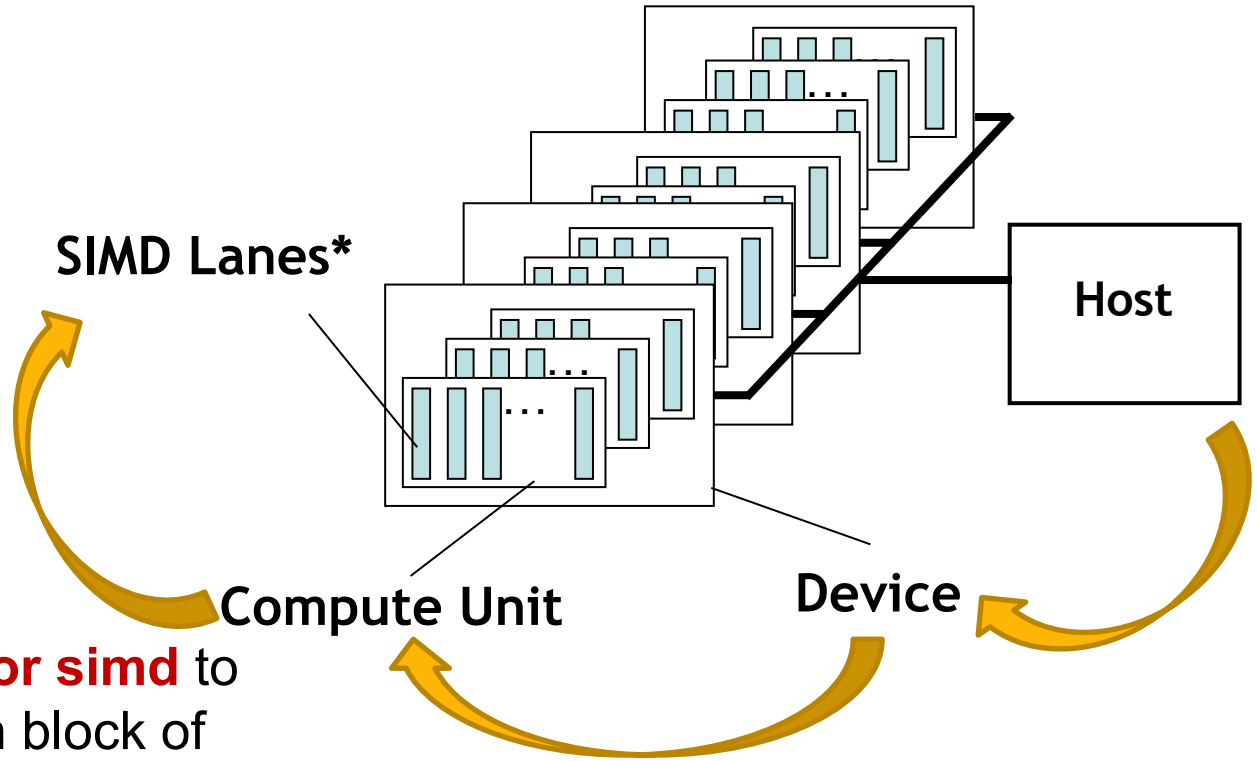
The loop construct is great, but sometimes you want more control.

Outline



- Introduction to OpenMP
- OpenMP and GPU Programming
- Managing memory on the Host and the GPU
- Fundamental principles of GPU program optimization
- Explicit control of how code maps onto a GPU
- More about Host/GPU memory management
- Using team-local memory: Blocked Matrix Multiplication
- This is the end: Summary/Conclusion
- Extra Content

Our host/device Platform Model and OpenMP



Parallel for simd to run each block of loop iterations on the **SIMD Lanes***

Target construct to get onto a device

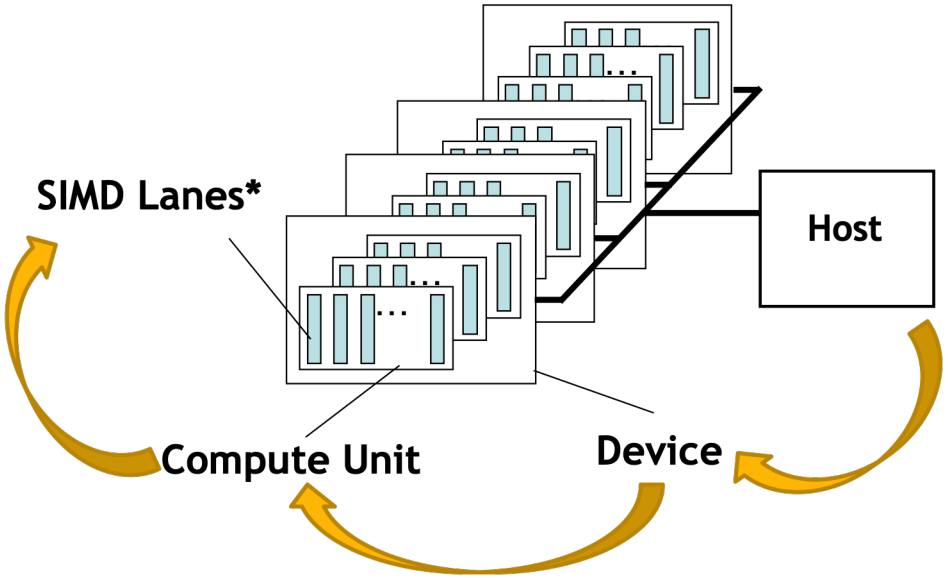
Teams construct to create a league of teams with one team of threads on each compute unit.

Distribute construct to assign blocks of loop iterations to teams.

***processing elements** in OpenCL terminology

teams and distribute constructs

- The **teams** construct
  - Similar to the **parallel** construct
  - It starts a league of thread teams
  - Each team in the league starts as one initial thread – a team of one
  - Threads in different teams cannot synchronize with each other
  - The construct must be “perfectly” nested in a **target** construct

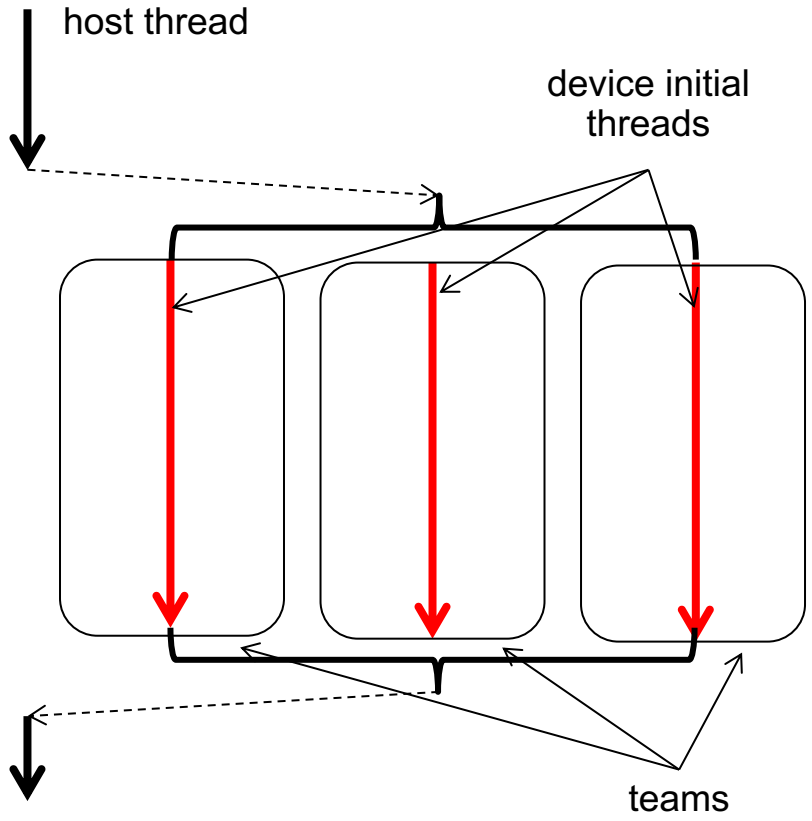


- The **distribute** construct
  - Similar to the **for** construct
  - Loop iterations are workshared across the initial threads in a league
  - No implicit barrier at the end of the construct
  - **dist_schedule(*kind[, chunk_size]*)**
    - If specified, scheduling kind must be static
    - Chunks are distributed in round-robin fashion in chunks of size ***chunk_size***
    - If no chunk size specified, chunks are of (almost) equal size; each team receives at least one chunk

Create a league of teams and distribute a loop among them

- teams construct
- distribute construct

```
#pragma omp target
#pragma omp teams
#pragma omp distribute
 for (i=0;i<N;i++)
   …
```

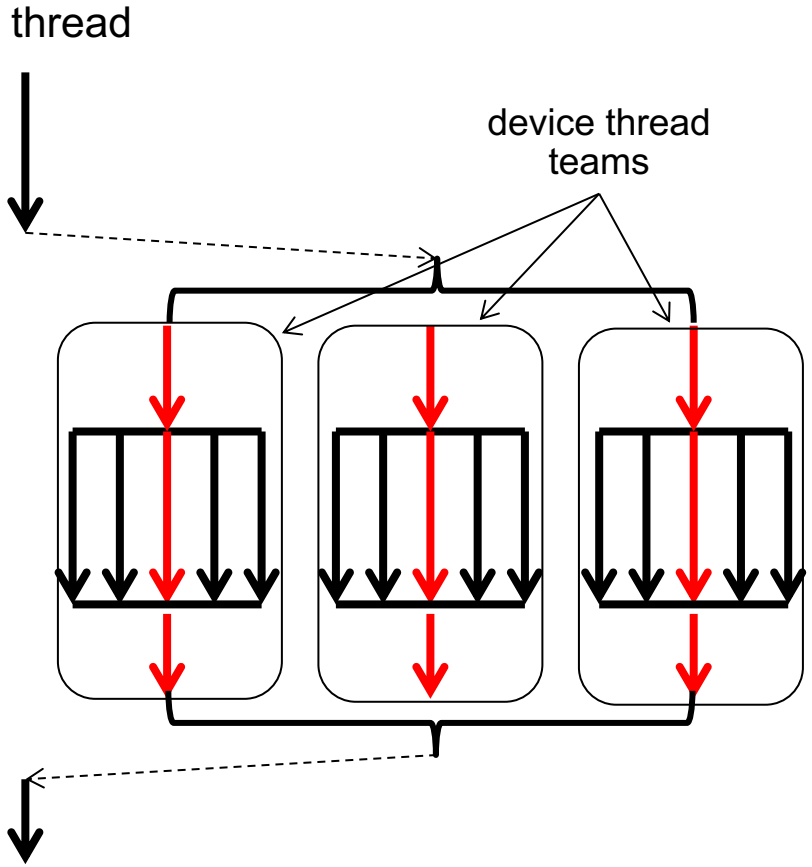


- Transfer execution control to MULTIPLE device initial threads
- Workshare loop iterations across the initial threads.

Create a league of teams and distribute a loop among them and run each team in parallel with its partition of the loop

- teams
- distribute
- parallel for

```
#pragma omp target
#pragma omp teams
#pragma omp distribute
#pragma omp parallel for
 for (i=0;i<N;i++)
   …
```

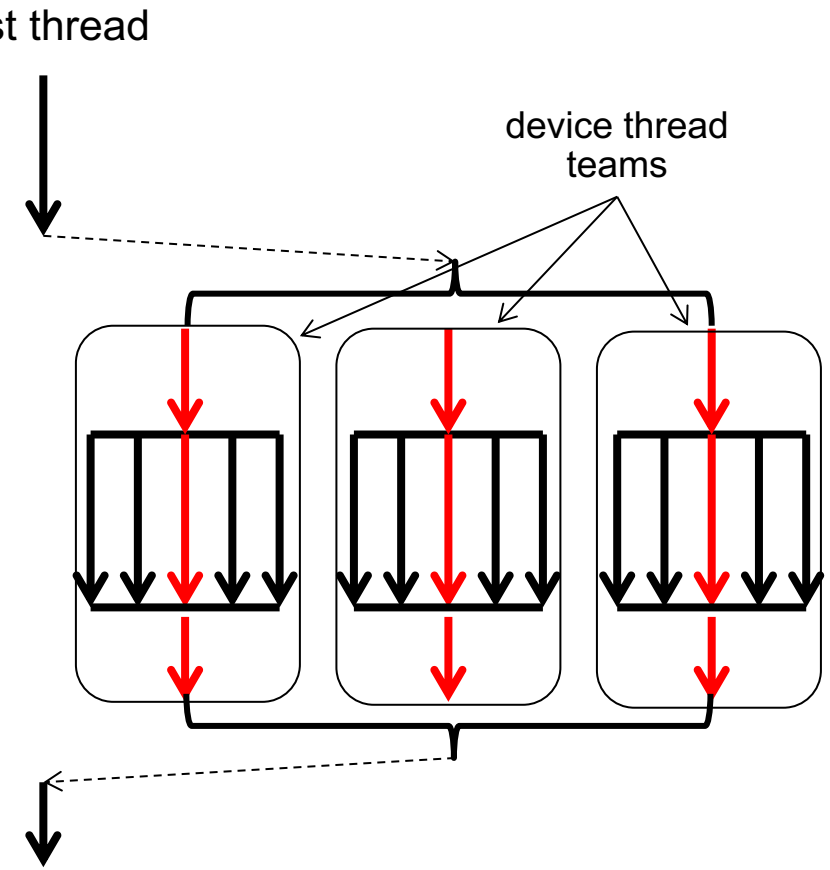


- Transfer execution control to MULTIPLE device initial threads
  - Workshare loop iterations across the initial threads (teams distribute)
- Each initial thread becomes the primary* thread in a thread team
  - Workshare loop iterations across the threads in a team (parallel for)

Create a league of teams and distribute a loop among them and run each team in parallel with its partition of the loop

- loop

```
#pragma omp target
#pragma omp teams
#pragma omp loop
 for (i=0;i<N;i++)
    …
```



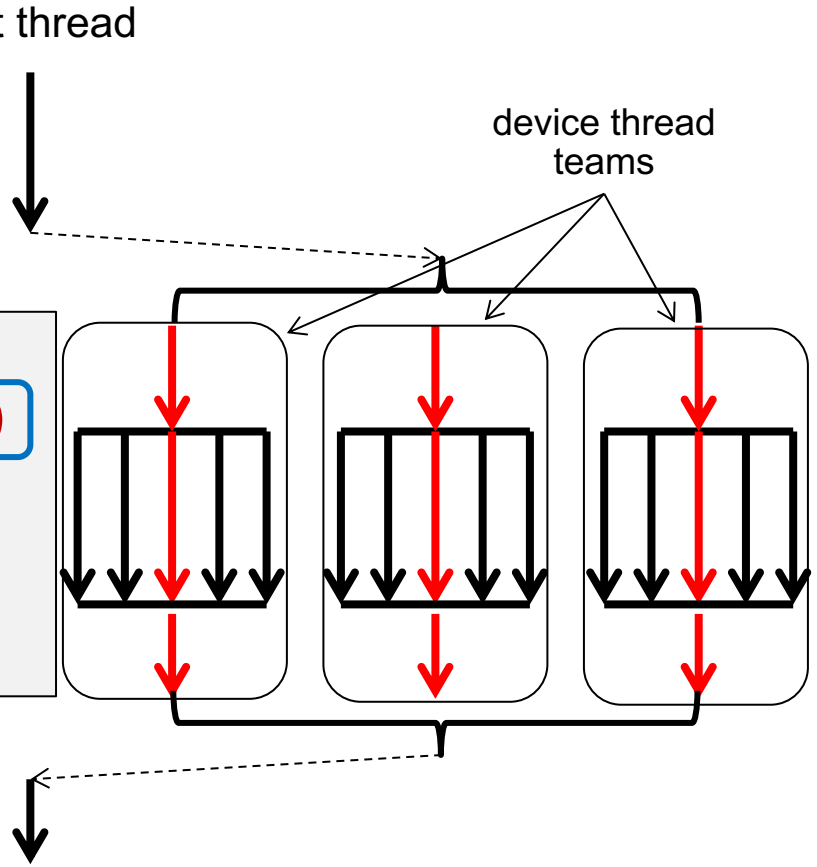
- Transfer execution control to MULTIPLE device initial threads
  - Workshare loop iterations across the initial threads (teams distribute)
- Each initial thread becomes the primary* thread in a thread team
  - Workshare loop iterations across the threads in a team (parallel for)

Create a league of teams and distribute a loop among them and run each team in parallel with its partition of the loop

- teams distribute
- parallel for simd

```
#pragma omp target
#pragma omp teams num_teams(3) thread_limit(5)
#pragma omp distribute
#pragma omp parallel for
 for (i=0;i<N;i++)
   …
```

Explicit control of number and size of teams

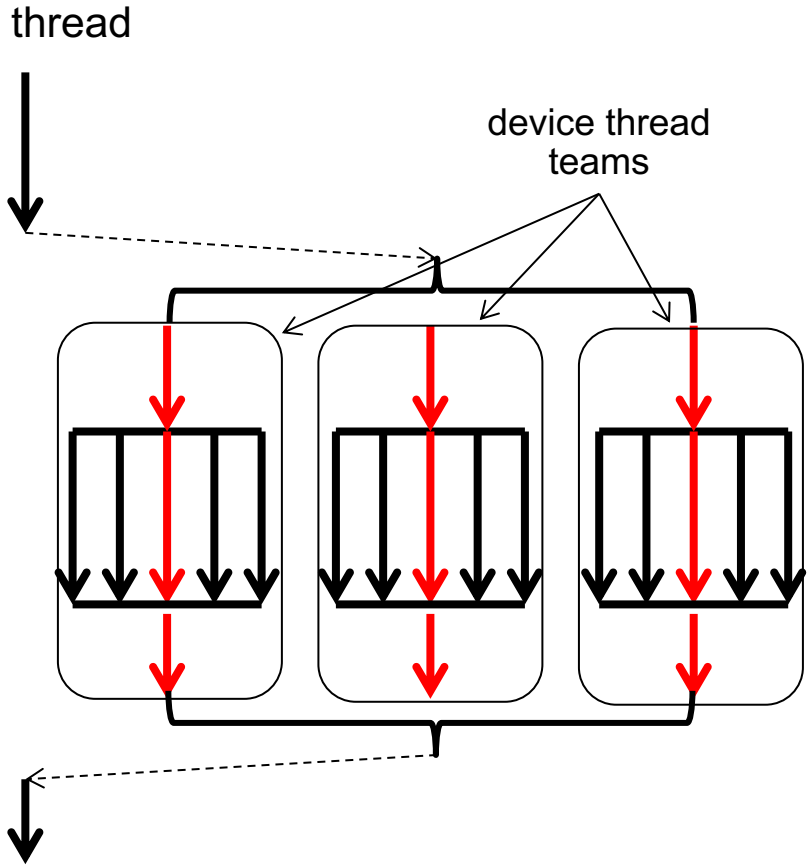


- Transfer execution control to MULTIPLE device initial threads
  - Workshare loop iterations across the initial threads (teams distribute)
- Each initial thread becomes the primary* thread in a thread team
  - Workshare loop iterations across the threads in a team (parallel for)

Create a league of teams and distribute a loop among them and run each team in parallel with its partition of the loop

- Combined construct

```
#pragma omp target  teams loop
for (i=0;i<N;i++)
    …
```



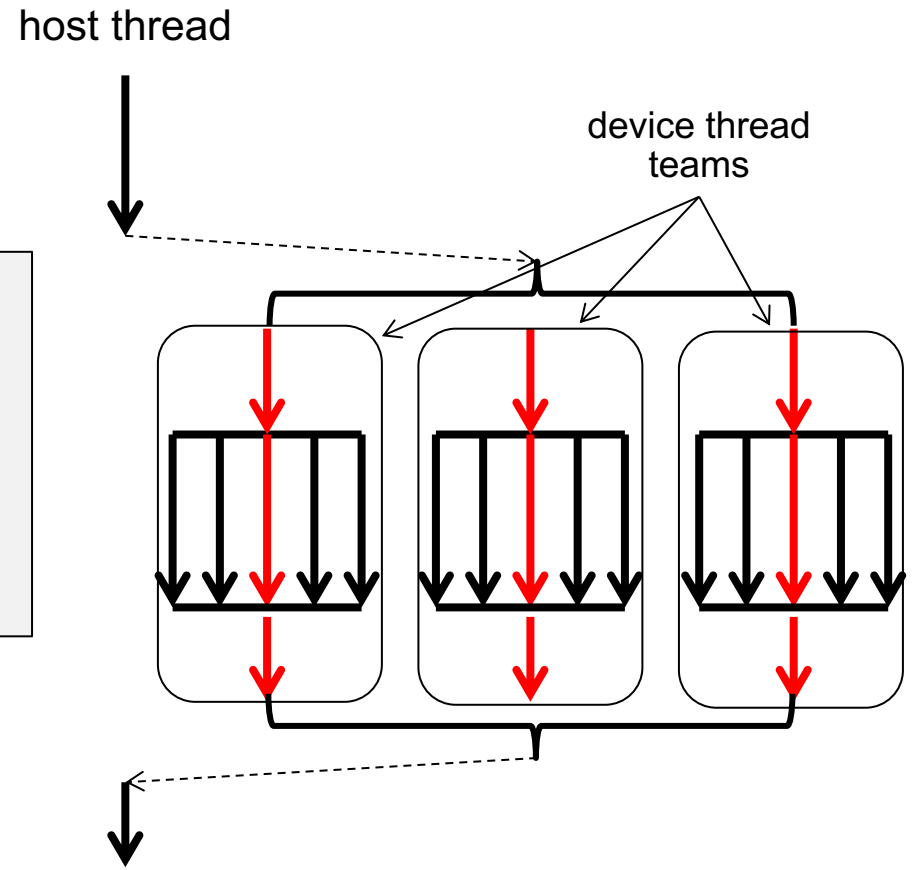
- Transfer execution control to MULTIPLE device initial threads
  - Workshare loop iterations across the initial threads (teams distribute)
- Each initial thread becomes the primary* thread in a thread team
  - Workshare loop iterations across the threads in a team (parallel for)

Create a league of teams and distribute a loop among them and run each team in parallel with its partition of the loop

- teams distribute
- parallel for simd

Works with nested loops as well

```
#pragma omp target
#pragma omp teams distribute
 for (i=0;i<N;i++)
 #pragma omp parallel for
 for (j=0;j<M;i++)
   ...
```



- Transfer execution control to MULTIPLE device initial threads
  - Workshare loop iterations across the initial threads (teams distribute)
- Each initial thread becomes the primary* thread in a thread team
  - Workshare loop iterations across the threads in a team (parallel for)

Exercise

- Consider the HIP code for vector addition.
- Create a version of our vadd.c code that uses OpenMP to explicitly address the work-groups and work-items (as you would do in HIP/CUDA kernel programming).

```
module load rocm
module load amdclang
amdclang –fopenmp --offload-arch=gfx942 vadd.c
sbatch job_gpu.sh
```

gpu_job.sh

```
#!/bin/bash
#SBATCH -p PPAC_MI300A_CPX
#SBATCH -N 1
#SBATCH --gpus=1
#SBATCH -t 10:00

module load rocm amdclang

OMP_TARGET_OFFLOAD=MANDATORY ./a.out
```

… or fetch an interactive session

```
salloc -N1 --gpus=1 -t 10:00 -p PPAC_MI300A_CPX
```

- #pragma omp target
- #pragma omp teams
- #pragma omp distribute
- #pragma omp parallel for
- num_teams(), thread_limit(), simd

vaddHIP.c

```
// Compute sum of length-N vectors: C = A + B
void __global__
vecAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) c[i] = a[i] + b[i];
}
int main () {
    int N = ... ;
    float *a, *b, *c;
    HIPMalloc (&a,  sizeof(float) * N);
  // ... allocate other arrays (b and c)
  // and fill with data

  // Use thread blocks with 256 threads each
    vecAdd <<< (N+255)/256, 256 >>> (a, b, c, N);
}
```

Solution: vadd exposing teams and threads

```
#define BLKSZ 256
#define MIN(a,b)(((a)<(b))?(a)(b))
int main()
{
     float a[N], b[N], c[N], res[N];
     int numTeams = (N+(BLKSZ-1))/BLKSZ;
     int err=0;

   // fill the arrays
   #pragma omp parallel for
   for (int i=0; i<N; i++){
      a[i] = (float)i;
      b[i] = 2.0*(float)i;
      c[i] = 0.0;
      res[i] = i + 2*i;
   }
```

```
// add two vectors
   #pragma omp target
   #pragma omp teams num_teams(numTeams) \
                     thread_limit(BLKSZ)
   #pragma omp distribute
   for (int ib=0;ib<numTeams; ib++){
     #pragma omp parallel for
     for (int i=ib; i<MIN((ib+1)*BLKSZ,N); i++){
       c[i] = a[i] + b[i];
     }
   }
  // test results
  #pragma omp parallel for reduction(+:err)
  for(int i=0;i<N;i++){
     float val = c[i] - res[i];
     val = val*val;
     if(val>TOL) err++;
  }
  printf("vectors added with %d errors\n", err);
  return 0;
}
```

Outline



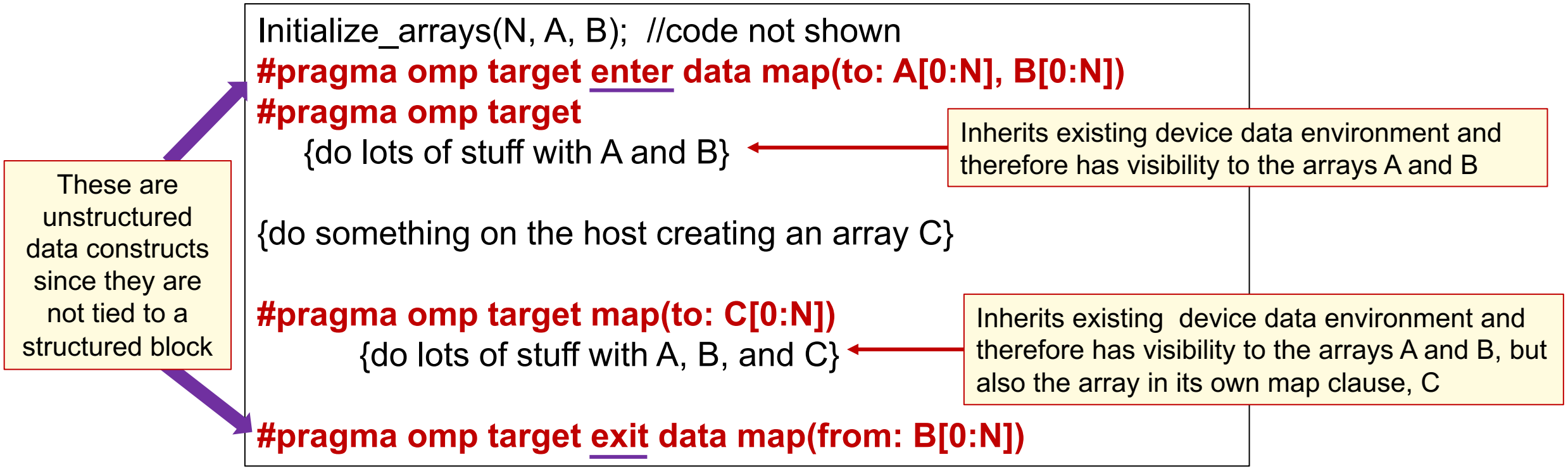
- Introduction to OpenMP
- OpenMP and GPU Programming
- Managing memory on the Host and the GPU
- Fundamental principles of GPU program optimization
- Explicit control of how code maps onto a GPU
- More about Host/GPU memory management
- Using team-local memory: Blocked Matrix Multiplication
- This is the end: Summary/Conclusion
- Extra Content

Target device data environment

- For each device, there is an associated **Device Data Environment**. This is the initial set of variables associated with the device.

- The various **target** directives create new **Target Device Data Environments**. When embedded inside other target constructs, the new Device Data Environments inherit the outer device data environment.

- Consider the two directives we've already encountered for manipulating a Device Data environment



Unstructured Target data directives

- The unstructured target data directives give you a great deal of flexibility on where you map variables between the host and the device.

- For example, consider the following C++ class and the use of unstructured data directives in the class constructor and destructor

```
Class Vector
{
    Vector(int n) {
        len = n;    v = new double[len];
        #pragma omp target enter data map(alloc: v[0:len])
    }

    ~Vector() {
        #pragma omp target exit data map(delete: v[0:len])
        delete[] v;
    }
    private:
        int len;     double* v;
};
```

Allocate memory in the device data environment but do not initialize it

Delete the data and unmap it in the device data environment. Note that the order is important … delete on the device before you delete the variable on the host.

Structured Target data directive

- You can create a target data environment associated with a structured block of code on the host

```
Initialize_arrays(N, A, B);  //code not shown
#pragma omp target data map(to: A[0:N]) map(tofrom:B[0:N])
{
  #pragma omp target
      {do lots of stuff with A and B}

  {do something on the host creating an array C}

  #pragma omp target map(to: C[0:N])
      {do lots of stuff with A, B, and C}

}
```

- The above is equivalent to what we shows earlier with the target enter and exit data directives

Target update directive

- You can update data between target regions with the target update directive.

```
#pragma omp target data map(to: A,B) map(from: C)
{
    #pragma omp target
        {do lots of stuff with A, B and C on the device}

    #pragma omp update from(A)

    host_do_something_with(A)

    #pragma omp update to(A)

    #pragma omp target
        {do lots of stuff with A, B, and C on the device}
}
```

Copy A on the device to A on the host.

Copy A on the host to A on the device.

Target update details

- #pragma omp target update clause[[[,]clause]...]new-line

- creates a target task to handle data movement between the host and a device

- clause is either motion-clause or one of the following:
  - if(scalar-expression)
  - device(integer-expression)
  - nowait
  - depend (dependence-type : list)

- the motion-clause is one of the following:
  - to(list)
  - from(list)

- This directive generates a target task.

- nowait and depend apply to the target task running on the host.

Nested Target directives with overlapping mapped variables

- You can create a target data environment associated with a structured block of code on the host

```
Initialize_arrays(N, A, B);  //code not shown
#pragma omp target data map(to: A[0:N]) map(tofrom:B[0:N])
{
  #pragma omp target map
      {do lots of stuff with A and B}

  {do something on the host creating an array C and modifying A}

  #pragma omp target map(to: C[0:N], A[0:N])
      {do lots of stuff with A, B, and C}

}
```

C is mapped to the device and copied from the host.

What happens with A?

Reference counting of mapped variables.

- For every mapped variable, there is a reference counter.

- For **map(to:A)**, it checks if the variable is present.
  - If the variable is not present on the device …. it sets the reference counter to zero, allocates the memory, initializes the variable on the device with the data from the Host, and increments the reference counter.
  - If the variable is present on the device and the reference counter is one or greater, it does NOT transfer data from the host to the device.

```
Initialize_arrays(N, A, B);  //code not shown
#pragma omp target data map(to: A[0:N]) map(tofrom:B[0:N])
{
  #pragma omp target map
      {do lots of stuff with A and B}

  {do something on the host creating an array C and changing A}

  #pragma omp target map(to: C[0:N], A[0:N])
      {do lots of stuff with A, B, and C}

}
```

C: Variable is not present on the device. Memory is allocated, reference counter set to zero, and data is transferred from the host.

A: reference counter is one. Data is NOT transferred from the host

Reference counting of mapped variables.

- For every mapped variable, there is a reference counter.

- For **map(to:A)**, it checks if the variable is present.
  - If the variable is not present on the device …. it sets the reference counter to zero, allocates the memory, initializes the variable on the device with the data from the Host, and increments the reference counter.
  - If the variable is present on the device and the reference counter is one or greater, it does NOT transfer data from the host to the device.

```
Initialize_arrays(N, A, B);  //code not shown
#pragma omp target data map(to: A[0:N]) map(tofrom:B[0:N])
{
  #pragma omp target map
      {do lots of stuff with A and B}

  {do something on the host creating an array C and changing A}

  #pragma omp target map(to: C[0:N]) map(always, to: A[0:N])
      {do lots of stuff with A, B, and C}

}
```

A's reference counter is one. Data is NOT transferred from the host by default … but you can force a transfer of data with **always** map-type modifier

`map` types and data transfer/allocation/deallocation

Data movement is expensive … so its is important to elide data transfers when semantics allow.
To support this optimization, Data has a reference counters to guide when elision if safe.

map types	Data Not Present on Device			Data Present on Device		
	Allocation	Deallocation	Data transfer	Allocation	Deallocation	Data transfer
`alloc`	YES	NO[1]	NO	Check size[5]	NO[6]	NO[7]
`to`	YES	NO[1]	H->D[2]	Check size[5]	NO[6]	NO[7]
`from`	YES	NO[1]	D->H[3]	Check size[5]	NO[6]	NO[7]
`tofrom`	YES	NO[1]	H->D, D->H[4]	Check size[5]	NO[6]	NO[7]
`release`	deletes if reference count is zero when decrementing, otherwise only decrements reference count (nested maps)					
`delete`	NO	YES	NO	NO	YES	NO

1. For scoped data regions (**target**, **target data**) data is deallocated from the device memory space if it wasn't present at scoped data region entry
2. For scoped data regions **data is transferred from host to device** at data region entry
3. For scoped data regions **data is transferred from device to host** at data region exit
4. For scoped data regions, **data is transferred from host to device** at data region entry and from **device to host** at data region exit
5. For scoped data regions, **reference counter is incremented**
6. For scoped data regions, **reference counter is decremented**
7. This behavior can be modified using the **always** map type modifier

Source: AMD Instinct™ GPU Training @ HLRS/MPCDF

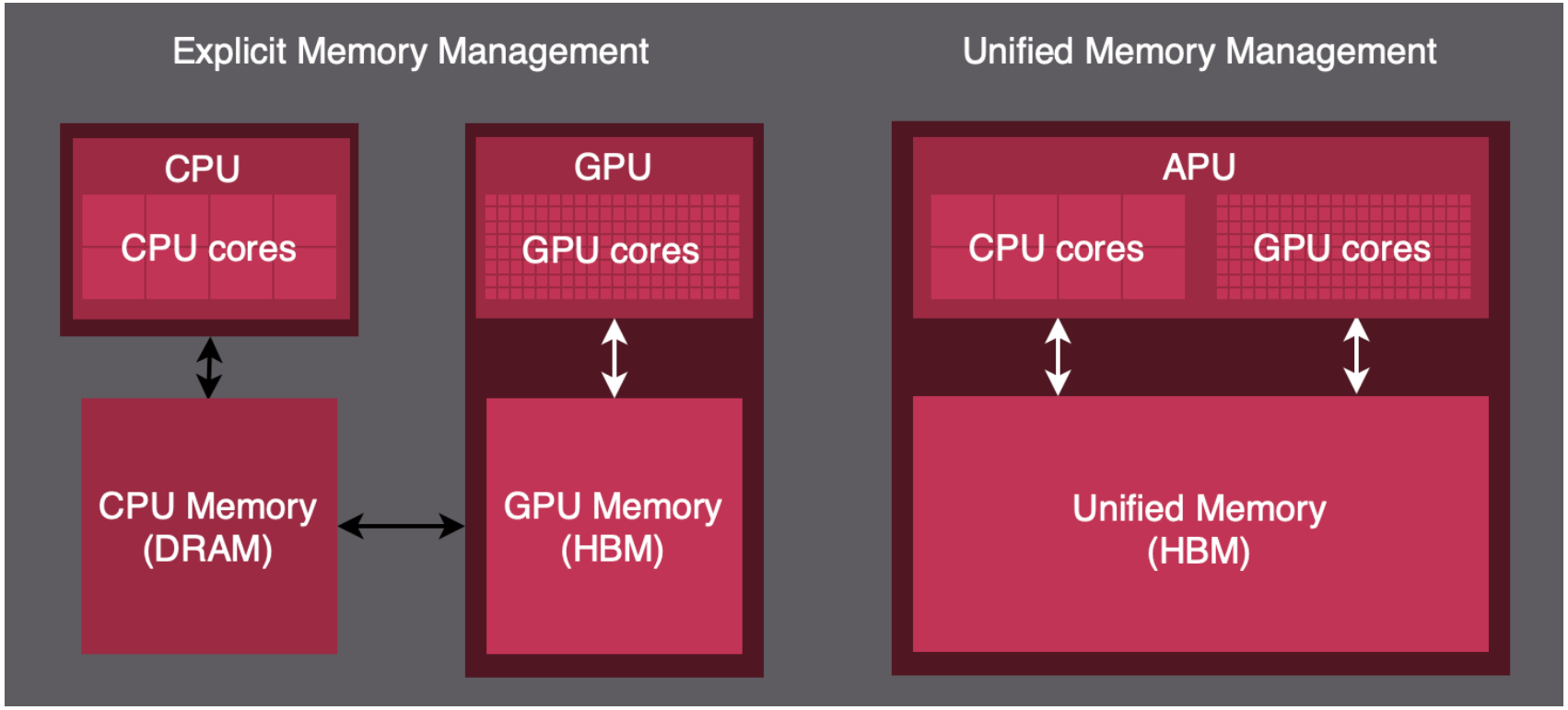
H->D Host to Device D->H Device to Host

Explicit memory management with all this mapping between host and device is hard and error-prone.

With new devices (such as Nvidia Grace Hopper and AMD Instinct™ MI300A) with unified memory shared by the CPU and the GPU, can’t we make this simpler?

AMD unified memory

- AMD has a Heterogenous System Architecture (HAS) that integrates the CPU and GPU in a single package. They call these sorts of systems an accelerated processing unit or APU.
- APUs utilize unified memory through the hipMallocManaged() memory allocator.

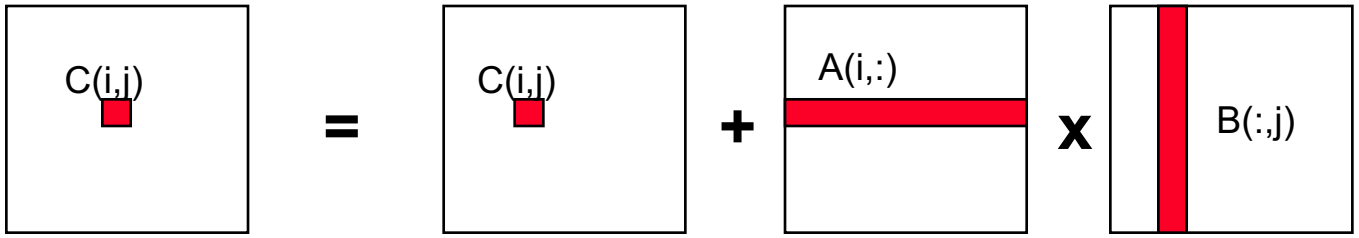


- For AMD MI200 and MI300 series products, data allocated through standard language-level memory allocators (e.g. malloc) are placed in unified memory but only if the environment variable HAS_XNACK=1 is set.

Image source: https://rocm.docs.amd.com/projects/HIP/en/docs-6.1.2/how-to/unified_memory.html

Does Unified Memory work? Consider matrix multiplication

- Consider Matrix Multiplication and the basic triply-nested-loop algorithm
- This computes a dot product or a row from A and a column from B for each element of the product matrix C.



- The serial code is straightforward

```
void mat_mult(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C)
{
   for (int i=0; i<Ndim; i++){
       for (int j=0; j<Mdim; j++)
           for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

$C \in R^{Ndim,Mdim}$

$A \in R^{Ndim,Pdim}$

$B \in R^{Pdim,Mdin}$

Does Unified Memory work? Consider matrix multiplication

- Using the OpenMP GPU directives for managing memory and mapping execution onto a GPU:

```
void mat_mult(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C)
{
#pragma omp target map(tofrom:C[0:Ndim*Mdim]) map(to:B[0:Pdim*Mdim],A[0:Ndim*Pdim])
#pragma omp teams distribute parallel for collapse(2)
  for (int i=0; i<Ndim; i++){
     for (int j=0; j<Mdim; j++)
          for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

$C \in R^{Ndim,Mdim}$

$A \in R^{Ndim,Pdim}$

$B \in R^{Pdim,Mdin}$

- If we assume unified memory between the Host (where arrays are allocated with malloc) and the GPU, we can leave off all the map clauses: **(export HSA_XNACK=1 before running the executable)**

```
void mat_mult(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){
{
#pragma omp target
#pragma omp teams distribute parallel for collapse(2)
  for (int i=0; i<Ndim; i++){
     for (int j=0; j<Mdim; j++)
          for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

Yes, Unified Memory works

- Matrix Multiplication performance

  $C \in R^{512,4096}$ $A \in R^{512,4096}$ $B \in R^{4096,4096}$

  - GPU with map clauses: **168,481 MFLOPS**
  - GPU with unified memory: **169,820 MFLOPS**

Best of five runs on a single XCD of an AMD Instinct™ MI300A GPU (in the PPAC_MI300A_CPX queue) and a 24-core Zen 4 CPU in the AMD AAC6 cluster.

THESE ARE NOT OFFICIAL BENCHMARKS.

No effort was made to optimize the programs or run them on dedicated systems set aside for proper benchmarking. The results were NOT formally validated.

Does Unified Memory work? Consider matrix multiplication

- Using the OpenMP GPU directives for managing memory and mapping execution onto a GPU:

```
void mat_mult(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C)
{
#pragma omp target map(tofrom:C[0:Ndim*Mdim]) map(to:B[0:Pdim*Mdim],A[0:Ndim*Pdim])
#pragma omp teams distribute parallel for collapse(2)
  for (int i=0; i<Ndim; i++){
    for (int j=0; j<Mdim; j++)
      for(int k=0;k<Pdim;k++)
        *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

$C \in R^{Ndim,Mdim}$

$A \in R^{Ndim,Pdim}$

$B \in R^{Pdim,Mdin}$

- If we assume unified memory between the Host (where arrays are allocated with malloc) and the GPU, we can leave off all the map clauses: **(export HSA_XNACK=1 before running the executable)**

```
void mat_mult(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){
{
#pragma omp target
#pragma omp teams distribute parallel for collapse(2)
  for (int i=0; i<Ndim; i++){
    for (int j=0; j<Mdim; j++)
      for(int k=0;k<Pdim;k++)
        *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

Unified Memory works … good performance with greatly simplified programming, especially for problems that involve complex data structures used in work split between the CPU and the GPU

MFLOPS: Millions of Floating-Point Operations per Second

An MI300A GPU has 6 XCD chiplets

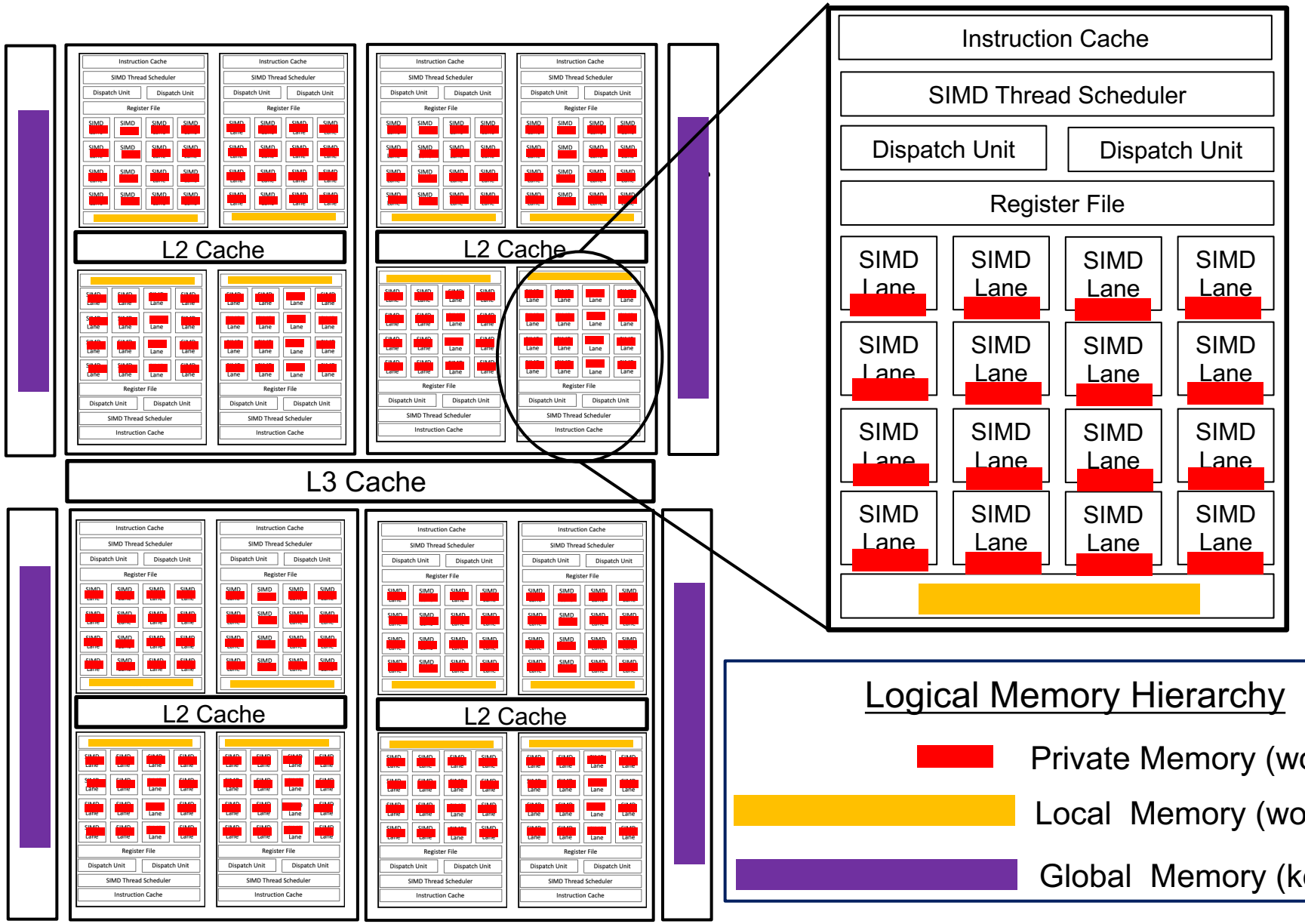
Outline



- Introduction to OpenMP
- OpenMP and GPU Programming
- Managing memory on the Host and the GPU
- Fundamental principles of GPU program optimization
- Explicit control of how code maps onto a GPU
- More about Host/GPU memory management
- Using team-local memory: Blocked Matrix Multiplication
- This is the end: Summary/Conclusion
- Extra Content

Back to the Generic GPU (following Hennessey and Patterson)

A GPU has distinct memories visible at different levels of execution



OpenMP memory spaces and allocators

- OpenMP provides control of **where** data is allocated using **allocators**
- Originally designed for systems with deeper memory hierarchy than just “main memory”
  (E.g., Combination of DDR and HBM such as on Intel Xeon Phi or Intel Sapphire Rapids)
  - **omp_high_bw_mem_alloc** allocates in the high bandwidth memory space
  - **omp_low_lat_mem_alloc** allocates in the low latency memory space
- The system and OpenMP implementation define what those spaces mean on a given platform
- These pre-defined allocators are used in the **omp_alloc()** library call and the **allocate()** clause

How does the concept of memory spaces and allocators in OpenMP translate to the GPU?

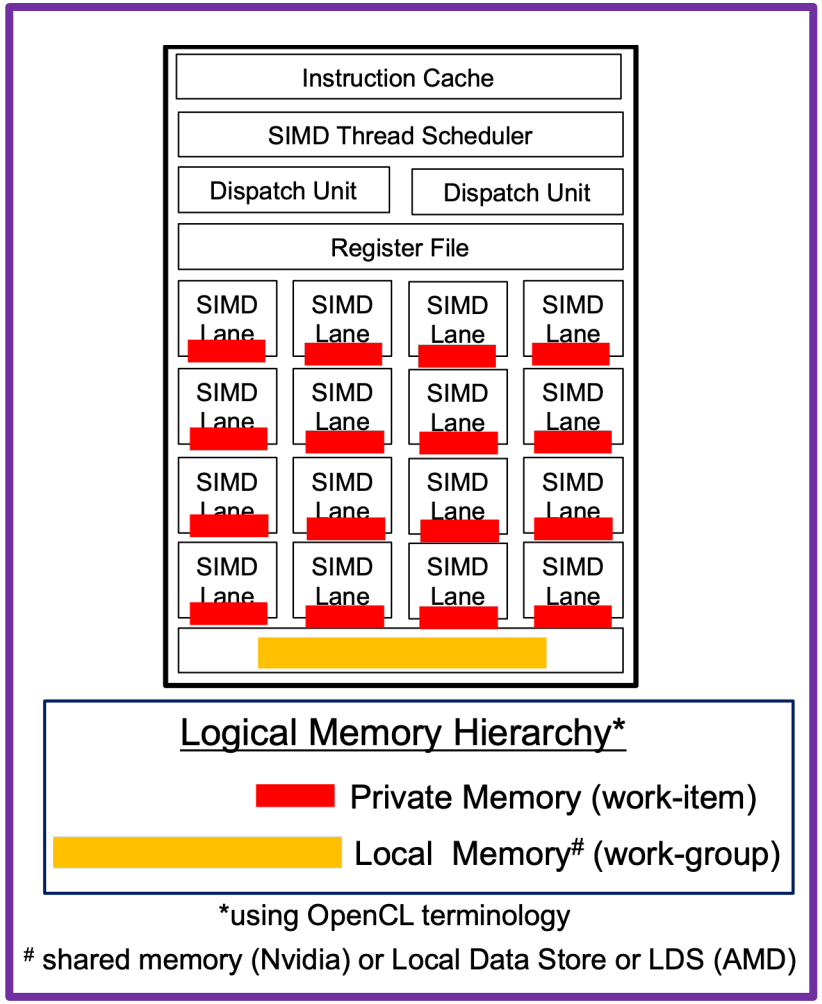
Xeon Phi: A many-core CPU with wide vector units
DRAM: Dynamic Random Access Memory
Sapphire Rapids: A multicore CPU microarchitecture
DDR: Double Data Rate DRAM
HBM: High Bandwidth Memory from stacked DRAM chips

OpenMP allocators for team-only[&] memory

- omp_pteam_mem_alloc
  - Data allocated in this memory space is available to threads in the team where one of those threads allocated the data

- omp_cgroup_mem_alloc
  - Data allocated in this memory space is available to threads in the team where one of those threads allocated the data, AND, and child threads
  - OpenMP threads that satisfy that condition are said to be in a *contention group*

- On current* implementations of OpenMP targeting GPUs, these target the same local/shared memory for each team … so we will only work with the **omp_pteam_mem_alloc** allocator



*As of 2024

[&]**Team-only** memory is also known as **Local Memory** or **Local Data Store** or **Shared memory**

Putting data in team-only memory

- Three step process:
  1. Add the **uses_allocators(omp_pteam_mem_alloc)** clause to the target construct
  2. Privatize the variables with the **private()** clause on the **teams** construct
  3. Use the **allocate()** clause to select the allocator used for teams-private variables

```
double arr[M];

#pragma omp target uses_allocators(omp_pteam_mem_alloc)
#pragma omp teams allocate(omp_pteam_mem_alloc: arr) private(arr)
#pragma omp distribute
for (int i = 0; i < N; ++i) {
  #pragma omp parallel for
  for (int j = 0; j < M; ++j) {
    arr[j] = 1.0;
    // do something interesting (not shown) with arr knowing its local
    // to the threads in a team and in memory close to those threads
  }
}
```

Notify system that the indicated allocator will be used with the kernel being offloaded to the device.

Give each team its own copy of **arr**

Loop over teams

One thread in the team will allocate **arr** and it will be shared with all the threads in the team

Loop over threads in each team

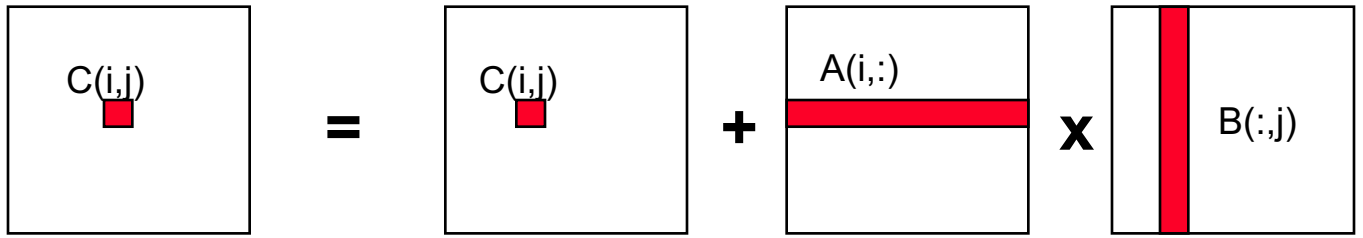
Let’s demonstrate how to use team-only memory with a practical operation … matrix multiplication

Matrix Multiplication:

- Matrix Multiplication cost: For order N matrices:
  - Floating Point Operations (FLOPS): $2*N^3 \rightarrow O(N^3)$ FLOPS
  - Memory Operations, $3*N^2$ floating point numbers $\rightarrow O(N^2)$ memory operations

- To optimize matrix multiplication, we must ensure that for every memory access we execute as many FLOPS as possible. Since $O(N^3) >> O(N^2)$, If we do things right, we can hide memory movement and approach peak performance on the device (which is why HPC people like this operation so much)

- There are better algorithms, but we'll keep things simple and consider the simple dot-product algorithm.



Dot-product of a row of A and a column of B for each element of C

- If we tile/block the loops, we can greatly improve data reuse

Matrix multiplication … serial code … C+= A*B

$C \in R^{N,M}$

$A \in R^{N,P}$

$B \in R^{P,M}$

```
for (int i = 0; i < N; ++i) {
  for (int j = 0; j < M; ++j) {
    for (int k = 0; k < P; ++k) {
      C[i * M + j] += A[i * P + k] * B[k * M + j];
    }
  }
}
```

Matrix multiplication … serial code … C+= A*B

$C, A, B \in R^{N,N}$

```
for (int i = 0; i < N; ++i)
  for (int j = 0; j < N; ++j)
    for (int k = 0; k < N; ++k)
      C[i * N + j] += A[i * N + k] * B[k * N + j];
```

Let's simplify things by specializing to square matrices of order Ndim.

Also, let's get rid of the extra brackets.

Blocked Matrix multiplication … C+= A*B

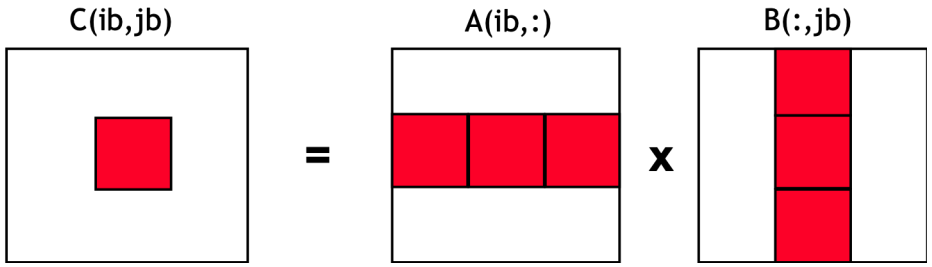
$C, A, B \in R^{N,N}$

Assume N%Bsize = 0

```
#define Bsize = 8     // the size of a block in each dimension
int Nblk = N/blksz;   // number of blocks

for (int ib = 0; i < Nblk; ++i)
      for(int i = ib; i<(ib+1)*Nblk; i++)
  for (int jb = 0; j < Nblk; ++j)
        for(int j = jb; j<(jb+1)*Nblk; j++)
    for (int kb = 0; k < Nblk; ++k)
          for(int k = kb; k<(kb+1)*Nblk; k++)
             C[i * N + j] += A[i * N + k] * B[k * N + j];
```

Split each of our three loops (i,j,and k) into a pair of loops … one over blocks and the other over indices inside a block



Blocked Matrix multiplication … C+= A*B

$C, A, B \in R^{N,N}$

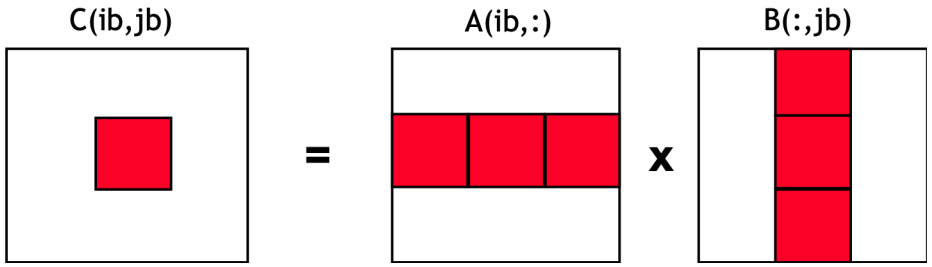
Assume N%Bsize = 0

```
#define Bsize = 8     // the size of a block in each dimension
int Nblk = N/blksz;   // number of blocks

for (int ib = 0; i < Nblk; ++i)
  for (int jb = 0; j < Nblk; ++j)
    for (int kb = 0; k < Nblk; ++k)

      for(int i = ib; i<(ib+1)*Nblk; i++)
        for(int j = jb; j<(jb+1)*Nblk; j++)
          for(int k = kb; k<(kb+1)*Nblk; k++)
             C[i * N + j] += A[i * N + k] * B[k * N + j];
```

Rearrange so we have two clusters of loops … one over blocks and the other for operations inside a block



Blocked Matrix multiplication … C+= A*B

$C, A, B \in R^{N,N}$

Assume N%Bsize = 0

```
#define Bsize = 8     // the size of a block in each dimension
int Nblk = N/blksz;   // number of blocks

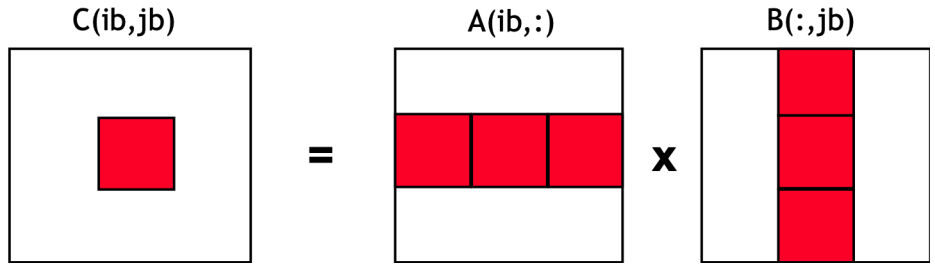
for (int ib = 0; i < Nblk; ++i)
  for (int jb = 0; j < Nblk; ++j)
    for (int kb = 0; k < Nblk; ++k)
```

Map Loops over blocks onto teams running in parallel across the GPU's compute units

```
for(int i = ib; i<(ib+1)*Nblk; i++)
  for(int j = jb; j<(jb+1)*Nblk; j++)
    for(int k = kb; k<(kb+1)*Nblk; k++)
      C[i * N + j] += A[i * N + k] * B[k * N + j];
```

GEMM computation inside each team

Rearrange so we have two clusters of loops … one over blocks and the other for operations inside a block



GEMM: General Matrix Multiply. This term comes from the BLAS, Basic Linear Algebra Subprograms.

Parallel Blocked Matrix multiplication … C+= A*B $C, A, B \in R^{N,N}$

Assume N%Bsize = 0

```
#define Bsize = 8
int Nblk = N/blksz;  Nsqu = N*N;
#pragma omp target data map(to:A[0:Nsqu],B[0:Nsqu]) map(tofrom:C[0:Nsqu])
{
   for (int ib = 0; i < Nblk; ++i)
     for (int jb = 0; j < Nblk; ++j)
       for (int kb = 0; k < Nblk; ++k)
```

Let's map arrays into a data environment on the device.

```
for(int i = ib; i<(ib+1)*Nblk; i++)
  for(int j = jb; j<(jb+1)*Nblk; j++)
    for(int k = kb; k<(kb+1)*Nblk; k++)
       C[i * N + j] += A[i * N + k] * B[k * N + j];
```

GEMM computation inside each team

We can have each team copy blocks of A and B into team-only (local) memory. Then the core computation will work from fast memory and go much faster

```
}
```

Parallel Blocked Matrix multiplication … C+= A*B $C, A, B \in R^{NxN}$

Assume N%Bsize = 0

```
#define Bsize = 8
int Nblk = N/blksz;  Nsqu = N*N;
#pragma omp target data map(to:A[0:Nsqu],B[0:Nsqu]) map(tofrom:C[0:Nsqu])
{  double Awrk[Bsize*Bsize], Bwrk[Bsize*Bsize];
   #pragma omp target uses_allocators(omp_pteam_mem_alloc)
   #pragma omp teams distribute collapse(2) num_teams(Nblk*Nblk) thread_limit(Bsize*Bsize)\
               allocate(omp_pteam_mem_alloc: Awrk, Bwrk) private(Awrk, Bwrk)
   for (int ib = 0; i < Nblk; ++i)
     for (int jb = 0; j < Nblk; ++j)
       for (int kb = 0; k < Nblk; ++k)
```

Offload the kernel to the GPU, setup local memory for the arrays Awrk and Bwrk, and distribute blocks (one to each team)

```
for(int i = ib; i<(ib+1)*Nblk; i++)
  for(int j = jb; j<(jb+1)*Nblk; j++)
    for(int k = kb; k<(kb+1)*Nblk; k++)
       C[i * N + j] += A[i * N + k] * B[k * N + j];
```

GEMM computation inside each team

```
}
```

Parallel Blocked Matrix multiplication: C+= A*B

$C, A, B \in R^{NxN}$

Assume N%Bsize = 0

```
#define Bsize 8
void mm_gpu_block(int Ndim, double *A, double *B, double *C){
   int Nblk = Ndim / Bsize;   int Nsqu = Ndim*Ndim;
  #pragma omp target data map(to:A[0:Nsqu],B[0:Nsqu]) map(tofrom:C[0:Nsqu])
  {
    double Awrk[Bsize*Bsize];  double Bwrk[Bsize*Bsize];
    #pragma omp target uses_allocators(omp_pteam_mem_alloc)
    #pragma omp teams distribute collapse(2) num_teams(Nblk*Nblk) thread_limit(Bsize*Bsize)\
              allocate(omp_pteam_mem_alloc: Awrk, Bwrk) private(Awrk, Bwrk)
    for (int ib=0; ib < Nblk; ib++)
      for (int jb=0; jb < Nblk; jb++)
        for (int kb=0; kb<Nblk; kb++)
    #pragma omp parallel num_threads(Bsize*Bsize)
    {
         #pragma omp for collapse(2) nowait
          for (int i=ib*Bsize; i<((ib+1)*Bsize); i++)
            for(int k=kb*Bsize; k<((kb+1)*Bsize); k++)
              Awrk[(i%Bsize)*Bsize + (k%Bsize)] = A[i*Ndim+k];
          #pragma omp for collapse(2)
          for (int j=jb*Bsize; j<((jb+1)*Bsize); j++)
            for(int k=kb*Bsize; k<((kb+1)*Bsize); k++)
              Bwrk[(k%Bsize)*Bsize + (j%Bsize)] = B[k*Ndim+j];
          #pragma omp for collapse(2)
          for (int i=ib*Bsize; i<((ib+1)*Bsize); i++)
            for (int j=jb*Bsize; j<((jb+1)*Bsize); j++)
              for(int k=kb*Bsize; k<((kb+1)*Bsize); k++)
                 *(C+(i*Ndim+j)) += Awrk[(i%Bsize)*Bsize + (k%Bsize)] * Bwrk[(k%Bsize)*Bsize + (j%Bsize)];
    } // end parallel
  } //end target data
}
```

Each team copies relevant block of A and B into local work arrays Awrk and Bwrk

The GEMM operation with memory references are to fast local memory.

Matrix Multiplication Performance

- Compare blocked matrix multiply code to non-blocked GPU code and CPU code

```
void mm_gpu(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){

#pragma omp target map(tofrom:C[0:Ndim*Mdim]) map(to:B[0:Pdim*Mdim],A[0:Ndim*Pdim])
#pragma omp teams distribute parallel for
  for (int i=0; i<Ndim; i++){
     for (int j=0; j<Mdim; j++)
          for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

```
void mm_cpu(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){

#pragma omp parallel for collapse(2)
  for (int i=0; i<Ndim; i++)
     for (int j=0; j<Mdim; j++)
          for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

Matrix Multiplication Performance

- Performance running on a single XCD of an AMD Instinct™ MI300A GPU (in the PPAC_MI300A_CPX queue) and a 24-core Zen 4 CPU in the AMD AAC6 cluster.

- For square matrices A, B, and C:
  - CPU 8 threads: 2,524 MFLOPS
  - GPU: 7,218 MFLOPS
  - GPU Blocked: **288,064 MFLOPS**

$$A, B, C \in R^{1024,1024}$$

- For the more general case (non-square):
  - CPU 16 threads: 1,295 MFLOPS
  - GPU: 3,707 MFLOPS
  - GPU Blocked: **324,405 MFLOPS**

$$C \in R^{512,4096}$$
$$A \in R^{512,4096}$$
$$B \in R^{4096,4096}$$

The Blocked Algorithm is dramatically faster!!!!

Actually … it's too much faster. What's wrong with the GPU non-blocked case?

THESE ARE NOT OFFICIAL BENCHMARKS.

These numbers are indicative of the sort of performance gains observed when moving from a CPU to a GPU to blocked algorithms for dense linear algebra. No effort was made to optimize the programs or run them on dedicated systems set aside for proper benchmarking. The results were NOT formally validated. We did not optimize the CPU code for well-known NUMA effects on a Zen 4 CPU.

MFLOPS: Millions of Floating-Point Operations per Second

An MI300A GPU has 6 XCD chiplets

Matrix Multiplication Performance

- Compare blocked matrix multiply code to non-blocked GPU code and CPU code

```
void mm_gpu(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){

#pragma omp target map(tofrom:C[0:Ndim*Mdim]) map(to:B[0:Pdim*Mdim],A[0:Ndim*Pdim])
#pragma omp teams distribute parallel for
  for (int i=0; i<Ndim; i++){
     for (int j=0; j<Mdim; j++)
          for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

I did not use collapse(2) for GPU so I did not have enough work to fully occupy the GPU

```
void mm_cpu(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){

#pragma omp parallel for collapse(2)
  for (int i=0; i<Ndim; i++)
     for (int j=0; j<Mdim; j++)
          for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

Matrix Multiplication Performance

- Compare blocked matrix multiply code to non-blocked GPU code and CPU code

```
void mm_gpu(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){

#pragma omp target map(tofrom:C[0:Ndim*Mdim]) map(to:B[0:Pdim*Mdim],A[0:Ndim*Pdim])
#pragma omp teams distribute parallel for collapse(2)
  for (int i=0; i<Ndim; i++){
     for (int j=0; j<Mdim; j++)
          for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

```
void mm_cpu(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){

#pragma omp parallel for collapse(2)
  for (int i=0; i<Ndim; i++)
     for (int j=0; j<Mdim; j++)
          for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

Matrix Multiplication Performance

- Performance running on a single XCD of an AMD Instinct™ MI300A GPU (in the PPAC_MI300A_CPX queue) and a 24-core Zen 4 CPU in the AMD AAC6 cluster.

- For square matrices A, B, and C:
  - CPU 8 threads: 2,524 MFLOPS
  - GPU: **171,552 MFLOPS**
  - GPU Blocked: **288,064 MFLOPS**

$$A, B, C \in R^{1024,1024}$$

- For the more general case (non-square):
  - CPU 16 threads: 1,295 MFLOPS
  - GPU: **164,477 MFLOPS**
  - GPU Blocked: **324,405 MFLOPS**

$$C \in R^{512,4096}$$
$$A \in R^{512,4096}$$
$$B \in R^{4096,4096}$$

The Blocked Algorithm is significantly faster!!!!

But fixing the occupancy problem radically improved the non-blocked algorithm

The CPU numbers are still really bad

THESE ARE NOT OFFICIAL BENCHMARKS.

These numbers are indicative of the sort of performance gains observed when moving from a CPU to a GPU to blocked algorithms for dense linear algebra. No effort was made to optimize the programs or run them on dedicated systems set aside for proper benchmarking. The results were NOT formally validated. We did not optimize the CPU code for well-known NUMA effects on a Zen 4 CPU.

MFLOPS: Millions of Floating-Point Operations per Second

An MI300A GPU has 6 XCD chiplets

Matrix Multiplication Performance

- Compare blocked matrix multiply code to non-blocked GPU code and CPU code

```
void mm_gpu(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){

#pragma omp target map(tofrom:C[0:Ndim*Mdim]) map(to:B[0:Pdim*Mdim],A[0:Ndim*Pdim])
#pragma omp teams distribute parallel for collapse(2)
  for (int i=0; i<Ndim; i++){
     for (int j=0; j<Mdim; j++)
          for(int k=0;k<Pdim;k++)
               *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

```
void mm_cpu(int Ndim, int Mdim, int Pdim, double *A, double *B, double *C){

#pragma omp parallel for collapse(2)
  for (int i=0; i<Ndim; i++)
          for(int k=0;k<Pdim;k++)
     for (int j=0; j<Mdim; j++)
          *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
}
```

NUMA Optimization (first touch, PLACES=cores, PROC_BIND=spread) didn't help at all.

Restrict keyword for vectorization didn't help

Swapped loop order to ikj. This made a HUGE difference. I did this for the GPU programs too, but it broke the programs (which makes no sense) so I kept them with the ijk loop order.

Matrix Multiplication Performance

- Performance running on a single XCD of an AMD Instinct™ MI300A GPU (in the PPAC_MI300A_CPX queue) and a 24-core Zen 4 CPU in the AMD AAC6 cluster.

- For square matrices A, B, and C:
  - CPU 8 threads, ikj: **39,961 MFLOPS**
  - GPU: **171,552 MFLOPS**
  - GPU Blocked: **288,064 MFLOPS**

$$A, B, C \in R^{1024,1024}$$

All cases are now well optimized … so this is our final set of numbers

- For the more general case (non-square):
  - CPU 8 threads, ikj: **24,388 MFLOPS**
  - GPU: **164,477 MFLOPS**
  - GPU Blocked: **324,405 MFLOPS**

$$C \in R^{512,4096}$$
$$A \in R^{512,4096}$$
$$B \in R^{4096,4096}$$

THESE ARE NOT OFFICIAL BENCHMARKS.

These numbers are indicative of the sort of performance gains observed when moving from a CPU to a GPU to blocked algorithms for dense linear algebra. No effort was made to optimize the programs or run them on dedicated systems set aside for proper benchmarking. The results were NOT formally validated. We did not optimize the CPU code for well-known NUMA effects on a Zen 4 CPU.

MFLOPS: Millions of Floating-Point Operations per Second

An MI300A GPU has 6 XCD chiplets

Outline



- Introduction to OpenMP
- OpenMP and GPU Programming
- Managing memory on the Host and the GPU
- Fundamental principles of GPU program optimization
- Explicit control of how code maps onto a GPU
- More details on managing Host/GPU memory
- Using team-local memory: Blocked Matrix Multiplication
- This is the end: Summary/Conclusion
- Extra Content

There is MUCH more … beyond what have time to cover

- Do as much as you can with a simple loop construct. It's portable and as compilers improve over time, it will keep up with compiler driven performance improvements.

- But sometimes you need more:
  - Management of data movement across the memory hierarchy: global vs. shared vs. private …
  - Calling optimized math libraries (such as cuBLAS)
  - Multi-device programming
  - Interfacing the OpenMP runtime to external/foreign runtimes
  - Asynchrony and Directed Acyclic Graphs (DAG) of kernel executions

- Ultimately, you may need to master all those advanced features of GPU programming. But start with loop. Start with how data on the host maps onto the device (i.e. the GPU). Master that level of GPU programming before worrying about the complex stuff.

The book, "Programming your GPU with OpenMP", Tom Deakin and Tim Mattson, MIT Press, 2023 covers these additional topics.

Summary

- Portable Parallel computing for a GPU works … if you use an open standard such as OpenMP.

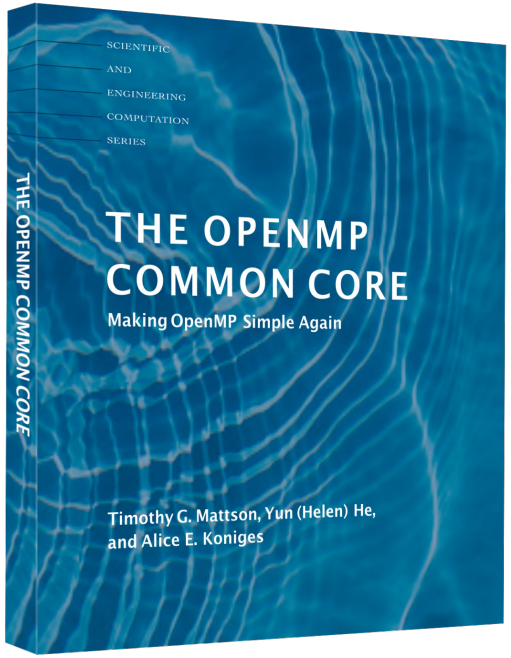
- It can be as easy as changing
  - #pragma omp parallel for
- To a GPU variant
  - #pragma omp target teams loop

- Of course there is more work to do in practice, but that’s mostly just a matter of optimizing the kernels to minimize memory movement.

To learn more about OpenMP

The OpenMP web site has a great deal of material to help you with OpenMP www.openmp.org
Reading the spec is painful … but each spec has a collection of examples. Study the examples, don't try to read the specs

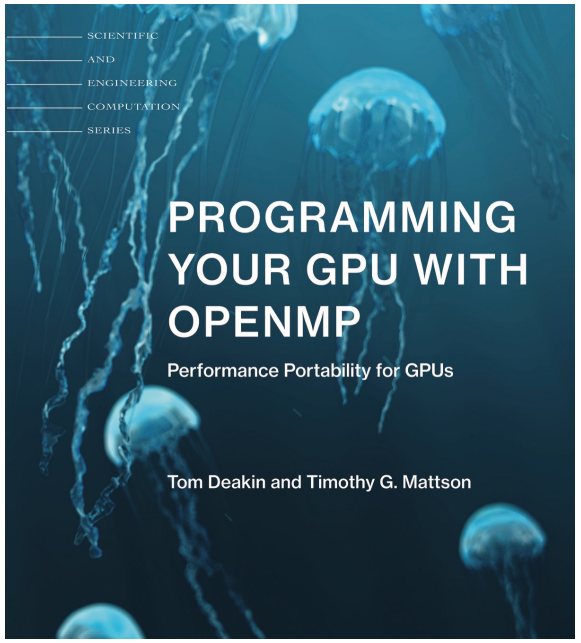
Since the specs are written ONLY for implementors … programmers need the OpenMP Books to master OpenMP.



Start here … learn the basics and build a foundation for the future



Learn advanced features in OpenMP including tasking and GPU programming (up to version 4.5)



Learn all the details of GPU programming with OpenMP
(up to version 5.2)
Released in November 2023

OpenMP BOF: Wed. @ 5:15pm, Rm. W180

OpenMP Booth #5217:

- **Daily Book Drawings:**
  (**Mon**: 8:30pm, **Tues**: 5:30, **Wed**: 4:45, **Thu**: 2:30)
- **Beverages: Tuesday @ 4pm**
- **Stickers, Lanyards, Badge Ribbons**
- **Meet OpenMP Experts…**





Ask an OpenMP Expert in Booth #5217

Bring your toughest questions!



Michael Klemm



Tim Mattson



Ruud van der Pas



Steven Olivier

Schedule and details at link.openmp.org/sc26experts

Backup ... and a bit of extra content

- A quick look at other GPU Application Programming Interfaces (API)
- The Jacobi solver case study
- Writing functions to call from inside a kernel

SIMT Programming models: it's more than just OpenMP

- CUDA:
  - Released ~2006. Made GPGPU programming "mainstream" and continues to drive innovation in SIMT programming.
    - Downside: proprietary to NVIDIA
- HIP
  - HIP is a C++ runtime API and kernel language for AMD GPUs. It allows developers to create portable applications by providing a programming interface that is similar to NVIDIA CUDA, making it straightforward to port existing NVIDIA CUDA code to run on AMD hardware. Open Source under and MIT license
- OpenCL:
  - Open Standard for SIMIT programming created by Apple, Intel, NVIDIA, AMD, and others. 1st release in 2009.
  - Supports CPUs, GPUs, FPGAs, and DSP chips. The leading cross platform SIMT model.
    - Downside: extreme portability means verbose API. Painfully low level especially for the host-program.
- Sycl:
  - C++ abstraction layer implements SIMT model with kernels as lambdas. Closely aligned with OpenCL. 1st release 2014
- Directive driven programming models:
  - **OpenACC**: they split from an OpenMP working group to create a competing directive driven API emphasizing descriptive (rather than prescriptive) semantics.
    - Downside: After starting as an NVIDIA API, its now open but still largely tied to NVIDIA.
  - **OpenMP**: Mixes multithreading and SIMT. Semantics are prescriptive which makes it more verbose. A truly Open standard supported by all the key GPU players..

Third party names are the property of their owners

Vector addition with CUDA

```
// Compute sum of length-N vectors: C = A + B
__global__ void
vecAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) c[i] = a[i] + b[i];
}

int main () {
    int N = ... ;
    float *a, *b, *c;
    cudaMalloc (&a,  sizeof(float) * N);
// ... allocate other arrays (b and c), fill with data

  // Use thread blocks with 256 threads each
    vecAdd <<< (N+255)/256, 256 >>> (a, b, c, N);
}
```

CUDA kernel as function

Unified shared memory … allocate on host, visible on device too

Enqueue the kernel to execute on the Grid

Note: Not all GPUs support a unified shared memory. We assume its available here so we don't need to clutter the code with explicit data movement.

Vector addition with ~~CUDA~~ → HIP

```
// Compute sum of length-N vectors: C = A + B
__global__ void
vecAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) c[i] = a[i] + b[i];
}

int main () {
    int N = ... ;
    float *a, *b, *c;
    hipMallocManaged (&a,  sizeof(float) * N);
  // ... allocate other arrays (b and c), fill with data

  // Use thread blocks with 256 threads each
    vecAdd <<< (N+255)/256, 256 >>> (a, b, c, N);
}
```

HIP kernel as function

Unified shared memory … allocate on host, visible on device too. Execute with environment variable HSA_XNACK=1

Enqueue the kernel to execute on the Grid

Note: Not all GPUs support a unified shared memory. We assume its available here so we don't need to clutter the code with explicit data movement.

Vector addition with SYCL

```
// Compute sum of length-N vectors: C = A + B
#include <CL/sycl.hpp>

int main () {
    int N = ... ;
    float *a, *b, *c;
    sycl::queue q;
    *a = (float *)sycl::malloc_shared(N * sizeof(float), q);
  // ... allocate other arrays (b and c), fill with data

        q.parallel_for(sycl::range<1>{N},
                [=](sycl::id<1> i) {
                  c[i] = a[i] + b[i];
                });
         q.wait();
}
```

Create a queue for SYCL commands

Unified shared memory … allocate on host, visible on device too

Kernel as a C++ Lambda function

[=] means capture external variables by value.

Note: Not all GPUs support a unified shared memory. We assume its available hear to we don't need to clutter the code with explicit data movement.

Vector addition with OpenACC

- Let's add two vectors together …. C = A + B

```
void vadd(int n,
          const float *a,
          const float *b,
          float *restrict c)
{
  int i;
 #pragma acc parallel loop
  for (i=0; i<n; i++)
    c[i] = a[i] + b[i];
}
int main(){
float *a, *b, *c;  int n = 10000;
// allocate and fill a and b

    vadd(n, a, b, c);

}
```

Assure the compiler that c is not aliased with other pointers

Turn the loop into a kernel, move data to a device, and launch the kernel.

Host waits here until the kernel is done. Then the output array c is copied back to the host.

Why so many ways to do the same thing?

- The parallel programming model people have failed you …
  - It's more fun to create something new in your own closed-community that work across vendors to create a portable API

- The hardware vendors have failed you …
  - Don't you love my "walled garden"? It's so nice here, programmers, just don't even think of going to some other platform since your code is not portable.

- The standards community has failed you …
  - Standards are great, but they move too slow. OpenACC stabbed OpenMP in the back and I'm pissed, but their comments at the time were spot-on (OpenMP was moving so slow … they just couldn't wait).

- The applications community failed themselves …
  - If you don't commit to a standard and use "the next cool thing" you end up with the diversity of overlapping options we have today. Think about what happened with OpenMP and MPI.

Backup ... and a bit of extra content

- A quick look at other GPU programming application APIs
- The Jacobi solver case study
- Writing functions to call from inside a kernel

Our running example: Jacobi solver

- An iterative method to solve a system of linear equations
  - Given a matrix A and a vector b find the vector x such that Ax=b
- The basic algorithm:
  - Write A as a lower triangular (L), upper triangular (U) and diagonal matrix

    $Ax = (L+D+U)x = b$
  - Carry out multiplications and rearrange

    $Dx=b-(L+U)x \rightarrow x = (b-(L+U)x)/D$
  - Iteratively compute a new x using the x from the previous iteration

    $X_{new} = (b-(L+U)x_{old})/D$
- Advantage: we can easily test if the answer is correct by multiplying our final x by A and comparing to b
- Disadvantage: It takes many iterations and only works for diagonally dominant matrices

Jacobi Solver

Iteratively update xnew until the value stabilizes (i.e. change less than a preset TOL)

```
<<< allocate and initialize the matrix A >>>
<<< and vectors x1, x2 and b             >>>

while((conv > TOL) && (iters<MAX_ITERS))
  {
    iters++;

    for (i=0; i<Ndim; i++){
        xnew[i] = (TYPE) 0.0;
        for (j=0; j<Ndim;j++){
            if(i!=j)
              xnew[i]+= A[i*Ndim + j]*xold[j];
        }
        xnew[i] = (b[i]-xnew[i])/A[i*Ndim+i];
    }
```

```
    // test convergence
    conv = 0.0;
    for (i=0; i<Ndim; i++){
        tmp  = xnew[i]-xold[i];
        conv += tmp*tmp;
    }
    conv = sqrt((double)conv);

    // swap pointers for next
    // iteration
    TYPE* tmp = xold;
    xold = xnew;
    xnew = tmp;

} // end while loop
```

Jacobi Solver (Parallel Target/loop, 1/2)

```
 while((conv > TOL) && (iters<MAX_ITERS))
   {
     iters++;
#pragma omp target map(tofrom:xnew[0:Ndim],xold[0:Ndim]) \
                   map(to:A[0:Ndim*Ndim], b[0:Ndim])
#pragma omp loop
for (i=0; i<Ndim; i++){
        xnew[i] = (TYPE) 0.0;
        for (j=0; j<Ndim;j++){
            if(i!=j)
              xnew[i]+= A[i*Ndim + j]*xold[j];
        }
        xnew[i] = (b[i]-xnew[i])/A[i*Ndim+i];
     }
```

Jacobi Solver (Parallel Target/loop, 2/2)

```
    //
    // test convergence
    //
    conv = 0.0;
#pragma omp target map(to:xnew[0:Ndim],xold[0:Ndim]) \
                   map(tofrom:conv)
#pragma omp loop private(i,tmp) reduction(+:conv)
for (i=0; i<Ndim; i++){
        tmp  = xnew[i]-xold[i];
        conv += tmp*tmp;
    }
    conv = sqrt((double)conv);
   TYPE* tmp = xold;
   xold = xnew;
   xnew = tmp;
} // end while loop
```

This worked but the performance was awful. Why?

System	Implementation	Ndim = 4096
NVIDA® K20X™ GPU	Target dir per loop	131.94 secs

Cray® XC40™ Supercomputer running Cray® Compiling Environment 8.5.3.
Intel® Xeon ® CPU E5-2697 v2 @ 2.70GHz with 32 GB DDR3. NVIDIA® Tesla® K20X, 6GB.

Data movement dominates!!!

```
while((conv > TOLERANCE) && (iters<MAX_ITERS))
  { iters++;
    xnew = iters % s ? x2 : x1;
    xold   = iters % s ? x1 : x2;

    #pragma omp target map(tofrom:xnew[0:Ndim],xold[0:Ndim]) \
                       map(to:A[0:Ndim*Ndim], b[0:Ndim] )
     #pragma omp loop private(i,j)
     for (i=0; i<Ndim; i++){
        xnew[i] = (TYPE) 0.0;
        for (j=0; j<Ndim;j++){
            if(i!=j)
              xnew[i]+= A[i*Ndim + j]*xold[j];
        }
        xnew[i] = (b[i]-xnew[i])/A[i*Ndim+i];
     }
// test convergence
     conv = 0.0;
     #pragma omp target map(to:xnew[0:Ndim],xold[0:Ndim]) \
                                map(tofrom:conv)
      #pragma loop reduction(+:conv)
      for (i=0; i<Ndim; i++){
          tmp  = xnew[i]-xold[i];
          conv += tmp*tmp;
      }
     conv = sqrt((double)conv);
  }
```

Typically over 4000 iterations!

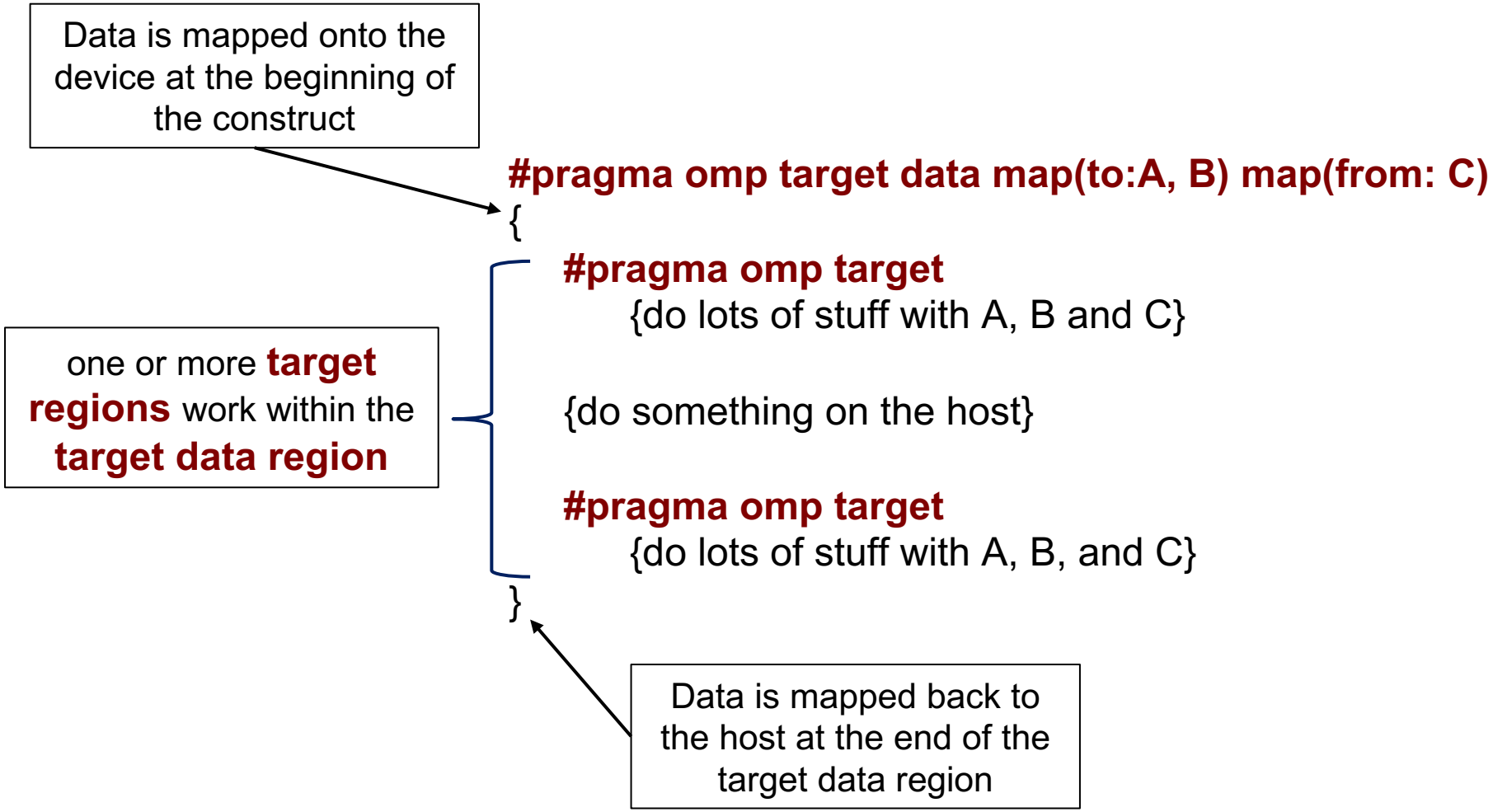
For each iteration, copy to device $(3*Ndim+Ndim^2)*sizeof(TYPE)$ bytes

For each iteration, copy from device $2*Ndim*sizeof(TYPE)$ bytes

For each iteration, copy to device $2*Ndim*sizeof(TYPE)$ bytes

Target data directive

- The **target data** construct creates a target data region
  … use **map** clauses for explicit data management



Jacobi Solver (Par Target Data, 1/2)

```
#pragma omp target data map(tofrom:xold[0:Ndim],xnew[0:Ndim]) \
                        map(to:A[0:Ndim*Ndim], b[0:Ndim] ,Ndim)
while((conv > TOL) && (iters<MAX_ITERS))
  {  iters++;

#pragma omp target
#pragma omp loop private(j) firstprivate(xnew,xold)
    for (i=0; i<Ndim; i++){
        xnew[i] = (TYPE) 0.0;
        for (j=0; j<Ndim;j++){
            if(i!=j)
              xnew[i]+= A[i*Ndim + j]*xold[j];
        }
        xnew[i] = (b[i]-xnew[i])/A[i*Ndim+i];
    }
```

Jacobi Solver (Par Target Data, 2/2)

```
// test convergence
conv = 0.0;
#pragma omp target map(tofrom: conv)
#pragma omp loop private(tmp) firstprivate(xnew,xold)  reduction(+:conv)

    for (i=0; i<Ndim; i++){
        tmp  = xnew[i]-xold[i];
        conv += tmp*tmp;
    }
// end target region
 conv = sqrt((double)conv);

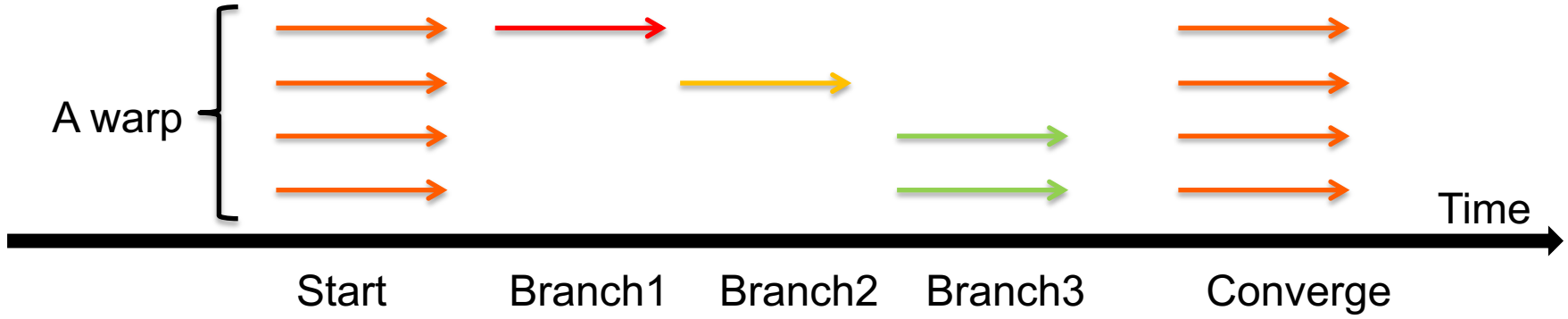
   TYPE* tmp = xold;
   xold = xnew;
   xnew = tmp;
} // end while loop
```

System	Implementation	Ndim = 4096
NVIDA® K20X™ GPU	Target dir per loop	131.94 secs
	Above plus target data region	18.37 secs

Third party names are the property of their owners.

Single Instruction Multiple Data

- Individual work-items of a warp start together at the same program address
- Each work-item has its own instruction address counter and register state
  - Each work-item is free to branch and execute independently
  - Supports the SPMD pattern.
- Branch behavior
  - Each branch will be executed serially
  - Work-items not following the current branch will be disabled



Branching

Conditional execution

```
// Only evaluate expression
// if condition is met
if (a > b)
{
  acc += (a - b*c);
}
```

Selection and masking

```
// Always evaluate expression
// and mask result
temp = (a - b*c);
mask = (a > b ? 1.f : 0.f);
acc += (mask * temp);
```

Coalescence

- Coalesce - to combine into one
- Coalesced memory accesses are key for high bandwidth
- Simply, it means, if thread *i* accesses memory location *n* then thread *i+1* accesses memory location *n+1*
- In practice, it's not quite as strict…

```
for (int id = 0; id < size; id++)
{
  // ideal
     float val1 = memA[id];

  // still pretty good
     const int c = 3;
     float val2 = memA[id + c];

  // stride size is not so good
     float val3 = memA[c*id];

  // terrible
     const int loc =
       some_strange_func(id);

     float val4 = memA[loc];
}
```

Jacobi Solver (Target Data/branchless/coalesced mem, 1/2)

```
#pragma omp target data map(tofrom:x1[0:Ndim],x2[0:Ndim]) \
                        map(to:A[0:Ndim*Ndim], b[0:Ndim] ,Ndim)
while((conv > TOL) && (iters<MAX_ITERS))
  {  iters++;
#pragma omp target
     #pragma omp loop private(j)
    for (i=0; i<Ndim; i++){
       xnew[i] = (TYPE) 0.0;
       for (j=0; j<Ndim;j++){
            xnew[i]+= (A[j*Ndim + i]*xold[j])*((TYPE)(i != j));
       }
       xnew[i] = (b[i]-xnew[i])/A[i*Ndim+i];
    }
```

We replaced the original code with a poor memory access pattern

```
xnew[i]+= (A[i*Ndim + j]*xold[j])
```

With the more efficient

```
xnew[i]+= (A[j*Ndim + i]*xold[j])
```

Jacobi Solver (Target Data/branchless/coalesced mem, 2/2)

```
      //
      // test convergence
      conv = 0.0;
#pragma omp target map(tofrom: conv)
 #pragma omp loop private(tmp) reduction(+:conv)
      for (i=0; i<Ndim; i++){
          tmp  = xnew[i]-xold[i];
          conv += tmp*tmp;
      }
conv = sqrt((double)conv);
    TYPE* tmp = xold;
    xold = xnew;
    xnew = tmp;
} // end while loop
```

System	Implementation	Ndim = 4096
NVIDA® K20X™ GPU	Target dir per loop	131.94 secs
	Above plus target data region	18.37 secs
	Above plus reduced branching	13.74 secs
	Above plus improved mem access	7.64 secs

Cray® XC40™ Supercomputer running Cray® Compiling Environment 8.5.3.
Intel® Xeon ® CPU E5-2697 v2 @ 2.70GHz with 32 GB DDR3. NVIDIA® Tesla® K20X, 6GB.

Third party names are the property of their owners.

A more complicated example:
Jacobi iteration: OpenACC (GPU)

```
#pragma acc data copy(A), create(Anew)
while (err>tol && iter < iter_max){
   err = 0.0;
   #pragma acc parallel loop reduction(max:err)
   for(int j=1; j< n-1; j++){
      for(int i=1; i<M-1; i++){
         Anew[j][i] = 0.25* (A[j][i+1] + A[j][i-1]+
                             A[j-1][i] + A[j+1][i]);
         err = max(err,abs(Anew[j][i] - A[j][i]));
       }
    }
    #pragma acc parallel loop
    for(int j=1; j< n-1; j++){
      for(int i=1; i<M-1; i++){
         A[j][i] = Anew[j]i];
       }
    }
    iter ++;
}
```

Create a data region on the GPU. Copy A once onto the GPU, and create Anew on the device (no copy from host)

Copy A back out to host … but only once

Source: based on Mark Harris of NVIDIA®, “Getting Started with OpenACC”, GPU technology Conf., 2012

A more complicated example:

Jacobi iteration: OpenMP target directives

```
#pragma omp target data map(A) map(alloc:Anew)
while (err>tol && iter < iter_max){
   err = 0.0;
   #pragma target
   #pragma omp teams loop reduction(max:err)
   for(int j=1; j< n-1; j++){
      for(int i=1; i<M-1; i++){
         Anew[j][i] = 0.25* (A[j][i+1] + A[j][i-1]+
                             A[j-1][i] + A[j+1][i]);
         err = max(err,abs(Anew[j][i] - A[j][i]));
       }
    }
    #pragma omp target
    #pragma omp teams loop
    for(int j=1; j< n-1; j++){
      for(int i=1; i<M-1; i++){
         A[j][i] = Anew[j]i];
       }
    }
    iter ++;
}
```

Create a data region on the GPU. Map A and Anew onto the target device

Copy A back out to host … but only once

Backup ... and a bit of extra content

- A quick look at other GPU programming application APIs
- The Jacobi solver case study
- Writing functions to call from inside a kernel

Defining a function to be called from inside a kernel

Tell OpenMP to compile this function for the GPU (and the CPU)

Call inside a target region, and the GPU version is called.

Call on the host, and a CPU version is called

```
#include mm_utils.h

#pragma omp declare target
void ddot(double *C, double *A, double *B, int i, int j, int Mdim, int Pdim){
   for(int k=0;k<Pdim;k++){
      /* C(i,j) = sum(over k) A(i,k) * B(k,j) */
      *(C+(i*Mdim+j)) += *(A+(i*Pdim+k)) *  *(B+(k*Mdim+j));
   }
}
#pragma omp end declare target


void mm_gpu(int Ndim, int Mdim, int Pdim, TYPE *A, TYPE *B, TYPE *C){
  int i, j, k;

#pragma omp target teams map(tofrom:C[0:Ndim*Mdim]) map(to:B[0:Pdim*Mdim],A[0:Ndim*Pdim])
#pragma omp loop  collapse(2)
  for (i=0; i<Ndim; i++){
     for (j=0; j<Mdim; j++){
        ddot(C, A, B, i, j, Mdim, Pdim);

     }
```

This is in my file mm_gpu.c